SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 01

Software Engineering Framework Fundamentals, Selection and Emerging Trends

Year 3 • Semester 1 • 4 Credits

Lecturer: Eishan Weerasinghe | eishan.w@sliit.lk | Course web: courseweb.sliit.lk

Module Delivery Team

Use the correct contact point for module coordination, lecture content, lab support, assessments and special lecture areas.



LIC

Lecture-in-Charge

Eishan Weerasinghe | eishan.w@sliit.lk



VL

Visiting Lecturer

Mr. Lasal Hettiarachchi | lasal.h.visiting@sliit.lk

Official Channels Course Web for announcements and submissions • Email for individual clarification

Teaching & Lab Delivery Team

Centre lecture conductors and module lab conductors for SE3090.

Centre Lecture Conductors

Centre / Group	Lecture Conductor	Email
KANDY UNI (WD/WE)	Ms. Kaushalya Premarathne	kaushalya.pr@kandyuni.lk
NORTHERN UNI (WE)	Ms. A. Sangeetha	sangeetha.b@northernuni.lk

Module Lab Conductors

	Lab Conductor Name(s)	
Ms. Madusha Sulakshi Weerasooriya	Mr. Janidu Illesinghe	Mr. Manuka Rashen
Ms. Piumi Navoda	Ms. Shehani Dehipola	Ms. Hansi De Silva

Lab delivery *Lab conductors support practical completion, environment setup and CourseWeb submission guidance.*

About This Module

- Aim: understand modern frameworks used to build, integrate, deploy and maintain software systems
- Hands-on focus: full-stack web + mobile apps, DevOps and cloud deployment, quality and security
- Responsible use of agentic AI tools in real development workflows

Delivery	Hours	What happens there
Lectures	3 h / week	Concepts, demos, case studies, framework decisions
Laboratories	2 h / week	Guided practicals: React, ASP.NET Core, Flutter, CI/CD, AI tools
Independent study	~80 h / semester	Reading, practice, assignment work

How You Will Be Assessed

10%	15%	25%	50%
Lab Submission	**Mini Hackathon**	**Main Assignment**	**Final Exam**
Continuous lab tasks (start this week)	Participation + Evaluation	Full-stack application with agentic AI integration	Covers all lectures (LO1–LO4)

- Pass requirement: 45% in continuous assessments AND 45% in the final examination
- The main assignment is your chance to apply every framework taught in this module to one product

The Module Journey – 11 Lectures

Foundations	Web Frameworks	Agentic AI	Mobile Frameworks	Quality & Delivery
L01 Framework fundamentals, selection & trends	L02 Advanced React frontend L03 C#, .NET & ASP.NET Core REST APIs L04 Databases, auth & integration	L05–L07 Agentic software development 1–3	L08 Flutter & mobile app development L09 Flutter UI, device features, data	L10 Git, CI/CD, security, code quality L11 DevOps & cloud hosting

Today: the map of the whole territory — every later lecture zooms into one region.

Learning Outcomes for This Lecture

LO1 — Evaluate frameworks

Evaluate different types of software engineering frameworks used in web, mobile, cloud and AI-assisted software development.

LO4 — Select and justify

Select and justify appropriate frameworks, tools and agentic AI-assisted approaches to meet specific project and industry requirements.

By the end of today you should be able to: define what a framework is, distinguish frameworks from libraries, classify common framework types, apply selection criteria to a project scenario, and describe the major emerging trends.

Why Frameworks Matter in Real Software Engineering

Speed to market

Instagram's first version shipped in ~8 weeks on Django. Frameworks remove months of plumbing work.

Built-in quality & security

Mature frameworks ship tested solutions for auth, validation and security — fewer DIY vulnerabilities.

Team scalability

Shared conventions let new developers contribute fast. Industry hires by framework skill: React, .NET, Flutter.

Career reality

Almost every job ad you will apply to names frameworks, not just languages. This module builds that profile.

Today's Roadmap – 3 Hours

Hour 1	**Framework fundamentals** Definitions • why frameworks exist • benefits & costs • frameworks vs libraries • checkpoint
Hour 2	**Types, industry use & selection** Framework landscape • frontend/backend/mobile/testing • industry case examples • selection criteria & workflow
Hour 3	**Trends, case study & wrap-up** Microservices • serverless • cloud-native • DevOps • AI-assisted development • case study + group activity • quiz & summary

Short breaks after each hour • Questions welcome at any point

PART 1 OF 4

Framework Fundamentals

- What a software engineering framework is
- The problems frameworks solve
- Benefits — and the costs nobody advertises
- Frameworks vs libraries (a classic exam question)



What Is a Software Engineering Framework?

Definition: A reusable, semi-complete software platform that provides a standard structure, common services and extension points on which developers build applications.

- **Skeleton of an application — architecture, plumbing and conventions are already in place**
  - You write the parts that make YOUR application unique; the framework runs them
  - Provides ready-made services: routing, data access, UI rendering, security, configuration
  - Examples: React (UI), ASP.NET Core (backend), Flutter (mobile), Spring Boot (Java backend)
- Analogy: a framework is the building's steel frame and utilities — you design the rooms

The Problems Frameworks Solve

Without a framework

- ✕ Re-implement routing, auth, validation, data access for every project
- ✕ Every developer invents a different structure — chaos at handover
- ✕ Security written from scratch → untested, vulnerable
- ✕ Slow starts: weeks of plumbing before the first feature

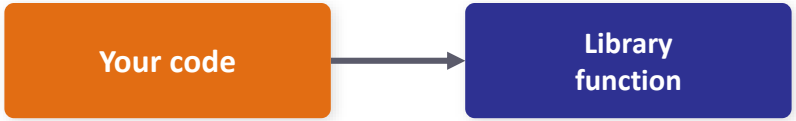
With a framework

- ✓ Common concerns solved once, by experts, and battle-tested
- ✓ One shared structure and naming convention for the whole team
- ✓ Security patches arrive as framework updates
- ✓ Day 1 productivity: scaffold an app in minutes, focus on features

“Don't reinvent the wheel — and definitely don't reinvent the security wheel.”

Inversion of Control – “Don't Call Us, We'll Call You”

Library: YOUR code is in control



You decide when to call lodash, axios, math utils…

Framework: the FRAMEWORK is in control



ASP.NET Core receives the request, then calls your controller. React decides when your component re-renders.

The Hollywood Principle: you plug your code into the framework's extension points; the framework's main loop decides when it runs. This single idea is the cleanest test for 'framework or library?'

Benefits of Using Frameworks

Productivity

Scaffolding, generators, hot reload — features ship faster

Reliability

Battle-tested code paths used by millions of apps

Security

Vetted auth, validation & patching out of the box

Maintainability

Conventions make code predictable and refactorable

Community & talent

Docs, tutorials, Stack Overflow, hireable skills

Best practices built in

MVC, components, DI — good architecture by default

The Costs Nobody Advertises

- Learning curve — Angular or Spring can take weeks before a team is productive
- Lock-in — switching frameworks later usually means a rewrite, not a refactor
- Overhead — abstraction layers can cost performance and bundle size
- “Magic” — hidden behaviour makes debugging harder when conventions are misunderstood
- Upgrade churn — major versions can break code (Angular 1→2, Python 2→3 style migrations)
- Over-engineering risk — a full framework for a tiny script is wasted complexity

Engineering judgement: a framework is an investment — pay the learning and lock-in cost only when the benefits outweigh it.

Frameworks vs Libraries

Aspect	Library	Framework
Control flow	Your code calls the library	Framework calls your code (IoC)
Scope	One focused task (HTTP, dates, charts)	Whole application structure
Architecture	You design it	Provided — conventions & structure
Flexibility	Mix and match freely	Commit to its rules & ecosystem
Replacement cost	Usually easy to swap	Expensive — often a rewrite
Examples	lodash, axios, NumPy, jQuery	ASP.NET Core, Angular, Flutter, Spring Boot

The decisive test: who is in control of the program's flow?

The Grey Zone – Where Do These Belong?

React	Calls itself a UI library — but its lifecycle controls your components, and React + Router + state libraries form a de-facto framework
jQuery	Pure library: you call $() whenever you choose; it never calls you
Express.js	Minimal web framework: it owns the request loop and calls your route handlers — even though it feels 'library-light'
Next.js / Angular / Flutter	Full frameworks: routing, build system, conventions, lifecycle — the platform is in charge

Checkpoint: is the .NET Base Class Library a framework or a library? Defend your answer.

Checkpoint 1 – Quick Thinking

1. In one sentence, what does Inversion of Control mean? Give one example from a tool you have used.

2. Your friend says: “I imported axios into my project, so I'm using a framework.” Correct or not? Why?

3. Name two benefits AND two costs of adopting a large framework like Angular for a 3-month student project.

Pair up — 3 minutes — then we discuss answers together.

PART 2 OF 4

Types of Frameworks & Industry Use

- The framework landscape — six major families
- Frontend, backend, mobile and testing frameworks
- How a full-stack architecture fits together
- Who uses what in industry — real examples



The Framework and Tool Landscape



Frontend (Web UI)

React • Angular • Vue • Next.js



Backend / Server-side

ASP.NET Core • Spring Boot • Express • Django



Mobile

Flutter • React Native • SwiftUI • Jetpack Compose



Testing & Quality

xUnit • JUnit • Jest • Selenium • Playwright



DevOps / CI-CD / Cloud

GitHub Actions • Docker • Kubernetes • Terraform



AI-assisted & Agentic

GitHub Copilot • Claude Code • LangChain • agents

Frontend Frameworks – Building Modern UIs

- Shared ideas: components, declarative UI, state management, client-side routing, virtual/reactive rendering

	React	Angular	Vue
Type	UI library + ecosystem	Full framework (batteries included)	Progressive framework
Language	JavaScript / TypeScript (JSX)	TypeScript	JavaScript / TypeScript
Best fit	Flexible SPAs, huge talent pool	Large enterprise apps, strict structure	Gentle learning curve, lightweight apps
Used by	Meta, Netflix, Airbnb	Google, Microsoft Office web	Alibaba, GitLab

This module: React (Lecture 02) — the most demanded frontend skill in the job market.

Backend Frameworks – APIs and Business Logic

- Role: expose REST APIs, run business rules, talk to databases, enforce security

Framework	Language	Strengths	Typical adopters
ASP.NET Core	C#	High performance, DI built in, enterprise tooling, cross-platform	Stack Overflow, banks, Microsoft
Spring Boot	Java	Mature ecosystem, microservices support	Netflix backend, enterprises
Express.js / NestJS	JavaScript / TS	Minimal & fast to start; same language as frontend	Uber, PayPal (Node)
Django / FastAPI	Python	Rapid development; FastAPI popular for AI/ML APIs	Instagram (Django)

This module: C# + ASP.NET Core (Lectures 03–04) — controllers, services, REST APIs, auth, database integration.

Mobile Frameworks – One Codebase or Two?

Native

- SwiftUI (iOS) • Jetpack Compose (Android)
- Best performance & full device API access
- Two codebases → double the team & cost

Cross-platform

- Flutter (Dart) • React Native (JS/TS)
- One codebase → iOS + Android (+ web/desktop)
- Flutter renders its own UI — consistent look, near-native speed

Industry examples: Google Pay & BMW apps use Flutter; Instagram and Shopify use React Native. Cross-platform now dominates startup and SME development.

This module: Flutter (Lectures 05–06).

Testing & Quality Frameworks

Level	Purpose	Frameworks / tools
Unit testing	Test one class/function in isolation	xUnit & NUnit (.NET) • JUnit (Java) • Jest (JS)
Integration / API	Test components working together	Postman/Newman • REST-assured • Supertest
End-to-end (UI)	Test full user journeys in a browser/app	Selenium • Playwright • Cypress • Flutter integration tests
Static quality	Analyse code without running it	SonarQube • ESLint • Roslyn analyzers

Why they are frameworks: the test runner discovers and calls YOUR test methods — Inversion of Control again. These tools plug into CI/CD pipelines (Lecture 07) so every commit is verified automatically.

How It All Fits – Full-Stack Architecture



Delivery layer: Git + GitHub → GitHub Actions CI/CD → Docker → Cloud hosting (Azure / AWS / Vercel)

AI-assisted tools (Copilot, Claude Code) accelerate every box in this diagram — Lectures 09–11.

Frameworks in Industry – Who Uses What

Company / product	Frameworks in production	Why it works for them
Netflix	React (UI) + Spring Boot microservices	Independent teams ship services on their own schedules
Stack Overflow	ASP.NET Core + SQL Server	Massive traffic on a famously small server footprint
Google Pay, BMW, Toyota	Flutter	One team, one codebase, consistent UX on both platforms
Airbnb, Meta	React (+ React Native history)	Reusable components across a very large product surface
Instagram	Django (Python)	Rapid iteration from startup to billions of users

Pattern: companies choose frameworks that match their scale, team structure and speed of change — not the newest or trendiest option.

Checkpoint 2 – Spot the Stack



- **Think of an app you used today (PickMe, WhatsApp, your bank, Daraz…)**
- **In pairs (4 minutes), sketch its likely architecture:**
  - Which frontend framework family? Web, mobile, or both?
  - What does the backend need to do? Which framework family fits?
  - Where is the data stored? What about payments or maps — frameworks or libraries?
- **Two pairs will present their guess — we'll critique it together as a class**

Goal: practise seeing every product as a set of framework decisions.

PART 3 OF 4

Framework Selection

- Selection criteria that professionals actually use
- A repeatable selection workflow
- Scoring candidates objectively
- Case study + your turn: a group selection activity



Framework Selection Criteria

Project fit

Does it solve THIS problem class well? (real-time, CRUD, mobile…)

Team skills

Current expertise beats theoretical superiority

Community & ecosystem

Docs, packages, Stack Overflow answers, hiring pool

Performance & scalability

Benchmarks under YOUR expected load, not marketing

Maturity & longevity

Release history, corporate backing, LTS versions

Security & compliance

Patch cadence, CVE history, audit requirements

Cost & licensing

Open-source licence terms, hosting cost, paid tiers

Integration

Plays well with your other frameworks, DBs, cloud

A Repeatable Selection Workflow



1. Requirements	2. Constraints	3. Shortlist	4. Prototype	5. Decide & record
Functional + non-functional (load, security, offline…)	Team skills, budget, deadline, existing systems	2–3 candidates per layer that fit the constraints	Build a small spike / proof-of-concept in each	Score, choose, document the rationale (ADR)

Architecture Decision Record (ADR): a one-page note capturing context, options considered, decision and consequences. Future maintainers (and your markers!) will thank you. You will write one for the main assignment.

Scoring Candidates – Weighted Decision Matrix

Example: choosing a frontend framework for a startup MVP (scores 1–5)

Criterion	Weight	React	Angular	Vue
Team familiarity	30%	4	2	3
Time to MVP	25%	4	3	5
Hiring pool (LK + remote)	20%	5	3	3
Ecosystem for our features	15%	5	4	4
Long-term maintainability	10%	4	5	4
Weighted total	**100%**	**4.35 ✓**	**3.05**	**3.85**

Weights encode YOUR context — a bank would weight security and maintainability, flipping the result.

When Framework Choices Go Wrong



Hype-driven choice

Adopting the newest framework for a critical system → immature ecosystem, breaking changes every month



Ignoring team skills

A Java team forced onto a Node stack → 6 months of slow, buggy delivery while 'learning on the job'



Longevity blindness

Building on AngularJS near its end-of-life (support ended 2022) → forced rewrite with zero new features



Over-engineering

Kubernetes + microservices for an app with 200 users → the team maintains infrastructure instead of features

Every failure above traces back to skipping a step in the selection workflow.

Case Study – “CampusEats” Food Delivery Startup

Brief: 3 developers, 4 months, limited budget. Needs: student-facing mobile app, restaurant web dashboard, orders + payments + live tracking, must scale if it succeeds.

Layer	Choice	Justification (criteria applied)
Mobile app	Flutter	One codebase for iOS + Android; 3-person team can't staff two native tracks
Web dashboard	React	Huge ecosystem (tables, charts); largest hiring pool when the team grows
Backend API	ASP.NET Core	One shared REST API for both clients; performance headroom; built-in auth & DI
Database	PostgreSQL / SQL Server	Relational fit for orders & payments; managed cloud hosting available
Delivery	GitHub Actions + Docker + Azure	Free tier CI/CD; containers keep dev = production

Notice: every choice cites a criterion, not a preference. This is the standard your assignment ADR must meet.

Group Activity – You Are the Tech Lead

- **Form groups of 4 — 12 minutes**
- Your scenario (pick one):
  - A) Hospital appointment system for a private hospital chain (security & compliance heavy)
  - B) Real-time exam-hall seat finder for SLIIT (3-week deadline, web only)
  - C) E-commerce app for a clothing brand (mobile-first, must scale for sales seasons)
- Deliverable: stack choice for frontend / mobile / backend / database / hosting + top 3 criteria you weighted and WHY
- **Two groups present (2 minutes each); the class challenges one decision per group**

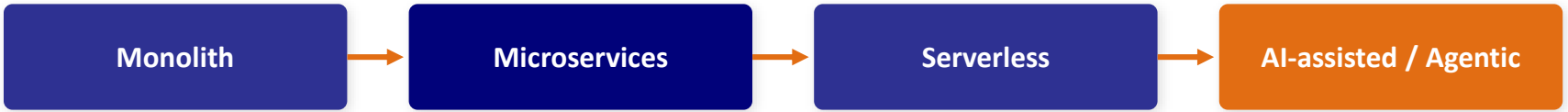
PART 4 OF 4

Emerging Trends in Frameworks

- From monoliths to microservices and serverless
- Cloud-native development and DevOps automation
- AI-assisted and agentic software development
- What these trends mean for YOUR framework choices



How Application Architecture Evolved



One deployable unit
Simple to start,
hard to scale teams

Many small services
Independent deploys,
operational complexity

Functions on demand
No servers to manage,
pay per execution

AI generates, tests
and reviews code with
human oversight

Key insight: newer ≠ better for every project. Each era added options, not replacements — monoliths still power excellent products (Stack Overflow!).

Trend 1 – Microservices Frameworks

- Idea: split the application into small, independently deployable services, each owning its data
  - Frameworks & tools: Spring Boot + Spring Cloud, ASP.NET Core microservices, Docker, Kubernetes for orchestration
  - Supporting patterns: API gateway, service discovery, message queues (RabbitMQ, Kafka)
- Wins: team autonomy, independent scaling, fault isolation, polyglot freedom
- Costs: distributed-system complexity, network failures, monitoring overhead, harder local dev
- Use when: large teams, different scaling needs per component — Netflix, Amazon, Uber scale

Rule of thumb: don't adopt microservices until a well-built monolith actually hurts.

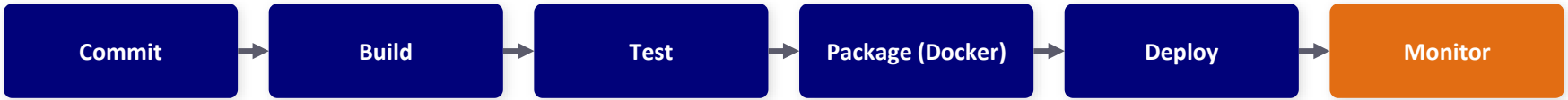
Trend 2 – Serverless Computing

- Idea: deploy functions, not servers — the cloud runs, scales and bills them per execution
  - Platforms: AWS Lambda, Azure Functions, Google Cloud Functions, Cloudflare Workers
  - Frameworks: Serverless Framework, AWS SAM/Amplify — define functions + triggers as code
- Great for: event-driven jobs (image resize, notifications), APIs with spiky traffic, scheduled tasks
- Watch out for: cold starts, execution time limits, vendor lock-in, tricky local testing
- Example: a CampusEats 'send order confirmation email' function — runs only when orders happen, costs cents

Serverless complements, not replaces, your main API — most real systems are hybrids.

Trend 3 – Cloud-Native & DevOps Automation

- Cloud-native: applications designed FOR the cloud — containers, declarative config, resilience, observability
  - Containers & orchestration: Docker packages the app + environment; Kubernetes runs it at scale
  - Infrastructure as Code: Terraform / Bicep — your servers are version-controlled text
- DevOps: culture + automation joining development and operations
  - CI/CD frameworks: GitHub Actions, GitLab CI, Jenkins, Azure DevOps



A push to GitHub can reach production in minutes — automatically tested at every step (Lectures 07–08).

Trend 4 – AI-Assisted & Agentic Development

AI-assisted coding

Copilots suggest code, tests and docs inside the IDE. Developer stays the author. Tools: GitHub Copilot, Claude, IDE assistants.

Agentic development

AI agents plan multi-step tasks: read the codebase, edit files, run tests, open pull requests. Tools: Claude Code, Copilot agents; frameworks like LangChain build custom agents.

Human-in-the-loop is non-negotiable:
engineers must review, test and take responsibility for AI-generated code. Risks: subtle bugs, security flaws, licence issues, over-reliance. Your value shifts toward requirements, architecture, evaluation — exactly LO1 and LO4.

What These Trends Mean for Your Choices

- Frameworks now compete on cloud & AI readiness — containers, serverless targets, AI tooling support
- Deployment is part of the stack decision: a framework without good CI/CD and hosting story costs you later
- Prefer frameworks with strong conventions — AI tools generate better code for well-documented, conventional frameworks
- Architecture flexibility matters: choose stacks that can start monolith and split into services later
- Evergreen skill: the evaluation method survives every trend — criteria, trade-offs, evidence, decision records

Frameworks will change every five years. Your selection method shouldn't.

Quick Quiz – Test Yourself

Q1. The defining difference between a framework and a library is: (a) size (b) language (c) who controls the flow of execution (d) licence

Q2. “Don't call us, we'll call you” describes: (a) REST (b) Inversion of Control (c) CI/CD (d) serverless

Q3. A 3-person startup needs iOS + Android apps in 4 months. The strongest first candidate is: (a) two native apps (b) Flutter (c) desktop app (d) static website

Q4. Which is the BEST reason to pick framework X? (a) trending on social media (b) used by Google (c) highest weighted score on your project's criteria (d) newest release

Q5. In agentic AI development, the human's essential role is: (a) typing faster (b) reviewing, testing & taking responsibility (c) avoiding documentation (d) none

Answer individually first — then we vote by show of hands.

Quiz Answers & Why

Q	Answer	Why
1	(c)	Control flow defines the relationship — IoC is the framework's signature
2	(b)	The Hollywood Principle is the classic phrasing of Inversion of Control
3	(b)	One codebase fits the team size and deadline; native doubles the work
4	(c)	Selection must trace to weighted project criteria, not popularity or novelty
5	(b)	Human-in-the-loop review and accountability are non-negotiable

Scored 4–5? You've met today's outcomes. Below 3? Revisit Parts 1 and 3 before next week.

Discussion – Take a Position

1. “In five years, AI agents will choose our frameworks for us.” Agree or disagree — what part of selection is hardest to automate?

2. Should a university teach the MOST POPULAR frameworks or the BEST-DESIGNED ones? Are they the same thing?

3. Your company's 8-year-old framework is stable but unfashionable, and juniors don't want to learn it. Rewrite, wrap, or stay? Defend your answer with criteria.

No marks, no wrong answers — only undefended ones.

Summary – What You Should Remember

- **A framework is a semi-complete application: it provides structure and calls YOUR code (Inversion of Control)**
- Frameworks trade learning curve and lock-in for productivity, quality, security and team scalability
- Library vs framework = who controls the flow of execution
- Six framework families: frontend, backend, mobile, testing, DevOps/cloud, AI-assisted — real systems combine them
- Selection is a method: requirements → constraints → shortlist → prototype → decide & record (ADR + weighted matrix)
- **Trends (microservices, serverless, cloud-native, agentic AI) change the options — never the method**

Next Week & How to Prepare

Lecture 02 — Advanced React Frontend Development: components, props & state, hooks, events, forms & validation, routing.

- **Before next week:**
  - Install Node.js (LTS) and VS Code; verify with `node -v` and `npm -v`
  - Refresh JavaScript ES6: arrow functions, destructuring, map/filter, modules
  - Skim: Banks & Porcello, Modern Patterns for Developing React Apps — Ch. 1–2
- **Lab connection:**
  - Lab 01 this week: environment setup + scaffold and explore your first React app (counts toward the 10% lab submission)

Learning Outcome Mapping & References

Lecture section	LO1 Evaluate	LO4 Select & justify
Fundamentals, benefits & costs, frameworks vs libraries	●	
Framework types, architecture & industry use	●	○
Selection criteria, workflow, matrix, case study & activity	○	●
Emerging trends (microservices, serverless, cloud-native, agentic AI)	●	●

References

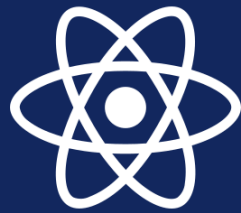
- Banks, A. & Porcello, E. (2020) Modern Patterns for Developing React Apps. 2nd edn. O'Reilly.
- Rose, R. (2022) Flutter and Dart Cookbook. O'Reilly.
- Ford, N., Richards, M., Sadalage, P. & Dehghani, Z. (2021) Software Architecture: The Hard Parts. O'Reilly.
- Biswas, A. & Talukdar, W. (2025) Building Agentic AI Systems. Packt.
- Official docs: react.dev • learn.microsoft.com/aspnet/core • docs.flutter.dev • docs.github.com/actions

SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 02

Advanced React Frontend Development



Year 3 • Semester 1 • 4 Credits

Lecturer: Eishan Weerasinghe | eishan.w@sliit.lk | Course web: courseweb.sliit.lk

Where This Lecture Fits in SE3090

L01	TODAY	L03	L04	Later
Framework fundamentals & trends	Advanced React frontend	C#, .NET & REST API backend	Database, AuthN/AuthZ & integration	Agentic dev & DevOps

- **Module content area: modern frontend frameworks, architecture and production practices**
- Builds on React basics from prior modules — today is the advanced, real-world layer
- The frontend you design today will call the REST APIs you build in Lecture 03 and 04
- Directly assessed in labs, the mini hackathon and the Main Assignment's frontend (UI/UX + code quality)

Lecture Learning Outcomes

LO1

Understand modern framework concepts

Explain component-based architecture, the React ecosystem and React's place in full-stack development.

LO2

Apply suitable frameworks & tools

Build production-grade React: feature structure, reusable components, custom hooks, forms, routing and data fetching.

LO3

Use best practices for quality

Apply separation of concerns, validation, server-state management, protected routes and performance optimization.

LO4

Select & justify approaches

Choose between Context, Redux Toolkit, Zustand, TanStack Query and RTK Query based on requirements.

Why Advanced React Development Matters

Basics don't scale

useState + fetch in every component works for a demo. At 50 components and 5 developers it becomes unmaintainable spaghetti.

Server state is different

Most app data lives on a server. Treating it like local state causes stale data, race conditions and duplicate requests.

Users feel performance

Unnecessary re-renders, huge bundles and unoptimized lists make apps janky. Real users abandon slow pages.

Real apps need structure & security

Roles, protected routes, validation and clean architecture are expected in industry — and graded in your assignment.

Lecture Roadmap – 3 Hours

1. **Modern React & production architecture** — ~30 min
   Components · ecosystem · feature-based structure · separation of concerns

2. **Component patterns & custom hooks** — ~40 min
   Composition · compound · controlled · headless · useFetch/useDebounce/useAuth …

3. **Advanced forms & validation** — ~30 min
   React Hook Form · Zod/Yup · dynamic fields · error handling

4. **API integration, server state & routing** — ~45 min
   Axios · loading/error · TanStack & RTK Query · nested/protected/lazy routes

5. **Performance & state-management decisions** — ~35 min
   Memoization · code splitting · virtualization · choosing the right tool

PART 1

Modern React & Production-Level Architecture

- Role of React & component-based architecture
- React in full-stack development & the ecosystem
- Feature-based folder structure
- Separation of UI, logic, services and hooks

The Role of React in Modern Frontend Development

- **A JavaScript library for building user interfaces from components**
  - Declarative: describe WHAT the UI should look like for a given state
  - React keeps the DOM in sync with your data via a virtual DOM diff
- **Component-based: build small pieces, compose them into screens**
- **Unidirectional data flow: state flows down, events flow up**
- **Huge ecosystem: routing, data, forms, UI kits, meta-frameworks (Next.js)**
- **Backed by Meta + a massive community — a safe industry bet**

Declarative in one line

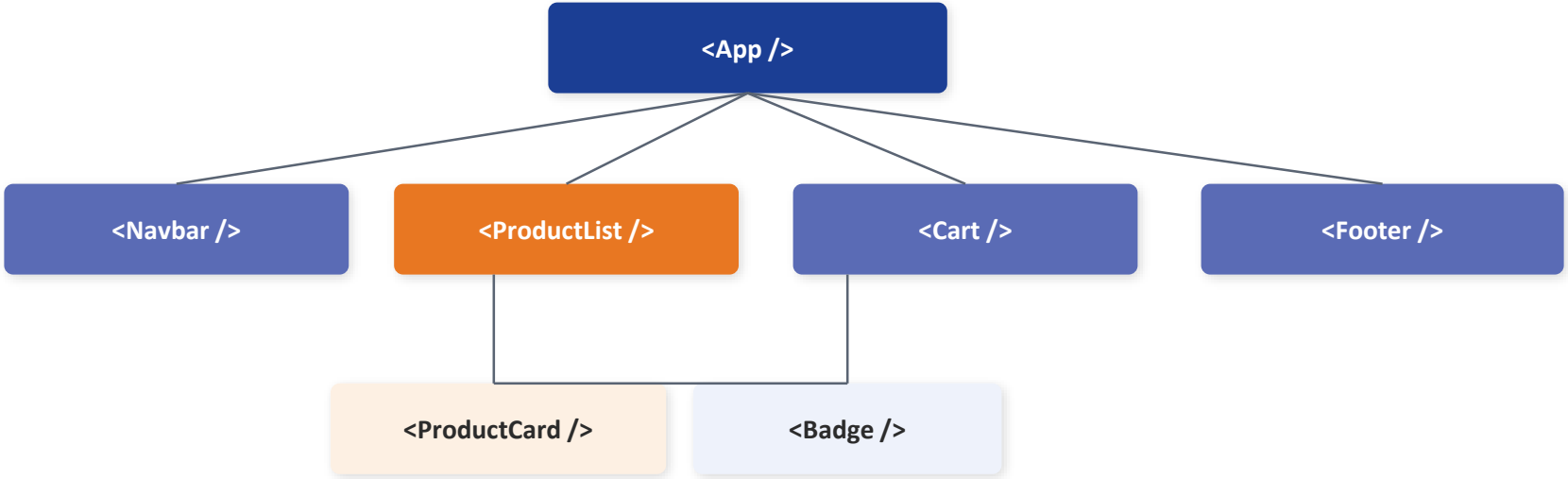
`UI = f(state)`

Change the state → React re-renders the right parts. You never touch the DOM by hand.

a component

```
function Greeting({ name }) {
  return <h1>Hi {name}!</h1>;
}
<Greeting name="Amal" />
```

Component-Based Architecture



- **Break a UI into a tree of independent, reusable components**
  - Each component owns its markup, styling and local behaviour
  - Compose small components (Badge, Card) into bigger ones (List, Page)
  - Reuse the same component with different props across the app

React in Full-Stack Application Development



- **React is the CLIENT tier — it renders UI and talks to the backend over HTTP**
- It sends requests (GET/POST/…) and receives JSON, then updates the UI
- It never touches the database directly — always through the API
- Today: the frontend half. Lectures 03–04: the API and database it consumes

The React Ecosystem & Industry Usage

Routing

React Router · TanStack Router · Next.js App Router

Server state

TanStack Query · RTK Query · SWR

Client state

Context · Redux Toolkit · Zustand · Jotai

Forms

React Hook Form · Formik + Zod / Yup

UI components

MUI · shadcn/ui · Chakra · Ant Design

Meta-frameworks

Next.js · Remix — SSR, routing, fullstack

Production Architecture: Feature-Based Structure

feature-based src/

```
src/
 ├─ components/        // shared, reusable UI
 │    ├─ Button.jsx  Modal.jsx  Table.jsx
 ├─ layouts/          // page shells
 │    └─ DashboardLayout.jsx
 ├─ features/         // group by DOMAIN
 │    ├─ products/
 │    │    ├─ components/  ProductCard.jsx
 │    │    ├─ hooks/       useProducts.js
 │    │    ├─ services/    productApi.js
 │    │    └─ pages/       ProductListPage.jsx
 │    └─ auth/  cart/  orders/ …
 ├─ hooks/            // global custom hooks
 ├─ lib/ services/    // api client, utils
 └─ routes/           // route config
```

Feature > file-type

- Group by DOMAIN (products, auth), not by type (all components together)
- Everything for a feature lives in one folder — easy to find & delete
- Truly shared UI → top-level components/
- Scales to large teams: features stay independent
- Avoid the 'giant components/ + giant pages/' anti-pattern

Separation of UI, Logic, Services & Hooks

UI components

Pure presentation: receive props, render markup. No fetch, minimal logic.

Custom hooks

Reusable LOGIC & state: useProducts(), useForm(). UI-free, testable.

Services / API layer

All HTTP lives here: productApi.getAll(). One place to change endpoints.

Server / data

The backend & database (Lectures 03–04).

Why separate?

- Test logic without rendering UI
- Swap the API client without touching components
- Reuse a hook across many components
- Components stay small & readable
- Each layer: one reason to change

A Vocabulary of Components

Page components	One per route; compose layout + features. Own data fetching & orchestration.	*ProductsPage, CheckoutPage*
Layout components	Shared shell: header, sidebar, footer. Wrap pages via layout routes.	*DashboardLayout, AuthLayout*
Feature components	A slice of business UI; may hold local state and call hooks.	*ProductCard, CartSummary*
Reusable UI primitives	Generic, domain-free building blocks. The design-system layer.	*Button, Modal, Table, Input*

Rule of thumb: the lower the layer, the more reusable and the less it should know about your domain.

Checkpoint 01 – Quick Questions

Answer with your neighbour — 4 minutes

Q1 In a feature-based structure, where do the files for the 'orders' feature live — and what goes in its folder?

Q2 A Button component imports the Order model and formats currency. What's wrong with that, and how would you fix it?

Q3 Which layer should contain an axios call: the component, a custom hook, or a services/api module? Why?

Q4 Name one concrete benefit of moving data-fetching logic out of a component and into a custom hook.

PART 2

Component Patterns & Custom Hooks

- Composition, compound components & controlled inputs
- Headless UI and reusable component libraries
- Custom hooks: useFetch, useDebounce, useAuth, usePagination …
- Separating logic from UI; hook design best practices

Composition over Configuration

```
// ✕ prop explosion — rigid
<Card title="..." subtitle="..." image="..."
      footerButton="Buy" badge="New" .../>

// ✓ composition with children & slots
<Card>
  <Card.Header>{title}</Card.Header>
  <Card.Body><ProductInfo p={p}/></Card.Body>
  <Card.Footer>
    <Button onClick={buy}>Buy</Button>
  </Card.Footer>
</Card>
```

Why compose?

- **children is React's built-in slot**
- Callers control content, not a rigid prop list
- Avoids 15-prop 'god components'
- Each part stays small and replaceable
- Foundation for compound components (next)

Heuristic: if a component keeps gaining boolean/string props to cover new cases, it wants composition instead.

Compound Components — Shared Implicit State

```
// parts share state via context, set by <Tabs>
<Tabs defaultValue="details">
  <Tabs.List>
    <Tabs.Trigger value="details">Details</Tabs.Trigger>
    <Tabs.Trigger value="reviews">Reviews</Tabs.Trigger>
  </Tabs.List>
  <Tabs.Panel value="details">...</Tabs.Panel>
  <Tabs.Panel value="reviews">...</Tabs.Panel>
</Tabs>

// internally: const Ctx = createContext();
// Tabs provides {active,setActive}; parts consume it
```

The idea

- Parent holds state in React Context
- Children read it implicitly — no prop drilling
- Clean, expressive API for the caller
- Flexible markup & ordering
- Pattern behind Tabs, Accordion, Select, Menu

Composition gives structure; compound components add shared state behind the scenes — so <Tabs.Trigger> just works.

Controlled vs Uncontrolled Components

Controlled

React state is the single source of truth

- value + onChange bound to state
- Instant validation, formatting, conditional UI
- More re-renders; more wiring
- Use for: live validation, dependent fields

Uncontrolled

The DOM holds the value; read via ref

- defaultValue + ref; read on submit
- Fewer re-renders; less code
- Harder to react to every keystroke
- Use for: simple/large forms, file inputs

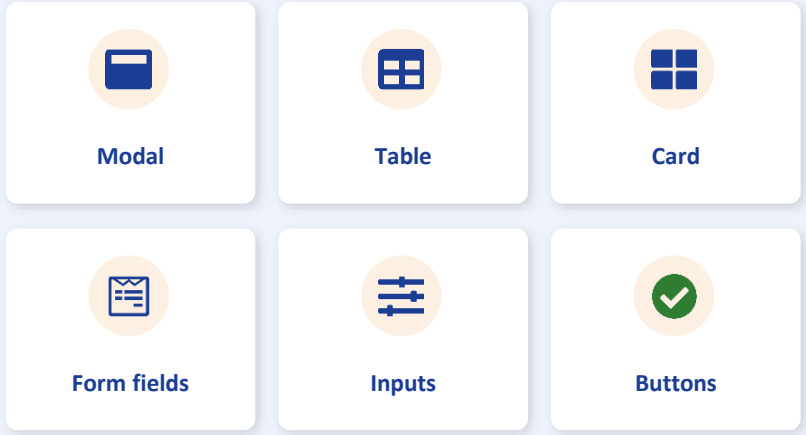
```
// controlled
const [q,setQ]=useState("");
<input value={q}
  onChange={e=>setQ(e.target.value)}/>
// uncontrolled
const ref=useRef();
<input defaultValue="" ref={ref}/>
```

Headless UI & a Reusable Component Library

Headless UI

- **Logic & accessibility, ZERO styling**
- You own the look; the library owns behaviour (focus, keyboard, ARIA)
- e.g. Radix UI, React Aria, Headless UI, TanStack Table
- Style with Tailwind/CSS to match your brand
- Accessible by default — a huge win

Your shared component library



Build once, reuse everywhere

- A consistent Modal/Table/Card/Form set enforces design consistency and slashes duplication
- Compose headless behaviour + your styles → accessible, on-brand, maintainable primitives

Custom Hooks — Logic You Can Reuse

- **A custom hook is a function starting with 'use' that calls other hooks**
  - Extracts stateful logic OUT of components so it can be reused & tested
- **Components become 'dumb and pretty'; hooks hold the brains**
- **Same logic, many components — write once, use everywhere**
- **Composable: hooks can call other hooks**
- **The idiomatic React way to share non-visual logic (replaces old HOC/render-props pain)**

```
// before: logic trapped in the component
function ProductList() {
  const [data,setData]=useState();
  const [loading,setLoading]=useState(true);
  useEffect(()=>{ /* fetch... */ },[]);
  ...
}

// after: logic in a reusable hook
function ProductList() {
  const {data,isLoading,error} =
        useFetch('/api/products');
}
```

Essential Custom Hooks (1): Data & Input

`useFetch(url)`

Wraps fetch/axios with { data, isLoading, error }. The bread-and-butter data hook.

`useDebounce(value, ms)`

Delays a fast-changing value (e.g. search text) so you don't fire an API call on every keystroke.

`useLocalStorage(key, init)`

useState that persists to localStorage — survives refresh. Great for theme, drafts, filters.

useDebounce.js

```
// useDebounce — the classic
function useDebounce(value, delay=400){
  const [v,setV]=useState(value);
  useEffect(()=>{
    const id=setTimeout(
      ()=>setV(value), delay);
    return ()=>clearTimeout(id);
  },[value,delay]);
  return v;    // debounced value
}
```

Essential Custom Hooks (2): App Concerns

`useAuth()`

Exposes the current user, login/logout and isAuthenticated from an Auth context. Used by route guards.

`usePagination(items)`

Holds page, pageSize and helpers (next/prev); returns the current slice. Reused on every list.

`useForm()`

Manages values, errors, touched & submit handling — the core idea behind React Hook Form.

useAuth.js

```
// useAuth — consume an Auth context
const AuthCtx = createContext();

export function useAuth(){
  const ctx = useContext(AuthCtx);
  if(!ctx) throw new Error(
    'useAuth outside <AuthProvider>');
  return ctx; // {user,login,logout}
}
```

Custom Hook Design — Best Practices

Name & rules

- Always prefix with use*
- Call hooks at the top level only — never in conditions/loops
- One hook, one responsibility

Clear contract

- Return an object {data,isLoading,error} (named) or a tuple for simple pairs
- Stable function identities (useCallback)
- Document inputs & outputs

Side-effect hygiene

- Always clean up timers, subscriptions, listeners
- Correct dependency arrays
- Avoid stale closures

Composability

- Build big hooks from small ones
- Keep them UI-free (no JSX)
- Make them testable in isolation

Checkpoint 02 – Discussion

Discuss in pairs — 5 minutes, then we compare

D1 Your Card component has grown to 14 props. Which pattern fixes it, and what does the refactored usage look like?

D2 A teammate hand-rolls a <Dropdown> with focus-trapping and arrow-key support. Argue for using a headless library instead.

D3 A search box calls the API on every keystroke and the server is overloaded. Which two hooks solve this together, and how?

D4 Where should the user's auth state live, and how does a component read it without prop drilling?

PART 3

Advanced Forms & Validation

- Why complex forms are hard in plain React
- React Hook Form — performant, minimal re-renders
- Schema validation with Zod / Yup
- Dynamic fields, error handling & a real example

Why Complex Forms Are Hard

- **Controlled-everything forms re-render on every keystroke**
  - 30 fields × live state = sluggish, lots of boilerplate
- **Validation is more than 'required'**
  - Cross-field rules (password = confirm), async checks (email taken), formats
- **Error & touched state must be tracked per field**
- **Dynamic fields: add/remove rows (order items, phone numbers)**
- **Submission: disable while sending, show server errors, reset**

```
// the boilerplate trap (per field!)
const [email,setEmail]=useState('');
const [emailErr,setEmailErr]=useState('');
const [touched,setTouched]=useState(false);
// ...repeat ×30 fields...

function validate(){ /* big if-else */ }
// hard to read, easy to get wrong,
// re-renders the whole form constantly
```

This is exactly the pain React Hook Form + a schema validator remove. Next slides →

React Hook Form — Fast, Minimal Re-renders

```
import { useForm } from 'react-hook-form';

const { register, handleSubmit,
        formState:{ errors, isSubmitting } }
      = useForm();

<form onSubmit={handleSubmit(onValid)}>
  <input {...register('email')} />
  {errors.email && <span>...</span>}
  <input {...register('password')} />
  <button disabled={isSubmitting}>
    Sign up</button>
</form>
```

Why teams use RHF

- **Uncontrolled under the hood → very few re-renders**
- register() wires inputs in one line
- handleSubmit runs validation, then your callback
- formState gives errors, isSubmitting, isDirty…
- Tiny, fast, integrates with Zod/Yup

Schema Validation with Zod / Yup

```
import { z } from 'zod';
import { zodResolver } from
  '@hookform/resolvers/zod';

const schema = z.object({
  name:  z.string().min(2),
  email: z.string().email(),
  price: z.number().positive(),
  password: z.string().min(8),
  confirm:  z.string()
}).refine(d => d.password===d.confirm,
  { message:'Passwords differ',
    path:['confirm'] });

useForm({ resolver: zodResolver(schema) });
```

One schema, many wins

- **Declare rules once, declaratively**
- Cross-field rules via .refine()
- Clear, per-field error messages
- Catches bad input at runtime, before use
- Reusable: validate the same shape elsewhere
- Yup is the older sibling — same idea

Dynamic Fields & Error Handling

```
// add/remove rows with useFieldArray
const { fields, append, remove } =
  useFieldArray({ control, name:'items' });

{fields.map((f,i)=>(
  <div key={f.id}>
    <input {...register(`items.${i}.qty`)}/>
    <button onClick={()=>remove(i)}>x</button>
  </div>))}
<button onClick={()=>append({qty:1})}>
  + Add item</button>
```

Good error UX

- Show errors after blur/submit, not first keystroke
- Message next to the field, in red, specific
- Summarise on submit; focus the first error
- Map server errors back to fields
- Keep the submit button's state honest

Two error sources: client (schema → instant, friendly) and server (e.g. 409 'email already registered' → map onto the email field). Handle both gracefully.

Putting It Together: A Registration Form

1. Define schema	Zod object with field rules + .refine() for password match
2. Wire RHF	useForm({ resolver: zodResolver(schema) }); register inputs
3. Validate	handleSubmit runs the schema; errors shown inline per field
4. Submit	POST to /api/auth/register; disable button while sending
5. Handle result	Success → redirect/login; 409 → map server error to email field

Reusable takeaways

- This pattern fits ANY form: login, checkout, settings
- Schema + RHF + good errors = robust & maintainable
- Client validates for UX; server validates for safety
- Reuse the schema to validate on both client and API

PART 4

API Integration, Server State & Routing

- Axios / Fetch best practices · loading, success & error states
- Client state vs server state — the key distinction
- TanStack Query & RTK Query
- Nested, protected, role-based and lazy routes

Axios / Fetch Best Practices & Request States

```
// one configured axios instance (services/api.js)
export const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL });

// interceptor attaches the auth token
api.interceptors.request.use(cfg => {
  cfg.headers.Authorization =
    `Bearer ${getToken()}`;
  return cfg; });

// callers just use endpoints
export const getProducts = () =>
  api.get('/products').then(r => r.data);
```

Best practices

- One configured instance; baseURL from env
- Interceptors for auth & error handling
- Centralise endpoints in a services layer
- Always handle errors — no silent failures

Loading	Success	Error
spinner / skeleton	render the data	message + retry

Every data fetch has three states — design the UI for all three, every time.

Client State vs Server State — The Key Distinction

Client state

Owned by the UI; you control it fully

- Modal open/closed, active tab, theme
- Form input values, wizard step
- Selected filters before applying
- **Tools: useState, Context, Redux, Zustand**

Server state

Lives on the server; you only borrow a copy

- Products, orders, user profile from the API
- Can be stale; others may change it
- Needs caching, refetch, sync, dedupe
- **Tools: TanStack Query, RTK Query**

Why it matters: storing API data in Redux/useState forces you to hand-build caching, loading, refetching and invalidation — server-state libraries give all of that for free.

TanStack Query — Server State Done Right

```
// read: caching, loading & error built in
const { data, isLoading, error } = useQuery({
  queryKey: ['products', { category }],
  queryFn: () => getProducts(category),
});

// write: mutate + invalidate to refetch
const qc = useQueryClient();
const add = useMutation({
  mutationFn: createProduct,
  onSuccess: () => qc.invalidateQueries(
    { queryKey:['products'] }) });
```

Out of the box

- **Caching by queryKey**
- Automatic loading & error states
- Background refetch & stale-while-revalidate
- Request dedupe across components
- Mutations + cache invalidation
- Pagination & infinite scroll helpers

queryKey is the cache identity: same key → shared cached data; change a key part (e.g. category) → it fetches and caches that variant. Invalidate a key to refetch.

RTK Query — When You're Already on Redux Toolkit

```
// define an API slice once
export const api = createApi({
  baseQuery: fetchBaseQuery({ baseUrl:'/api' }),
  endpoints: (b) => ({
    getProducts: b.query({ query:()=>'products' }),
    addProduct:  b.mutation({
      query:(body)=>({url:'products',
        method:'POST', body}) }),
  }) });

// auto-generated hooks!
const { data, isLoading } = useGetProductsQuery();
```

Why & when

- **Built into Redux Toolkit**
- Auto-generates query/mutation hooks
- Caching, dedupe, invalidation — like TanStack
- Tag-based cache invalidation
- **Pick it if Redux is already in the app**
- Else TanStack Query is lighter for pure server state

Pagination, Filtering, Optimistic Updates & Caching

Pagination

Encode page & pageSize in the queryKey; keep previous data while the next page loads for a smooth UI.

Filtering

Filters/search go in the key too — each combination is its own cached query; debounce the search input.

Optimistic updates

Update the cache immediately, then confirm with the server; roll back on error. Feels instant.

Caching & staleness

staleTime controls when data refetches; cached data shows instantly while a fresh copy loads in the background.

Advanced Routing: Nested, Layout & Lazy Routes

```
// React Router — nested + layout + lazy
const Settings = lazy(()=>import('./Settings'));

<Routes>
  <Route element={<DashboardLayout/>}>
    <Route path='products' element={<Products/>}/>
    <Route path='products/:id'
           element={<ProductDetail/>}/>
    <Route path='settings' element={<Settings/>}/>
  </Route>
  <Route path='*' element={<NotFound/>}/>
</Routes>
```

Concepts

- **Layout route renders shared shell + <Outlet/>**
- Nested routes share that layout
- :id = URL parameter (useParams)
- lazy() + Suspense → code-split per route
- path='*' → 404 / error page

Layout routes remove repetition (one header/sidebar for many pages); lazy routes keep the initial bundle small.

Protected & Role-Based Routes (Route Guards)

```
// a guard component using useAuth()
function RequireAuth({ roles }) {
  const { user } = useAuth();
  const loc = useLocation();
  if (!user)
    return <Navigate to='/login'
             state={{from: loc}} replace/>;
  if (roles && !roles.includes(user.role))
    return <Navigate to='/forbidden' replace/>;
  return <Outlet/>;   // allowed
}

<Route element={<RequireAuth roles={['Admin']}/>}>
  <Route path='admin' element={<AdminPanel/>}/>
</Route>
```

How guards work

- Wrap routes in a guard that checks auth/role
- Not logged in → redirect to /login (remember origin)
- Wrong role → /forbidden
- Allowed → render <Outlet/>
- **UI hiding is UX only — the API must enforce real security**

Error Pages & a Look at the Next.js App Router

Robust error handling

- 404 route (path='*') for unknown URLs
- Error boundaries catch render-time crashes
- Friendly message + a way back, not a blank screen
- Per-route loading & error UI
- Log errors for monitoring

Next.js App Router (overview)

- File-system routing: folders = routes
- layout.jsx, page.jsx, loading.jsx, error.jsx
- Server Components & data fetching on the server
- Built-in code splitting & image optimization
- Great for SEO & performance (SSR/SSG)

React Router vs Next.js: React Router = routing for a client-side SPA (what we use here). Next.js = a full framework adding server rendering, file routing and built-in optimizations. Choose per project needs (LO4).

Checkpoint 03 – Discussion

Discuss in pairs — 5 minutes, then we compare

D1 Classify each as client or server state: theme toggle, product list, modal open, logged-in user's orders, current wizard step.

D2 A teammate stores all API data in Redux and writes loading flags by hand. What would you suggest instead, and why?

D3 TanStack Query vs RTK Query — give one concrete situation where each is the better pick.

D4 A user disables JavaScript guards and navigates to /admin. Is your data safe? What actually protects it?

PART 5

Performance Optimization & State-Management Decisions

- Profiling & detecting unnecessary re-renders
- React.memo · useMemo · useCallback
- Lazy loading, code splitting & list virtualization
- Choosing the right state tool — the decision table

Profiling & Unnecessary Re-Renders

- **Rule #1: measure before optimizing — guesses are usually wrong**
- **React DevTools Profiler records what rendered, why, and how long**
  - "Highlight updates" visually flashes components as they re-render
- **A component re-renders when its state, props or context change…**
  - …and ALL its children re-render by default — even if their props didn't change
- **Common culprits:**
  - New object/array/function created inline on every render
  - Context value changing too often; state lifted too high



Profiler workflow

1. Record an interaction
2. Read the flame graph (wide = slow)
3. Check 'why did this render?'
4. Fix the cause → re-measure

```
// ✗ new array every render → child re-renders
<List items={data.filter(x => x.active)} />
// ✓ memoize the derived value
const active = useMemo(
  () => data.filter(x => x.active),
[data]);
```

React.memo, useMemo & useCallback

Tool	What it memoizes	Use when…
React.memo	A whole component's render output (skips render if props are equal)	An expensive child re-renders with the same props
useMemo	A computed VALUE between renders	A calculation is costly, or you need a stable object/array reference
useCallback	A FUNCTION reference between renders	Passing a callback to a memoized child or a hook dependency

```
const Row = React.memo(({ item, onPick }) => ( ... ));   // skips equal-props renders
const total  = useMemo(() => heavy(items), [items]);     // recompute only if items change
const onPick = useCallback((id) => select(id), []);      // stable fn → memo stays effective
```

These have a cost (memory + comparisons). Apply them to PROVEN hotspots, not everywhere. useCallback is pointless unless the child is memoized.

Bundles, Lazy Loading & List Virtualization

Code splitting + lazy

- Don't ship the whole app up front
- React.lazy + <Suspense> per route
- Smaller initial bundle → faster first load

Route-based splitting

- Each page becomes its own chunk
- Admin code not downloaded by shoppers
- Pairs with lazy routes from Part 5 routing

List virtualization

- Render only visible rows (windowing)
- react-window / TanStack Virtual
- 1000s of rows stay smooth

Image optimization

- Right size & modern formats (WebP)
- Lazy-load below-the-fold images
- Often the biggest payload on a page

State Management — Know Your Options

- **First, classify the state — this decides the tool**
  - Local UI state (is the modal open?) → local useState
  - Shared client state (theme, current user) → Context or a store
  - Server state (data from the API) → a server-cache library
- **Most 'state management pain' is using a CLIENT tool for SERVER data**

The key reframe

Client state and server state are different problems. Server state is shared, async, cached and can go stale — purpose-built libraries (TanStack/RTK Query) handle fetching, caching and refetching so you don't reinvent it in Redux.

The mainstream toolbox

useState Local UI state	**Context API** Low-frequency shared state	**Redux Toolkit** Large, complex client state
Zustand Simple global store	**TanStack Query** Server state (any backend)	**RTK Query** Server state (Redux apps)

State Management Decision Table (LO4)

If you need…	Use	Why / notes
State used by one component	**useState**	Simplest; never reach for more until you must
A few components share rarely-changing data	**Context API**	Theme, locale, current user; avoid for fast-changing values
Large, complex, interconnected client state	**Redux Toolkit**	Predictable, debuggable, DevTools; more setup
Simple global state, minimal boilerplate	**Zustand**	Tiny API, no providers; great middle ground
Data from an API (any stack)	**TanStack Query**	Caching, loading/error, refetch, pagination built-in
API data + already using Redux	**RTK Query**	Same benefits, integrated with the Redux store

Golden rule: start local, lift only when sharing is needed, and keep server state OUT of client stores. Justify every choice against the requirement — that is LO4.

Case Study: Designing the LankaMart Frontend



Requirements → frontend design decisions (with justification — LO4)

Requirement	Design decision	Justification
Organise a growing codebase	Feature-based structure: features/products, cart, auth	Locality & team scaling; easy to find and delete features
Browse & search products	TanStack Query + useDebounce on the search box	Server data cached; debounce avoids a request per keystroke
Reusable UI everywhere	Shared Button, Modal, Table; compound <Table> pattern	Consistency; build once, use across features
Checkout form with validation	React Hook Form + Zod schema	Performant, declarative validation; one shared schema
Admin-only dashboard	Protected + role-based, lazy-loaded route	Security gate (server-enforced) + smaller shopper bundle
Fast product listing	List virtualization + image optimization	Smooth scrolling for large catalogues
Cart state while shopping	Zustand (small global store)	Simple global client state; no async, no Redux overhead

Group Activity: Choose the Architecture (20 min)

Groups of 3–4 · whiteboard or paper · present in 60 seconds

Scenario: Campus Event Portal

Students browse & search events, register (form), and see 'my tickets'. Organisers have an admin area to create events and view attendees.

- **Task 1 — Sketch the feature-based folder structure**
- **Task 2 — For each piece of state (search results, the registration form, the theme, the logged-in user, the cart of tickets), pick a tool & justify it**
- **Task 3 — Name two reusable components and one custom hook you'd build**
- **Task 4 — Which routes are protected / role-based / lazy-loaded?**

Marking lens (= the rubric)

- Feature-based, not type-based structure?
- Server state via a query library, not Redux/Context?
- Local state kept local; right tool per state?
- Reusable components identified, logic in hooks?
- Admin routes protected (and server-enforced) + lazy?

Quick Quiz — 6 Questions

1	Data fetched from your REST API is which kind of state?	**A) local UI B) server state C) form state**
2	Best default tool for caching API data with loading/error built in?	**A) Redux B) Context C) TanStack Query**
3	React.memo on a child still re-renders. Most likely cause?	**A) React bug B) an unstable prop C) too much CSS**
4	Which hook gives a STABLE function reference?	**A) useMemo B) useEffect C) useCallback**
5	A frontend route guard hides /admin. Is the data secure?	**A) yes B) only if the API enforces authz C) yes, if lazy**
6	200 product rows scroll slowly. Best fix?	**A) React.memo all B) list virtualization C) remove images**

Lecture Summary — Key Takeaways

Architect for growth

Feature-based structure; separate UI, logic (hooks) & services.

Treat server state specially

TanStack/RTK Query for caching, loading, errors & refetch — not Redux.

Design reusable components

Composition, compound components, controlled inputs & headless UI.

Route & optimize deliberately

Protected & lazy routes; profile, memoize, virtualize, split bundles.

Extract logic into hooks

useFetch, useDebounce, useAuth… thin components, smart hooks.

Choose tools by requirement

Local → Context → store; client vs server state. Justify every choice (LO4).

Learning Outcomes, Next Lecture & Preparation

LO	Covered today by…
LO1	Component model, the React ecosystem, and the client-state vs server-state distinction
LO2	Feature-based architecture, reusable components, custom hooks, forms, routing & data fetching
LO3	Performance optimization, validation, error handling and separation of UI/logic/services
LO4	The state-management decision table and the case study / architecture activity



Next: Lecture 03 — C#, .NET & REST APIs

Build the backend this frontend will call: C#/.NET fundamentals and ASP.NET Core REST API development — the GET/POST endpoints behind today's components.



Before the lab / next week

- Scaffold a Vite + React (JavaScript) app; set up the feature-based folders
- Build one reusable component and one custom hook (e.g. useDebounce)
- Read: React docs 'Thinking in React' & TanStack Query overview
- Sketch the component tree for YOUR assignment frontend

References & Further Reading

- **Official documentation (primary sources)**
  - React Documentation — react.dev (Learn React, 'Thinking in React', Hooks reference)
  - TanStack Query & TanStack Virtual — tanstack.com
  - Redux Toolkit & RTK Query — redux-toolkit.js.org
  - React Router — reactrouter.com · Zustand — docs.pmnd.rs
  - React Hook Form — react-hook-form.com · Zod — zod.dev
- **Guides & tooling**
  - Vite — vitejs.dev · React DevTools Profiler (Chrome/Firefox extension)
  - web.dev — performance, Core Web Vitals & image optimization (Google)
- **Recommended reading**
  - Banks, A. & Porcello, E. (2020) Learning React, 2nd edn. O'Reilly Media.
  - Roldán, C. S. (2021) React Design Patterns and Best Practices. Packt.

SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 03

C#, .NET Fundamentals, ASP.NET Core and REST API Development

Stack: C# 12 · .NET 8 · ASP.NET Core Duration: 3 hours Learning outcomes: LO2 · LO3 · LO4

Year 3 • Semester 1 • 4 Credits

Lecturer: Eishan Weerasinghe • eishan.w@sliit.lk

Where This Lecture Fits in SE3090

L01	L02	TODAY	Next
Framework fundamentals & trends	Advanced React frontend	C#, .NET, ASP.NET Core & REST APIs	Database, AuthN/AuthZ & integration

- **Module content areas 2(c)–2(e): REST communication, framework-based backend development, server-side applications**
- The API tier you used as a 'black box' in the database lecture becomes today's subject
- Directly assessed in labs, the mini hackathon and the Main Assignment backend (25%)
- Foundation for CI/CD, deployment and AI-assisted backend development in later lectures

Lecture Learning Outcomes

LO2

Apply suitable frameworks & tools

Build a working ASP.NET Core REST API using C# OOP, controllers, services, models and routing.

LO3

Use best practices for integration & quality

Structure a maintainable layered backend with DI, validation, exception handling, logging and tested endpoints (Swagger/Postman).

LO4

Select & justify approaches

Justify the choice of .NET/ASP.NET Core and REST design decisions (resources, methods, status codes) for project requirements.

Why C#, .NET and ASP.NET Core Matter

Industry footprint

Enterprise systems, banking, government, healthcare, games (Unity) and cloud services run on .NET — strong demand in Sri Lanka and globally.

Performance

ASP.NET Core consistently ranks among the fastest mainstream web frameworks (TechEmpower benchmarks) — millions of requests/sec on Kestrel.

One language, many targets

C# covers web APIs, desktop, mobile (MAUI), games, IoT and cloud functions — skills transfer across domains.

Modern & open source

Cross-platform (Windows/Linux/macOS), MIT-licensed, 6-monthly releases, strong typing + rich tooling (Visual Studio, Rider, VS Code).

Lecture Roadmap – 3 Hours

1	**C# fundamentals** Syntax · types · control flow · methods · classes	~30 min
2	**OOP in C#** Encapsulation · inheritance · polymorphism · abstraction · interfaces	~35 min
3	**.NET & ASP.NET Core** Platform · runtime · DI · middleware pipeline	~25 min
4	**REST API architecture** Resources · HTTP methods · status codes · JSON · lifecycle	~35 min
5	**Controllers, services, models & routing** MVC-style structure · DTOs · attribute routing	~25 min
6	**Requests, responses & testing + best practices** Binding · responses · Swagger · Postman · layered design	~30 min

PART 1

C# Programming Fundamentals

- Role of C# in modern software development
- Program structure, variables, types and operators
- Control structures and methods
- First look at classes and objects

The Role of C# in Modern Development

- **General-purpose, statically typed, object-oriented language (2000, Microsoft; now open standard)**
  - Compiled to IL (Intermediate Language), executed by the .NET runtime (JIT)
- **Static typing = errors caught at compile time, great IDE support, safer refactoring**
- **Modern features: async/await, LINQ, pattern matching, records, nullable reference types**
- **Where you'll meet it:**
  - Web APIs & microservices (ASP.NET Core) — our focus
  - Enterprise/desktop apps, Azure cloud functions, Unity games, MAUI mobile

C# vs languages you know

vs JavaScript:
compile-time types instead of runtime surprises; classes are real, not prototypes

vs Java:
very similar feel; C# adds properties, LINQ, async/await ergonomics, value types

vs Python:
more ceremony, but far faster and safer for large team codebases

Anatomy of a C# Program

Program.cs

```
// Program.cs — modern top-level statements
using System;                          // import namespace

Console.WriteLine("Hello, SE3090!");

// Classic explicit form (same program)
namespace LankaMart.Api;               // organises code

public class Program {
    public static void Main(string[] args) {
        Console.WriteLine("Hello, SE3090!");
    }
}
```

Building blocks

- using — imports a namespace
- namespace — logical grouping; mirrors folder structure
- class — the unit of code; (almost) everything lives in one
- Main / top-level statements — the entry point
- Statements end with ; — blocks use { }

Try it: `dotnet new console → dotnet run.` ASP.NET Core's Program.cs uses exactly this top-level style.

Variables, Data Types and Operators

```
int     qty      = 3;            // whole numbers
double  rating   = 4.7;          // floating point
decimal price    = 2500.00m;     // money — exact!
bool    inStock  = true;
string  name     = "Wireless Mouse";
char    grade    = 'A';

var total = price * qty;         // type inferred: decimal
string? note = null;             // nullable reference

string label =
  $"{name}: {total:C} ({(inStock ? "in" : "out")})";
```

Key points

- **decimal for money — never double (rounding errors)**
- var = inference, still strongly typed
- string? — nullable reference types make null explicit
- $"..." string interpolation
- Operators as expected: + - * / %, == != < >, && || !

Backend link: these types map directly to your PostgreSQL columns — decimal ↔ NUMERIC, int ↔ INT, string ↔ TEXT, bool ↔ BOOLEAN. Choosing the C# type IS a schema decision.

Control Structures

```
// if / else
if (qty <= 0)
    return "Quantity must be positive";
else if (qty > stock)
    return "Not enough stock";

// switch expression (modern C#)
var fee = method switch {
    "card"  => 0m,
    "cod"   => 250m,
    _       => throw new ArgumentException()
};
```

```
// loops
for (int i = 0; i < items.Count; i++)
    Console.WriteLine(items[i]);

foreach (var item in items)   // preferred
    total += item.Price * item.Qty;

while (retries < 3 && !ok)
    ok = TryConnect(ref retries);

// LINQ — declarative alternative
var total = items.Sum(i => i.Price * i.Qty);
```

Guard clauses (validate-and-return-early) keep backend methods flat and readable — you'll see them in every controller and service today.

Methods and Parameters

```
// access  return-type  Name(parameters)
public decimal CalculateTotal(
    List<OrderItem> items,
    decimal discountPct = 0)        // optional param
{
    if (items is null || items.Count == 0)
        throw new ArgumentException("Empty order");

    var subtotal = items.Sum(i => i.Price * i.Qty);
    return subtotal * (1 - discountPct / 100m);
}

// expression-bodied — for one-liners
public bool IsInStock(Product p) => p.StockQty > 0;
```

Reading the signature

- public — visible outside the class
- decimal — return type (void = none)
- Parameters are typed; optional ones take defaults
- PascalCase method names, verbs first (CalculateTotal, GetById)
- One method = one job — small methods are testable methods

Classes and Objects — First Look

```
public class Product
{
    // properties — data with controlled access
    public long    Id       { get; set; }
    public string  Name     { get; set; } = "";
    public decimal Price    { get; set; }
    public int     StockQty { get; set; }

    // behaviour — methods that use the data
    public bool CanFulfil(int qty) => StockQty >= qty;
}

var mouse = new Product { Name = "Mouse", Price = 2500m };
bool ok = mouse.CanFulfil(2);     // object in action
```

Class vs object

- Class = blueprint (the Product design)
- Object = a concrete instance made with new
- Properties hold state; methods provide behaviour
- new creates an instance; object initializer { } sets properties
- This Product maps 1:1 to the products table from Lecture 03

Checkpoint 01 – Quick Questions

Answer with your neighbour — 4 minutes

Q1 Why must an order total use decimal rather than double? What could go wrong in production?

Q2 var x = 10; x = "ten"; — does this compile? What does that tell you about var?

Q3 Rewrite mentally with a guard clause: if (user != null) { if (user.IsActive) { /* 30 lines */ } }

Q4 In one sentence: what is the difference between the Product class and the mouse object?

PART 2

Object-Oriented Programming in C#

- Constructors and properties
- Encapsulation · inheritance · polymorphism · abstraction
- Interfaces — the backbone of ASP.NET Core design
- Practical OOP for backend models and services

Constructors and Properties

```
public class Order
{
    public long Id { get; private set; }      // read-only outside
    public long CustomerId { get; }           // immutable
    public string Status { get; private set; }
    public DateTime CreatedAt { get; } = DateTime.UtcNow;

    // constructor — guarantees a valid object
    public Order(long customerId)
    {
        if (customerId <= 0)
            throw new ArgumentException("Invalid customer");
        CustomerId = customerId;
        Status = "PENDING";
    }
}
```

Why it matters

- Constructor = the object's front door: no Order exists without a valid customer
- Properties control WHO can change WHAT (get; private set;)
- An invalid state becomes unrepresentable — fewer bugs by design
- UtcNow default: timestamps in UTC, echoing Lecture 03

Encapsulation — Protecting Object State

```
public class BankAccount
{
    private decimal _balance;          // hidden state

    public decimal Balance => _balance;  // read-only view

    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException();
        _balance += amount;
    }

    public bool Withdraw(decimal amount) =>
        amount > 0 && amount <= _balance
            ? (_balance -= amount) >= 0 : false;
}
```

The principle

- State is private; behaviour is public
- All changes pass through methods that enforce the rules
- Impossible to set _balance = -1,000,000 from outside
- Access modifiers: public · private · protected · internal
- Rule of thumb: start private, open up only with reason

Backend reality: encapsulation is why business rules live in services/domain objects — not spread across controllers that poke at raw data.

Inheritance — Sharing Structure and Behaviour

```
public class User                       // base class
{
    public long Id { get; set; }
    public string Name { get; set; } = "";
    public string Email { get; set; } = "";
    public virtual string Describe() => $"User {Name}";
}

public class Staff : User               // derived class
{
    public string Branch { get; set; } = "";
    public override string Describe()
        => $"Staff {Name} ({Branch})";
}
```

Mechanics

- : User — Staff IS-A User; inherits all members
- virtual — base allows overriding
- override — derived replaces behaviour
- base.Member reaches the parent
- C# = single inheritance (one base class only)

Use with care: prefer inheritance for true IS-A relationships (Staff is a User). For sharing capabilities, prefer composition or interfaces — "composition over inheritance".

Polymorphism — One Interface, Many Behaviours

```
List<User> users = new()
{
    new User  { Name = "Amal"  },
    new Staff { Name = "Nadia", Branch = "Kandy" },
    new Admin { Name = "Ruwan" }
};

foreach (var u in users)
    Console.WriteLine(u.Describe());

// Output — same call, three behaviours:
// User Amal
// Staff Nadia (Kandy)
// Admin Ruwan [full access]
```

What just happened

- The variable's type is User; the OBJECT decides which Describe runs (runtime dispatch)
- Calling code stays generic — new user types need zero changes here
- This is the Open/Closed Principle: open for extension, closed for modification
- Same idea powers ASP.NET Core: the framework calls YOUR controller methods polymorphically

Abstraction and Interfaces

```
// interface — a pure contract, no implementation
public interface IEmailSender
{
    Task SendAsync(string to, string subject, string body);
}

public class SmtpEmailSender : IEmailSender { ... }
public class FakeEmailSender : IEmailSender { ... } // tests

// consumers depend on the CONTRACT, not a class:
public class OrderService(IEmailSender email) { ... }
```

Interface vs abstract class

- Interface: contract only; a class can implement MANY
- Abstract class: partial implementation + contract; single inheritance
- Naming: IName (IEmailSender, IProductService)

Why backends are built on interfaces

Swap implementations (SMTP → SendGrid) without touching consumers · plug in fakes for unit tests · ASP.NET Core dependency injection registers and resolves BY INTERFACE — this single concept unlocks the next part of the lecture.

OOP in Practice — A Backend Domain Model

```
public class Order
{
    private readonly List<OrderItem> _items = new();
    public IReadOnlyList<OrderItem> Items => _items;
    public string Status { get; private set; } = "PENDING";
    public decimal Total => _items.Sum(i => i.LineTotal);

    public void AddItem(Product p, int qty)
    {
        if (Status != "PENDING")
            throw new InvalidOperationException("Order locked");
        if (!p.CanFulfil(qty))
            throw new InvalidOperationException("No stock");
        _items.Add(new OrderItem(p.Id, qty, p.Price));
    }

    public void Confirm() => Status = "CONFIRMED";
}
```

Spot all four pillars

- Encapsulation: _items private, rules in AddItem
- Abstraction: callers see AddItem/Confirm, not list mechanics
- Computed property: Total derives from items — never stored stale
- Snapshot price captured (p.Price) — Lecture 03's unit_price rule, now in code!
- Polymorphism & inheritance arrive via EF Core & controllers

Checkpoint 02 – OOP Quick Questions

Pairs — 4 minutes, then we discuss

Q1 User u = new Admin(); u.Describe(); — whose Describe runs, and what is this mechanism called?

Q2 A teammate makes Order.Status a public { get; set; }. What bug becomes possible that was impossible before?

Q3 PaymentService needs card, cash and bank-transfer handling that can grow. Interface or abstract class for the payment types — and why?

Q4 True or false: 'Inheritance is the main tool for code reuse in modern backends.' Defend your answer.

PART 3

.NET and ASP.NET Core

- What .NET is: platform, runtime, libraries, CLI
- .NET vs C# vs ASP.NET Core — untangled
- Why ASP.NET Core for modern web APIs
- Dependency injection & the middleware pipeline

What is .NET? Platform, Runtime and Ecosystem

Your application	LankaMart.Api — your C# code
Frameworks	ASP.NET Core (web) · MAUI (mobile) · Worker services · Blazor
Base Class Library (BCL)	Collections · LINQ · IO · networking · JSON — batteries included
Runtime (CLR)	JIT compilation · garbage collection · type safety · async scheduling
Operating system	Windows · Linux · macOS — one codebase, all of them

Plus the tooling

- dotnet CLI: new, run, build, test, publish
- NuGet: package manager (like npm)
- IDEs: Visual Studio, Rider, VS Code
- .NET 8 = current LTS (3-yr support)

A development platform: a runtime that executes your code + a huge standard library + tooling to build and run it.

.NET vs C# vs ASP.NET Core — Untangled

C#

The language

- Syntax, types, OOP — what you WRITE
- Compiles to IL
- Other .NET languages exist (F#, VB)

The language you speak

.NET

The platform

- Runtime (CLR) + libraries (BCL) + tools
- WHERE your code runs
- Cross-platform, open source

The country and its roads

ASP.NET Core

The web framework

- Built ON .NET for web apps & APIs
- Routing, controllers, DI, middleware, Kestrel
- What we use to SERVE HTTP

A delivery company using those roads

Analogy: C# is to .NET what JavaScript is to Node.js; ASP.NET Core is the Express/NestJS of that world — language → platform → web framework.

- You write C#, which runs on .NET, using ASP.NET Core to handle the web part

Why ASP.NET Core for Modern Web APIs

Fast by default

Kestrel + async I/O top the TechEmpower benchmarks among mainstream frameworks; async/await is idiomatic, not bolted on.

Cross-platform & container-ready

Develop on Windows/macOS, deploy to Linux Docker containers — the standard cloud path (Azure, AWS).

Batteries included

Routing, model binding, validation, DI container, configuration, logging, auth, OpenAPI — in the framework, not 30 npm packages.

Enterprise-grade & supported

LTS releases, security patches, massive ecosystem, first-class tooling — safe choice to justify for serious systems (LO4).

Dependency Injection (DI) — The Framework's Backbone

```
// 1. Register (Program.cs) — interface → implementation
builder.Services.AddScoped<IProductService, ProductService>();
builder.Services.AddScoped<IEmailSender, SmtpEmailSender>();
builder.Services.AddDbContext<AppDbContext>(...);

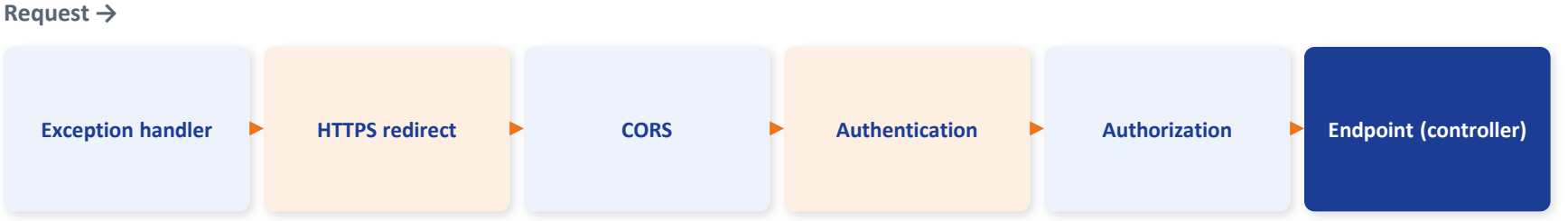
// 2. Request (constructor) — the container injects
public class ProductsController(IProductService svc)
    : ControllerBase
{
    [HttpGet] public Task<List<ProductDto>> GetAll()
        => svc.GetAllAsync();   // svc arrived automatically
}
```

Lifetimes

- Singleton — one instance forever (config, caches)
- **Scoped — one per HTTP request (services, DbContext)**
- Transient — new every time (lightweight, stateless)

Why: classes declare WHAT they need (interfaces); the container decides WHICH implementation and WHEN to create it. Result: loose coupling, easy testing (inject fakes), and one place to change wiring. This is 'Inversion of Control' — you don't new your dependencies; they are handed to you.

The Middleware Pipeline




```
// Program.cs — order matters!
app.UseExceptionHandler("/error");
app.UseHttpsRedirection();
app.UseCors("Frontend");
app.UseAuthentication();    // who are you?
app.UseAuthorization();     // may you?
app.MapControllers();
```

Mental model

- Each middleware can inspect, modify, short-circuit, or pass along
- 401 from JWT middleware = a short-circuit — your controller never runs
- Order matters: AuthN must precede AuthZ; exception handler wraps everything (first)

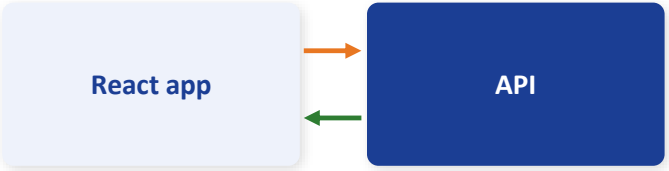
PART 4

REST API Architecture

- What an API is; what makes one RESTful
- Resources, endpoints and HTTP methods
- Status codes and the request/response lifecycle
- JSON and REST design best practices

What is an API? What is a Web API?

- **API = Application Programming Interface — a contract for software to talk to software**
  - You have used APIs all day: Console.WriteLine is the BCL's API; React hooks are React's API
- **A Web API exposes that contract over HTTP**
  - Client sends a request → server returns a response (usually JSON)
- **Why web APIs power full-stack systems:**
  - One backend serves web, mobile and partner integrations
  - Frontend and backend evolve and deploy independently
  - Clear security boundary (the only door to your data)




```
GET /api/products
200 OK + JSON
```

Restaurant analogy

The menu is the API contract; the waiter carries requests and responses (HTTP); the kitchen is your service + database. Diners never enter the kitchen — encapsulation at system scale.

REST — The Architectural Principles

Client–server

UI concerns separated from data/logic concerns; each evolves independently.

Stateless

Every request carries ALL context (incl. the JWT). No server session memory between calls.

Uniform interface

Resources identified by URLs; manipulated via standard HTTP methods; self-descriptive messages.

Resource-oriented

Model the domain as nouns (products, orders) — not actions (getProduct, doOrder).

Cacheable

Responses declare cacheability — GETs can be cached by browsers/CDNs for speed.

Layered system

Clients can't tell if they talk to the API, a gateway, or a load balancer — layers are invisible.

REST (Roy Fielding, 2000) is a STYLE, not a protocol or standard — 'RESTful' means following these constraints. Statelessness is the one you must never violate: it is what lets APIs scale horizontally.

Resources and Endpoint Design

Rule	✓ Good	✗ Avoid
Nouns, not verbs — the METHOD is the verb	GET /api/products	GET /api/getAllProducts
Plural collection names	/api/products, /api/orders	/api/product, /api/orderList
Identify items by id in the path	/api/products/42	/api/products?id=42 (for identity)
Nest ONLY true ownership, max 1 level	/api/orders/7/items	/api/customers/3/orders/7/items/2
Filters/paging via query string	/api/products?category=it&page=2	/api/products/it/page2
Consistent casing & style	kebab-or-lowercase, no trailing /	/api/ProductItems/, mixedStyles

A resource is a domain noun (usually an entity or aggregate from your ER diagram). The endpoint set for LankaMart almost writes itself from the Lecture 03 schema: /products, /categories, /orders, /orders/{id}/items, /auth/login.

HTTP Methods — The Verbs of REST

Method	Meaning	Example	Idempotent?	Body?
GET	Read a resource/collection — never changes state	GET /api/products/42	Yes (and safe)	No
POST	Create a new resource (server assigns id)	POST /api/orders	No	Yes
PUT	Replace a resource entirely	PUT /api/products/42	Yes	Yes
PATCH	Partially update specified fields	PATCH /api/products/42	Not guaranteed	Yes
DELETE	Remove a resource	DELETE /api/products/42	Yes	Usually no

Idempotent = repeating the request gives the same result. Why it matters: networks retry. Retrying DELETE /products/42 is harmless; retrying POST /orders can charge a customer twice — handle duplicates deliberately.

HTTP Status Codes — The Response Contract

2xx — Success

200 OK

GET/PUT/PATCH succeeded; body included

201 Created

POST succeeded; Location header points to the new resource

204 No Content

Success with nothing to return (DELETE)

4xx — Client's fault

400 Bad Request

Validation failed; malformed input

401 Unauthorized

Not authenticated (missing/invalid token)

403 Forbidden

Authenticated but not permitted

404 Not Found

Resource doesn't exist

409 Conflict

State conflict (duplicate email)

5xx — Server's fault

500 Internal Server Error

Unhandled exception — your bug

503 Service Unavailable

Overloaded / down for maintenance

Request → Response Lifecycle, and JSON

1. **React fetch()**
2. **HTTP request**
3. **Middleware pipeline**
4. **Routing → Controller**
5. **Service → EF Core → PostgreSQL**
6. **JSON response**

```
// HTTP request
GET /api/products/42 HTTP/1.1
Host: api.lankamart.lk
Authorization: Bearer eyJhbGc...
Accept: application/json

// HTTP response
HTTP/1.1 200 OK
Content-Type: application/json

{ "id": 42, "name": "Wireless Mouse",
  "price": 2500.00, "inStock": true }
```

JSON: lightweight, human-readable, language-neutral. ASP.NET Core serializes objects ↔ JSON automatically (System.Text.Json), converting PascalCase C# to camelCase JSON by default.

REST API Design Best Practices

Consistency above all

- Same naming style, casing, error shape and paging pattern on EVERY endpoint
- Consumers learn your API once

Paginate collections

- Never return unbounded lists: ?page=1&pageSize=20
- Return total count metadata for UIs

Version from day one

- /api/v1/products — breaking changes go to v2
- Old clients keep working

Uniform error shape

- Problem Details (RFC 9457) for all errors
- Machine-readable type + human title + traceId

Secure by default

- HTTPS only · [Authorize] unless explicitly public
- Validate every input (next part)

Document as you build

- Swagger/OpenAPI generated from code
- The docs ARE the contract — keep them honest

Checkpoint 03 – REST Design

Pairs — 5 minutes: design and defend

Q1 Design the endpoint (method + URL + success status) for: 'customer cancels order 17'. Two defensible answers exist — give one and justify it.

Q2 A teammate returns 200 OK with { "error": "product not found" }. Explain to them — kindly — everything wrong with this.

Q3 PUT /api/products/42 with body { "price": 2600 } wiped the product's name. What happened, and which method should have been used?

Q4 Why must POST /api/orders NOT be retried blindly on timeout, while DELETE /api/orders/17 can be?

PART 5

Building & Testing the API in ASP.NET Core

- MVC-style structure: controllers, services, models & DTOs
- Routing, attribute routing and endpoint mapping
- Request handling and JSON response formatting
- Testing with Swagger / OpenAPI and Postman

Clean ASP.NET Core Project Structure

folder layout

```
LankaMart.Api/
  ├─ Program.cs            // startup, DI, middleware
  ├─ appsettings.json      // configuration (no secrets!)
  ├─ Controllers/
  │    └─ ProductsController.cs   // HTTP layer
  ├─ Services/
  │    ├─ IProductService.cs      // contract
  │    └─ ProductService.cs       // business logic
  ├─ Models/
  │    └─ Product.cs              // domain entity
  ├─ Dtos/
  │    ├─ CreateProductDto.cs     // input shape
  │    └─ ProductDto.cs           // output shape
  └─ Repositories/         // data access (next lecture)
```

Why this layout?

- Folders mirror responsibilities — find anything fast
- Each layer depends only on the one below via interfaces
- New feature = one file per folder (Orders → 5 files)
- Scales from assignment to enterprise unchanged
- Markers can navigate it instantly

Models and DTOs — and Why They Differ

```
// Models/Product.cs — internal domain entity
public class Product {
  public long Id { get; set; }
  public string Name { get; set; } = "";
  public decimal Price { get; set; }
  public int Stock { get; set; }
  public string SupplierCostNote = ""; // internal!
}
```

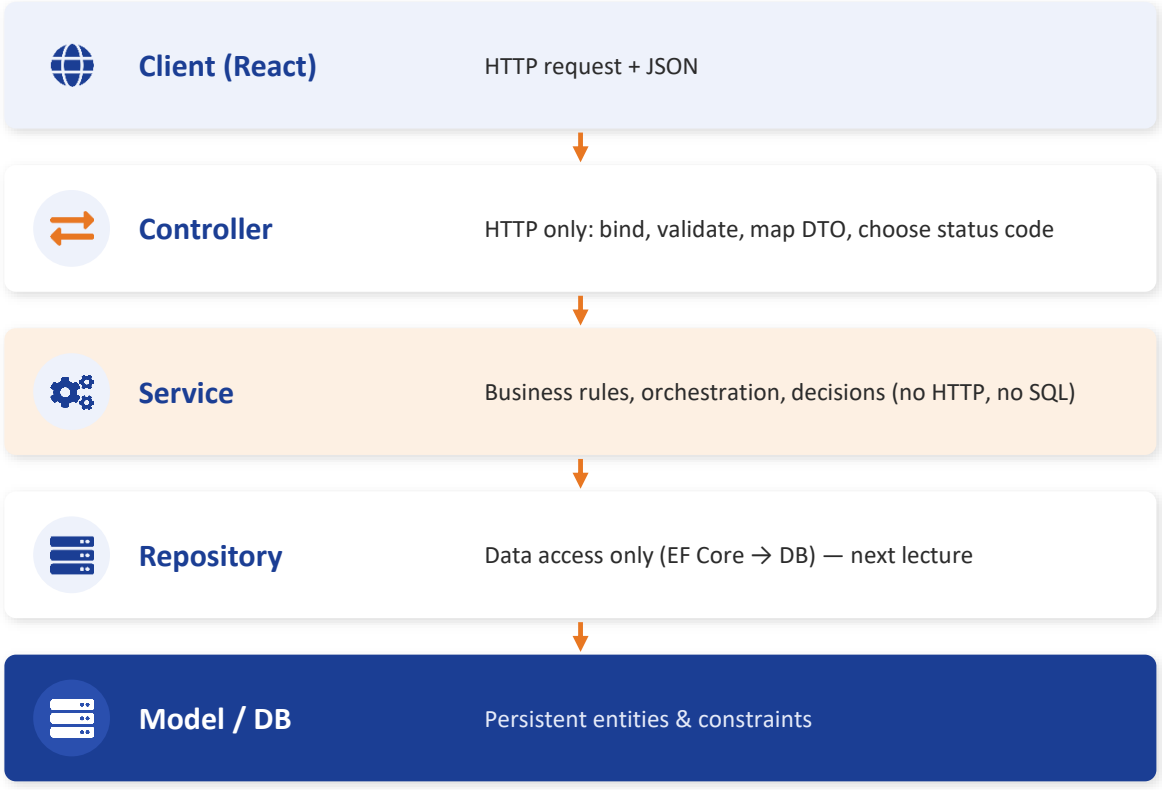
```
// Dtos — what crosses the API boundary
public record CreateProductDto(
  string Name, decimal Price, int Stock);

public record ProductDto(
  long Id, string Name, decimal Price,
  bool InStock);   // no internal fields exposed
```

Model ≠ DTO

- **Model = internal shape (maps to the DB)**
- **DTO = Data Transfer Object = the public contract**
- Hide internal/sensitive fields from clients
- Input DTO ≠ output DTO (no Id on create)
- Decouples API from database changes
- record = concise, immutable DTO type

Controller → Service → Model Workflow



Separation of concerns

- Each box: one reason to change
- Controller thin, service rich
- Test the service without HTTP
- Swap the repo without touching controllers
- Same pattern as Lecture 03's diagram — now in C#

Controllers, Attribute Routing & Endpoint Mapping

ProductsController.cs

```
[ApiController]                          // model binding + 400s
[Route("api/[controller]")]              // → api/products
public class ProductsController : ControllerBase
{
    private readonly IProductService _svc;
    public ProductsController(IProductService svc)
        => _svc = svc;                   // injected by DI

    [HttpGet]                            // GET api/products
    public async Task<IEnumerable<ProductDto>> GetAll()
        => await _svc.GetAllAsync();

    [HttpGet("{id}")]                    // GET api/products/42
    public async Task<ActionResult<ProductDto>> Get(long id)
    {
        var p = await _svc.GetByIdAsync(id);
        return p is null ? NotFound() : Ok(p);
    }
}
```

Reading it

- [ApiController] auto-returns 400 on invalid models
- [Route] + [Http*] attributes map URL+verb to a method
- [controller] token = class name minus 'Controller'
- Service arrives via constructor injection
- Ok(), NotFound(), Created() set the status code
- async Task<...> for non-blocking I/O

Services and Dependency Injection in Practice

```
// Services/IProductService.cs — the contract
public interface IProductService {
  Task<IEnumerable<ProductDto>> GetAllAsync();
  Task<ProductDto?> GetByIdAsync(long id);
  Task<ProductDto> CreateAsync(CreateProductDto dto);
}

// Services/ProductService.cs — the logic
public class ProductService : IProductService {
  public async Task<ProductDto> CreateAsync(
      CreateProductDto dto) {
    if (dto.Price < 0)
      throw new ValidationException("Price < 0");
    /* map, persist, return ProductDto */
  }
}
```

```
// Program.cs — register once
builder.Services
  .AddScoped<IProductService,
             ProductService>();
```

DI lifetimes

- **Scoped — one per request (most services)**
- Transient — new every time
- Singleton — one for the whole app
- **Depend on the INTERFACE; swap implementations freely**

Reading Route, Query and Body Parameters

```
// 1) Route parameter — part of the path
[HttpGet("{id}")]
public ... Get(long id) { ... }      // GET /products/42

// 2) Query parameters — filtering & paging
[HttpGet]
public ... Search(string? category,
                  int page = 1, int pageSize = 20)
// GET /products?category=it&page=2

// 3) Request body — JSON → DTO
[HttpPost]
public ... Create([FromBody] CreateProductDto dto)
// POST /products  { "name": "Mouse", ... }
```

Binding sources

- **Route → identity (which item)**
- **Query → options (filter, sort, page)**
- **Body → the payload (create/update data)**
- ASP.NET Core binds by name & source automatically
- Defaults make query params optional
- Never read raw JSON by hand — bind to a DTO

Returning Proper Responses & JSON

```
[HttpPost]
public async Task<ActionResult<ProductDto>> Create(
    CreateProductDto dto) {
  var created = await _svc.CreateAsync(dto);
  return CreatedAtAction(nameof(Get),
    new { id = created.Id }, created);  // 201 + Location
}

// helpers map intent → status code
Ok(dto)           // 200  OK
CreatedAtAction   // 201  Created (+ where to find it)
NoContent()       // 204   (successful delete/update)
NotFound()        // 404
BadRequest(err)   // 400
```

JSON by default

ASP.NET Core serializes returned objects to JSON automatically (System.Text.Json). camelCase output by convention — matches JavaScript clients.

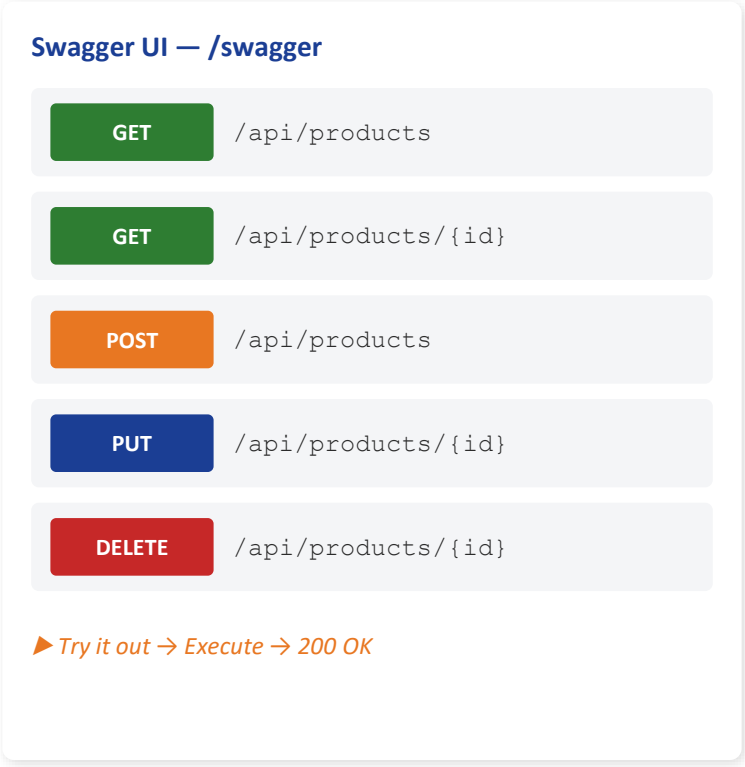
200 OK body

```
{
  "id": 42,
  "name": "Mouse",
  "price": 2500.00,
  "inStock": true
}
```

Match the status code to what happened: created → 201, found → 200, deleted → 204, missing → 404. Don't return 200 for everything.

API Testing with Swagger / OpenAPI

- **OpenAPI = a standard, machine-readable description of your API**
  - Swagger UI renders it as an interactive test page in the browser
- **Ships with the ASP.NET Core Web API template — almost zero setup**
- **Try endpoints live: fill parameters, Execute, see real responses**
- **Auto-documents routes, parameters, DTO shapes and status codes**
- **Frontend & mobile teams can generate client code from the spec**
  - Great first stop before reaching for Postman



Testing with Postman — A Practical Workflow

1	**Set up**	Create a collection per API; an environment var baseUrl = https://localhost:5001
2	**Send requests**	Build GET/POST/PUT/DELETE with headers & JSON body; inspect status, body, time
3	**Save & organise**	Save each request; group by resource; share the collection with the team
4	**Assert**	Add test scripts: status is 200, body has an id, response < 500 ms
5	**Automate**	Run the whole collection (Runner / Newman) in CI on every push

Swagger vs Postman

- Swagger: quick manual checks, lives with the code, always in sync
- Postman: saved collections, scripted assertions, environments, automation
- **Use both: Swagger while coding, Postman for repeatable test suites**

End-to-End: From Browser Click to JSON Response

1 React	User clicks Buy → fetch('POST /api/orders', body)
2 Kestrel + middleware	Routing, CORS, (auth next lecture), model binding & validation
3 Controller	Maps DTO, calls service, picks status code
4 Service	Business rules → calls repository
5 Repository + DB	Persists data, returns entity (next lecture)
6 Response	DTO → JSON → 201 Created → back to React → UI updates

PART 6

Backend Best Practices: Maintainability, Scalability & Security

- Layered architecture & separation of concerns (recap in practice)
- Validation, exception handling & logging
- Secure configuration — no hardcoded secrets
- Writing code that survives the assignment AND the real world

Layered Architecture & DI Best Practices

Respect the layers

- Controller → Service → Repository, one way only
- No SQL in controllers, no HTTP in services
- Each layer testable in isolation

Depend on abstractions

- Inject interfaces, not concrete classes
- Register in Program.cs with the right lifetime
- Swap implementations without edits upstream

Small, focused units

- One class, one responsibility (SRP)
- Short methods with clear verb names
- Feature = one file per folder

Consistency wins

- Same naming, same folder layout everywhere
- Predictable code is maintainable code
- New teammates ramp up fast

Validation, Exception Handling & Logging

```
// 1) Declarative validation on the DTO
public record CreateProductDto(
  [Required, StringLength(120)] string Name,
  [Range(0, 1_000_000)]         decimal Price,
  [Range(0, int.MaxValue)]      int Stock);
// [ApiController] auto-returns 400 + details

// 2) Global exception middleware (Program.cs)
app.UseExceptionHandler();   // one place for errors

// 3) Structured logging via injected logger
_logger.LogInformation("Created product {Id}",
                       created.Id);
```

Principles

- **Validate at the edge; fail fast with 400**
- Handle errors in ONE place, not try/catch everywhere
- Map exceptions → correct status codes
- Log full detail server-side; return safe messages
- **Never expose stack traces to clients**
- Use a traceId to link reports to logs

Secure Configuration & Basic API Security

Configuration & secrets

- **NEVER hardcode secrets or commit them to Git**
- Connection strings, API keys, JWT secrets → environment variables / user-secrets
- appsettings.json for non-secret config only
- Different values per environment (dev/test/prod)
- Production secrets from the platform (Azure Key Vault, etc.)
- Read via IConfiguration / IOptions<T>

Baseline API security

- **HTTPS everywhere (TLS) — encrypt in transit**
- Validate & sanitize all input (prevents injection)
- Authentication + authorization (JWT + RBAC — next lecture)
- Least-privilege database account
- CORS allow-list — only your frontend origin
- Don't leak details in errors; rate-limit sensitive endpoints

Case Study: Designing the LankaMart Backend API

Requirements → API design decisions (with justification — LO4)

Requirement	Endpoint / design	Justification
Browse & search products	GET /api/products?category=&page=	Collection + query params for filtering/paging; cache-friendly
View one product	GET /api/products/{id} → 200 / 404	Identity in the path; correct status codes
Staff add a product	POST /api/products → 201 + Location	Server assigns id; CreatedAtAction; input via CreateProductDto
Update / partial edit	PUT vs PATCH /api/products/{id}	PUT full replace, PATCH partial — avoids accidental field wipe
Place an order	POST /api/orders (service + transaction)	Business logic in service layer; not a controller concern
Clean, layered code	Controllers · Services · DTOs · DI	Maintainable, testable, scalable — meets the rubric

Same method applies to your assignment domain: property management, student management, clinic booking, event ticketing …

Group Activity: Design REST Endpoints (20 min)

Groups of 3–4 · whiteboard or paper · present in 60 seconds

Scenario: Campus Library System

Members borrow books; librarians manage the catalogue; admins manage members. A member can borrow up to 5 books at once.

- **Task 1 — List the resources (nouns) for the system**
- **Task 2 — Design endpoints (method + path) for browse, view one, add, update, delete a book**
- **Task 3 — Design the 'borrow a book' endpoint; which layer holds the 5-book rule?**
- **Task 4 — Pick the right status code for: created, not found, borrow-limit exceeded**

Marking lens (= the rubric)

- Nouns as resources, plural, no verbs in paths?
- Identity in the path, options in the query?
- Correct HTTP method per action?
- Business rule placed in the SERVICE, not the controller?
- Sensible status codes (201 / 404 / 400 or 409)?

Quick Quiz — 6 Questions

1	Which type stores money exactly in C#?	A) double B) float C) decimal
2	What does the [ApiController] attribute give you automatically?	A) a database B) 400 on invalid models C) HTTPS
3	A controller needs IProductService. How does it get one?	A) new ProductService() B) constructor injection (DI) C) a global variable
4	POST that creates a resource should return…	A) 200 OK B) 201 Created C) 204 No Content
5	Why return a DTO instead of the entity?	A) it's faster B) hide internal fields / decouple C) required by C#
6	Where should the JWT signing secret live?	A) appsettings.json in Git B) hardcoded C) environment / secret store

Lecture Summary — Key Takeaways

C# is your backend language

Strongly typed, OOP, modern (properties, var, async, LINQ); decimal for money.

REST is the contract

Resources + HTTP methods + status codes + JSON; design URLs around nouns.

OOP structures the code

Encapsulation, inheritance, polymorphism, abstraction & interfaces drive clean, testable design.

Build it in layers

Controllers (HTTP) → Services (logic) → Repositories (data); DTOs at the boundary.

.NET + ASP.NET Core run it

Cross-platform, fast, batteries-included: DI and middleware out of the box.

Do it well

Validate, handle errors centrally, log, and keep secrets out of code.

Learning Outcomes, Next Lecture & Preparation

LO	Covered today by…
LO2	Writing C# OOP classes and building working ASP.NET Core REST endpoints (controllers, services, models)
LO3	Layered architecture, dependency injection, validation, exception handling, correct HTTP semantics & JSON
LO4	Justifying C#/.NET, ASP.NET Core and REST design choices in the case study and endpoint-design activity



Next: Database, AuthN/AuthZ & Integration

Give the API memory and a lock: PostgreSQL schema design with EF Core, then JWT authentication and role-based access control securing these very endpoints.



Before the lab / next week

- Install the .NET 8 SDK; run dotnet new webapi
- Build & open the project; explore Swagger at /swagger
- Read: Microsoft Learn 'Create a web API with ASP.NET Core'
- Sketch the resource list for YOUR assignment domain

References & Further Reading

- **Official documentation (primary sources)**
  - Microsoft Learn — C# documentation & language reference (learn.microsoft.com/dotnet/csharp)
  - Microsoft Learn — ASP.NET Core fundamentals & 'Create a web API' tutorial
  - Microsoft Learn — Dependency injection, configuration & logging in .NET
  - OpenAPI Specification (openapis.org) · Swagger documentation (swagger.io)
- **Standards & guidelines**
  - RFC 9110 — HTTP Semantics (methods & status codes) · RFC 8259 — JSON
  - Microsoft REST API Guidelines · OWASP API Security Top 10 (owasp.org)
- **Recommended texts**
  - Albahari, J. (2024) C# 12 in a Nutshell. O'Reilly Media.
  - Price, M. J. (2023) C# 12 and .NET 8 – Modern Cross-Platform Development. Packt.
  - Lock, A. (2023) ASP.NET Core in Action, 3rd edn. Manning Publications.

SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 04

Database Design, Authentication, Authorization and Integration

PostgreSQL schema design • JWT authentication • Role-based access control • Secure full-stack integration

Year 3 • Semester 1 • 4 Credits

Lecturer: Eishan Weerasinghe • eishan.w@sliit.lk

Where This Lecture Fits in SE3090

L01	L02	L03*	TODAY	Next
Framework fundamentals & trends	Advanced React frontend	C# / .NET & REST APIs	Database, AuthN/AuthZ & integration	Agentic software development

- **Module content area 2(f)–2(h): database design, authentication & authorization, secure integration**
- Builds on your React frontend (Lecture 02) and ASP.NET Core REST API knowledge
- Directly assessed in lab submissions, the mini hackathon and the Main Assignment (25%)
- Foundation for DevOps deployment topics later (database hosting, environment configuration)

Lecture Learning Outcomes

LO2

Apply suitable frameworks & tools

Design a normalized PostgreSQL schema and integrate it with an ASP.NET Core backend for a full-stack application.

LO3

Use best practices for integration & quality

Implement secure frontend–backend–database integration with validation, safe error handling and secure configuration.

LO4

Select & justify approaches

Justify choices of database design, token-based authentication (JWT) and role-based access control for project requirements.

Why Database Design & Authentication Matter

Data outlives code

Frontends are rewritten every few years; your data and schema live for decades. Poor design today is expensive forever.

Most breaches target data

Broken access control is the #1 risk in the OWASP Top 10. Injection and identification failures also remain in the list.

Performance starts at the schema

No amount of caching rescues a schema with duplicated, inconsistent data and missing indexes.

Every real app has users & roles

E-commerce, banking, LMS, healthcare — all need login, identity and per-role permissions to function legally and safely.

Lecture Roadmap – 3 Hours

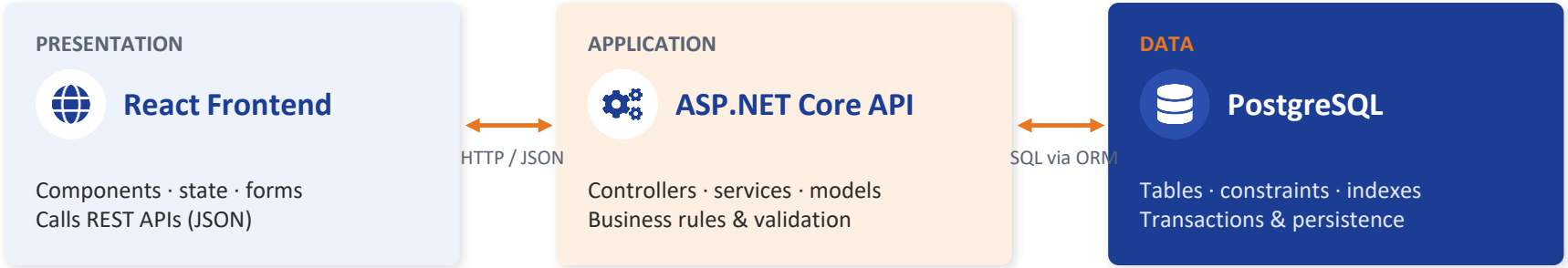
1	**Database design fundamentals** Role of PostgreSQL in full-stack apps · logic vs persistence	~25 min
2	**ER modelling, normalization & schema** Entities · relationships · 1NF–3NF · PostgreSQL schema design	~45 min
3	**Backend ↔ PostgreSQL integration** Connections · env vars · CRUD · repository layer · ORM · pooling	~35 min
4	**Authentication & JWT** AuthN vs AuthZ · sessions vs tokens · JWT flow · token security	~40 min
5	**RBAC & secure integration** Roles & permissions · protected endpoints · validation · safe errors	~35 min

PART 1

Database Design Fundamentals for Full-Stack Applications

- Role of the database in the three-tier architecture
- Relational database concepts refresher
- Why PostgreSQL for modern systems
- Application logic vs data persistence

The Database in Full-Stack Architecture



- **The database is the single source of truth — every tier above it depends on its correctness**
- Stores state that must survive restarts, crashes and deployments (persistence)
- Enforces integrity with constraints even when application code has bugs
- Serves many clients at once: web app, mobile app, reports, background jobs
- The frontend NEVER talks to PostgreSQL directly — always through the API tier

Relational Database Concepts – A Quick Refresher

- **Data is organised into tables (relations) of rows and columns**
  - Row (tuple) = one record, e.g. one customer; Column (attribute) = one fact about it
- **Every table should have a primary key that uniquely identifies each row**
- **Tables are connected through foreign keys — this is the 'relational' part**
- **SQL is the declarative language for defining and querying data**
- **Database transactions follow the ACID principles:**

  Atomicity · Consistency · Isolation · Durability

`customers`

id (PK)	name	email
1	Amal Perera	amal@mail.lk
2	Nadia Silva	nadia@mail.lk

ACID in one line each

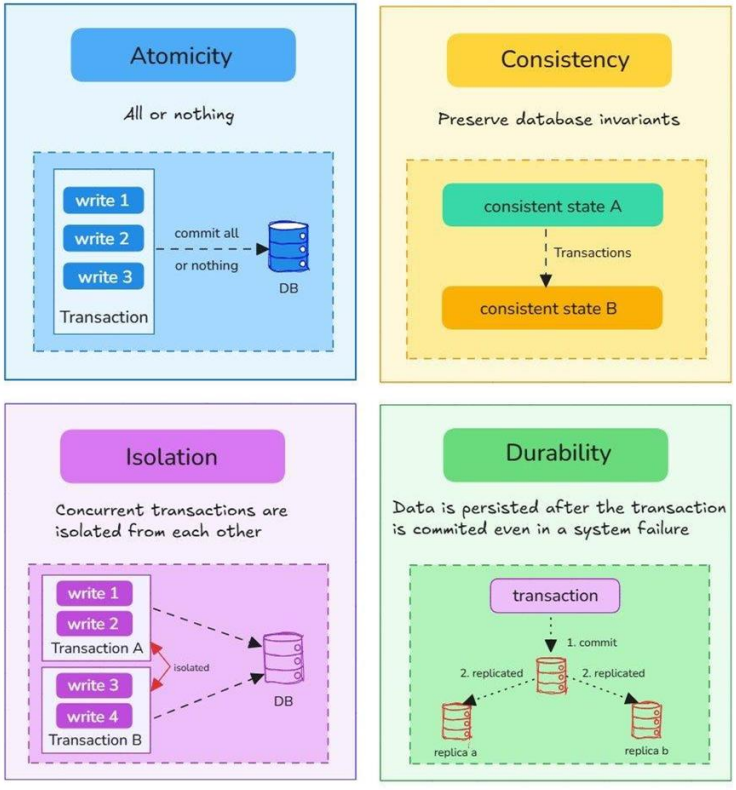
A — the entire operation succeeds or fails.

C — the data always follows the database rules.

I — multiple users can work without interfering with one another.

D — saved data remains available even after a system failure

ACID Principles



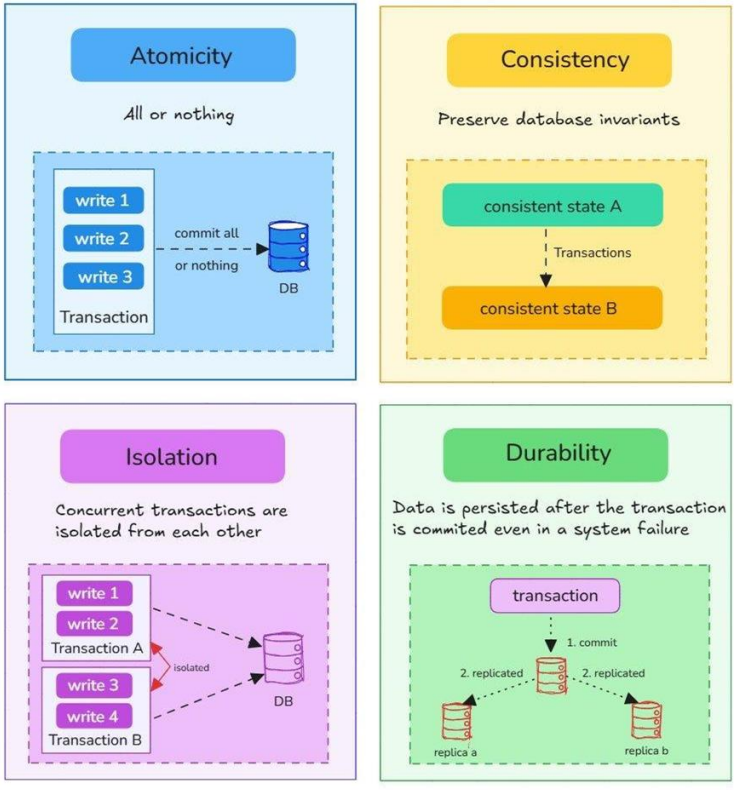
Atomicity — *All or nothing*

- Entire transaction succeeds or fails as a single unit
- If any part fails, database is left completely unchanged
- Prevents partial writes and database corruption

Isolation — *Independent concurrency*

- Concurrent transactions execute without interfering
- Intermediate states are completely hidden from other users
- Provides the illusion of sequential execution

ACID Principles



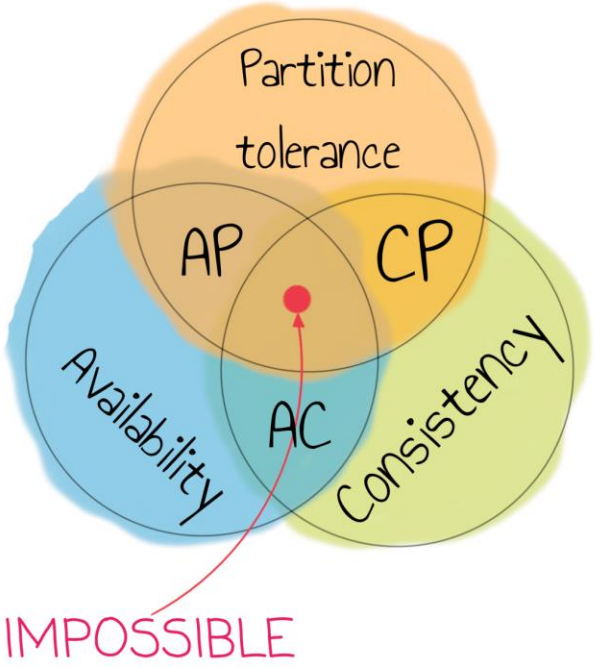
Consistency — *Preserve database invariants*

- Enforces all schema constraints, rules, and triggers
- Database transitions only between valid consistent states
- Prevents illegal data states or foreign key violations

Durability — *Guaranteed persistence*

- Once committed, transaction results are permanent
- Changes survive unexpected power loss or system crashes
- Ensured via write-ahead logging and multi-node replication

CAP Theorem



Consistency (C) — *Absolute correctness*

Every read receives the most recent write, or an error.

Availability (A) — *Non-stop responses*

Every request receives a non-error response, without guaranteeing it contains the most recent write.

Partition Tolerance (P) — *Network resilience*

The system continues to operate despite network delays or dropped connections between nodes.

The Iron Rule: In a distributed system, when a network partition occurs, you must choose between Consistency and Availability.

CAP Theorem

CP Scenario (Consistency + Partition)

- **Prioritizes:** Absolute data correctness and consistency.
- **System Behavior:** Rejects or blocks updates if nodes cannot synchronize, preventing stale reads.
- **Key Tradeoff:** Sacrifices system availability to guarantee data integrity during partitions.
- **Best For:** Banking transactions, financial systems, and billing ledgers.

AP Scenario (Availability + Partition)

- **Prioritizes:** Continuous system uptime and non-stop availability.
- **System Behavior:** Nodes remain fully responsive, returning local data even if it is stale or outdated.
- **Key Tradeoff:** Sacrifices strict consistency; updates propagate eventually once resolved.
- **Best For:** Social media feeds, web caching, and shopping carts.

The Iron Rule: In a distributed system, network partitions are inevitable. You must dynamically choose to prioritize either Consistency (CP) or Availability (AP) during a failure.

Why PostgreSQL for Modern Software Systems?

Open source & free

Permissive licence, no per-core fees; huge community; powers Instagram, Reddit, Skype-scale workloads.

Standards & integrity first

Strong SQL-standard compliance, rich constraints, full ACID transactions, MVCC concurrency.

Rich data types

JSONB for document data, arrays, UUID, full-text search, PostGIS for geo — relational + flexible.

Ecosystem & tooling

First-class support in EF Core (Npgsql), Prisma, Hibernate; runs everywhere — Docker, Azure, AWS RDS, Supabase, Neon.

LO4 link: in your assignment, justify the choice — e.g. "PostgreSQL chosen for ACID guarantees, JSONB flexibility and free cloud hosting tiers."

Application Logic vs Data Persistence

Application logic (API tier)

- Business rules: pricing, discounts, workflows
- Validation of incoming requests
- Orchestration: call services, send emails
- Authentication & authorization decisions
- Changes often — redeployed weekly/daily

Data persistence (DB tier)

- Durable storage of facts (orders, users)
- Structural integrity: keys, constraints
- Transactions & concurrency control
- Efficient retrieval: indexes, query planner
- Changes rarely — schema evolves carefully via migrations

Rule of thumb: the database guarantees data is valid; the application decides what to do with it. Keep business rules out of the schema, and persistence details out of controllers.

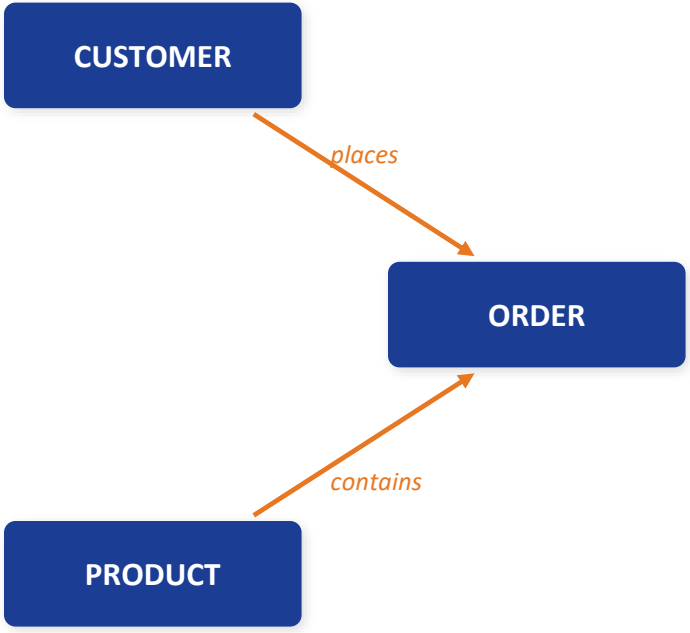
PART 2

Entity Relationships, Normalization & Schema Design

- Entities, attributes and relationship types
- ER diagrams for full-stack applications
- Normalization: 1NF, 2NF, 3NF
- PostgreSQL schema design best practices

Entities, Attributes and Relationships

- **Entity — a real-world 'thing' we store data about**
  - Customer, Product, Order, Category → become tables
- **Attribute — a property of an entity**
  - Customer: id, name, email, phone → become columns
- **Relationship — a meaningful association between entities**
  - A Customer places Orders; an Order contains Products
- **Identify entities from nouns and relationships from verbs in the requirements**



Relationship Cardinalities: 1:1, 1:N, M:N

One-to-one

e.g. each user has one profile



FK with UNIQUE on the child side, or merge into one table.

One-to-many

e.g. one customer, many orders



Most common. FK customer_id **stored** on the **MANY side (orders).**

Many-to-many

e.g. orders ↔ products



Cannot be stored directly — resolve with a **junction table** (order_items).

Key rule: the foreign key always lives on the 'many' side; M:N always becomes a junction (bridge) table.

Primary Keys and Foreign Keys

- **Primary key (PK): uniquely identifies each row — never NULL, never changes**
  - Prefer surrogate keys: id BIGINT GENERATED ALWAYS AS IDENTITY (or UUID)
  - Natural keys (email, NIC) can change — keep them UNIQUE, not primary
- **Foreign key (FK): a column referencing another table's PK**
  - Enforces referential integrity: no order without a real customer
  - ON DELETE behaviour: RESTRICT (default safety) · CASCADE · SET NULL
- **Composite keys: junction tables often use (order_id, product_id) together**

PostgreSQL

```
CREATE TABLE orders (
  id BIGINT GENERATED ALWAYS
     AS IDENTITY PRIMARY KEY,
  customer_id BIGINT NOT NULL
    REFERENCES customers(id)
    ON DELETE RESTRICT,
  status TEXT NOT NULL
    DEFAULT 'PENDING',
  created_at TIMESTAMPTZ
    NOT NULL DEFAULT now()
);
```

Why RESTRICT? Deleting a customer who has orders should fail loudly — losing order history silently (CASCADE) is usually a business disaster.

Surrogate Keys vs Natural Keys

SURROGATE KEY System-generated, meaningless identifier (e.g. Auto-Increment ID, UUID)

When to Use & Advantages

- **Guaranteed Stability:** Never changes, shielding database relationships from shifting real-world data schemas.
- **Performance:** Integer-based keys enable highly optimized indexing, faster joins, and smaller foreign key storage footprint.
- **Universality:** Perfect for entities lacking a clean, immutable natural unique identifier.

Scenario Example

User Accounts: *Storing an internal user ID (e.g.* ***id: 10943****) ensures that if the user updates their registered email address, no foreign key joins cascade or break across order history tables.*

NATURAL KEY Real-world attribute with inherent business meaning (e.g. ISBN, SSN)

When to Use & Advantages

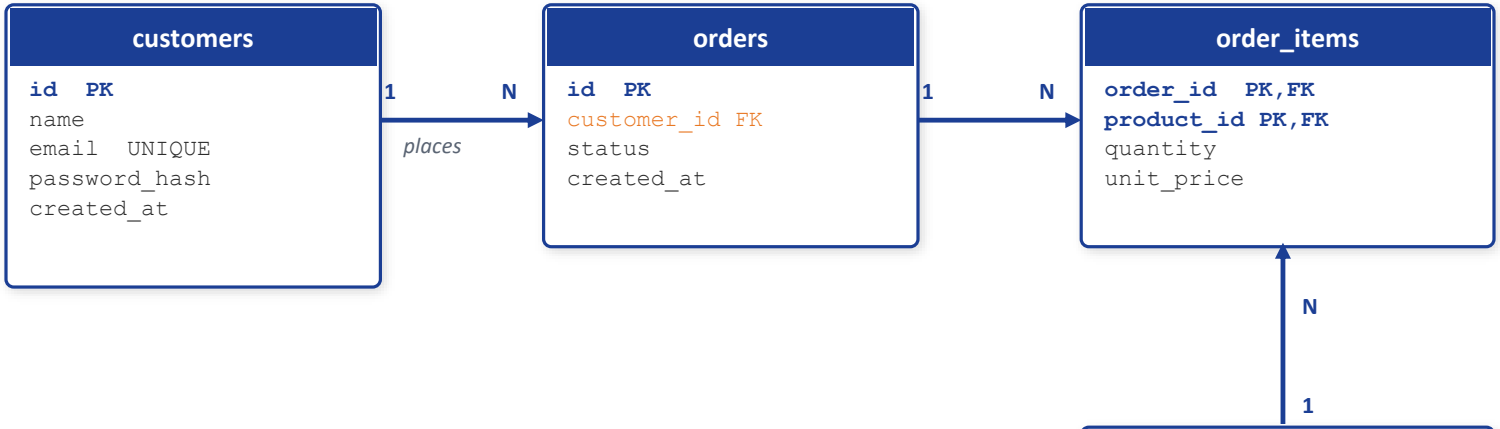
- **Inherent Uniqueness:** Leverages globally unique, strictly regulated standards already guaranteed by business rules.
- **Simpler Queries:** Reduces SQL JOIN density because parent natural key attributes are already directly readable in child tables.
- **Schema Cleanliness:** Avoids creating redundant artificial columns when a completely stable natural ID exists.

Scenario Example

Book Inventory: *Using* ***ISBN-13*** *as the Primary Key. The ISBN of a physical book is globally unique, legally immutable, and standardized worldwide, making an auto-increment ID redundant.*

Core Architectural Rule: When in doubt, prefer **Surrogate Keys** for relational flexibility. If utilizing a Natural Key, ensure the attribute is strictly immutable; if a natural unique attribute is prone to updates (e.g. Email, Username), enforce uniqueness via a **UNIQUE constraint** instead of a Primary Key.

ER Diagram – LankaMart E-Commerce Example



Read the diagram:

- orders↔products was M:N — resolved by order_items
- unit_price copied into order_items: prices change, history must not

Normalization & First Normal Form (1NF)

- **Normalization = organising tables to reduce redundancy and prevent anomalies**
  - Insert, update and delete anomalies all come from storing one fact in many places
- **1NF rules:**
  - Every column holds atomic (indivisible) values — no lists in a cell
  - No repeating groups (phone1, phone2, phone3 …)
  - Each row is unique (has a primary key)
- **Fix: move multi-valued data into its own related table**

✗ **Violates 1NF**

order_id	products
501	Mouse, Keyboard, USB Hub
502	Monitor

✓ **1NF — atomic rows in a child table**

order_id	product	qty
501	Mouse	1
501	Keyboard	1
501	USB Hub	2
502	Monitor	1

Why it matters: try writing SQL to find 'all orders containing a Keyboard' against the comma-separated version — slow LIKE scans, broken counts, impossible joins.

Second and Third Normal Form (2NF & 3NF)

2NF — no partial dependency

- Applies to tables with **composite primary keys**
- Every non-key column must depend on the **entire primary key**, not only part of it

  order_items(order_id, product_id, qty, product_name)
- product_name depends only on product_id, so it should be moved to the ***Products table.***

3NF — no transitive dependency

- Non-key columns must not depend on other non-key columns

  orders(id, customer_id, customer_city)

  – *customer_city* depends on the customer, not the order, so it should be stored in the **Customers table**.

Practical guidance

- 3NF is the standard target for transactional (OLTP) full-stack applications
- Each table should describe exactly ONE entity; each fact stored exactly once
- Denormalize only deliberately, for measured performance reasons — and document why (LO4)

Worked Example: Un-normalized → 3NF

✗ One flat 'orders' spreadsheet-style table

order	customer	email	city	product	cat	price	qty
501	Amal	amal@m.lk	Galle	Mouse	IT	2500	1
501	Amal	amal@m.lk	Galle	Keyboard	IT	5500	1
502	Nadia	nadia@m.lk	Kandy	Mouse	IT	2500	2

Redundancy everywhere: Amal's details ×2, Mouse price ×2 → update/insert/delete anomalies

✓ Decomposed to 3NF (our LankaMart schema)

`customers`

id	name	city
1	Amal	Galle
2	Nadia	Kandy

`products`

id	name	price
10	Mouse	2500
11	Keyboard	5500

`orders`

id	cust_id
501	1
502	2

`order_items`

ord	prod	qty
501	10	1
501	11	1
502	10	2

Each fact lives exactly once; JOINs reassemble the original view when needed.

Normalize or Not?

Trade-offs of Schema Normalization

▲ Advantages (Why we normalize):

- **Eliminates redundancy:** Saves storage and keeps each fact in exactly one place.
- **Data Integrity:** Avoids update, insert, and delete anomalies completely.
- **Simple Updates:** Writing or updating transactional data is extremely fast and consistent.

▼ Disadvantages (The cost):

- **Performance:** Requires complex SQL JOINs to reassemble the original entity view.
- **Overhead:** More tables and relationships mean higher architectural complexity.

Scenarios: When to Choose What

✓ When to Normalize (OLTP Target):

- **Transactional Applications:** Standard target for transactional, full-stack database engines.
- **Frequent Writes:** Systems with highly frequent updates where inconsistencies cannot be tolerated.
- **Strict Consistency:** Financial systems, user profiles, or core operational data stores.

✕ When to Denormalize (OLAP/Scale):

- **Data Warehousing & Dashboards:** Read-heavy analytics where query performance dominates.
- **High-Scale Read APIs:** To eliminate heavy JOIN bottlenecks, do so deliberately and measure results.

PostgreSQL Schema Design Best Practices

Naming & types

- snake_case, plural table names (orders)
- TEXT over VARCHAR(n) unless limits matter
- NUMERIC for money — never FLOAT
- TIMESTAMPTZ (UTC) for all timestamps

Constraints = free correctness

- NOT NULL by default; allow NULL deliberately
- UNIQUE for emails, usernames, codes
- CHECK (price >= 0), CHECK (qty > 0)
- FKs on every relationship

Indexes

- PK & UNIQUE are indexed automatically
- Index FK columns and frequent WHERE/JOIN columns
- Don't over-index — each index slows writes
- EXPLAIN ANALYZE to verify usage

Evolution & safety

- Schema lives in versioned migrations (EF Core)
- Never edit production tables by hand
- Seed data scripts for dev/test
- One schema diagram in your repo & report

Example: LankaMart Core Schema in PostgreSQL

schema.sql

```
CREATE TABLE customers (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now());

CREATE TABLE products (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL,
  price NUMERIC(10,2) NOT NULL CHECK (price >= 0),
  stock_qty INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0));

CREATE TABLE order_items (
  order_id BIGINT REFERENCES orders(id) ON DELETE CASCADE,
  product_id BIGINT REFERENCES products(id),
  quantity INT NOT NULL CHECK (quantity > 0),
  unit_price NUMERIC(10,2) NOT NULL,
  PRIMARY KEY (order_id, product_id));

CREATE INDEX idx_orders_customer ON orders(customer_id);
```

Spot the practices

- Surrogate IDENTITY keys
- UNIQUE email (natural key kept unique, not primary)
- password_hash — never plain text
- NUMERIC for money + CHECK guards
- Composite PK on the junction table
- CASCADE only where children are meaningless alone
- Explicit index on the FK used by every 'my orders' query

Checkpoint 01 – Quick Questions

Answer with your neighbour — 4 minutes

Q1 A table stores course codes as "SE3090; IT2030; SE3050" in one column. Which normal form is violated and how do you fix it?

Q2 students(id, module_id, module_name, grade) with key (id, module_id) — which normal form does module_name break?

Q3 Why does order_items store unit_price when products already has price? Is that redundancy?

Q4 Which side of a one-to-many relationship holds the foreign key — and why can't it be the other side?

PART 3

Backend & Database Integration with PostgreSQL



- Connection configuration & environment variables
- CRUD workflow through layers
- Repository / service pattern & ORM (EF Core + Npgsql)
- Error handling & connection pooling

Connecting the API to PostgreSQL — Securely

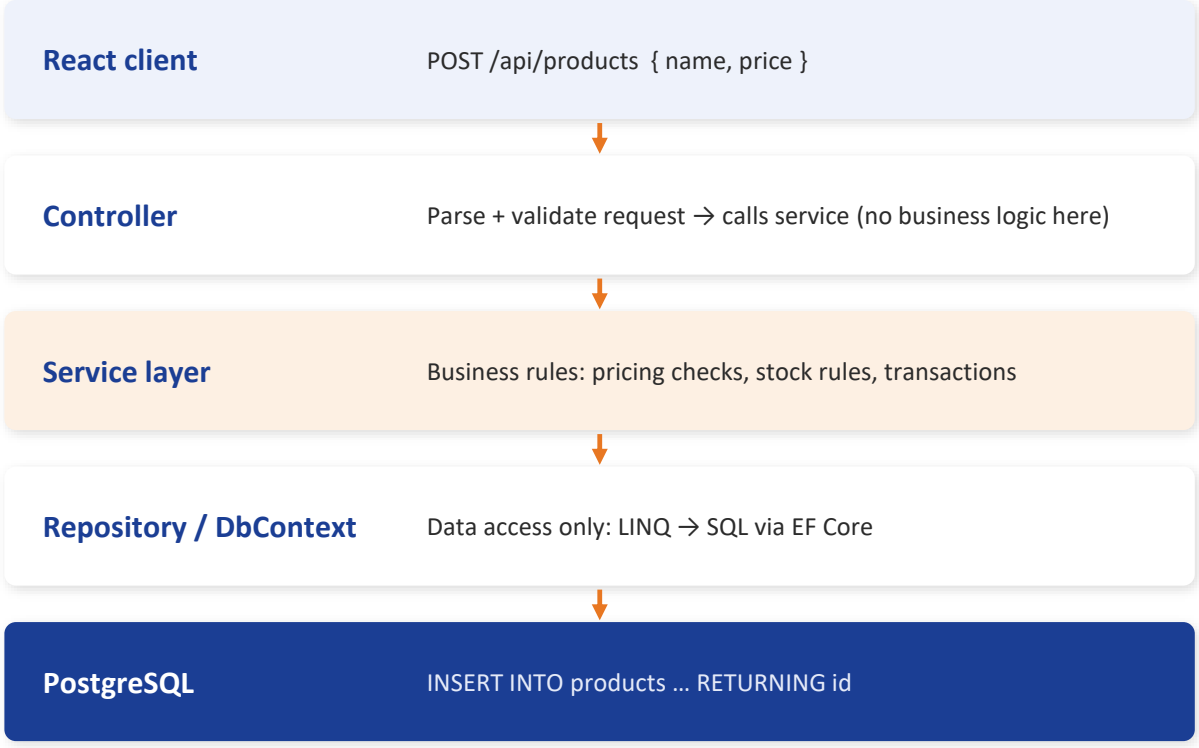
- **A connection string tells the driver where & how to connect**
  - Host, port (5432), database, username, password, SSL mode
- **NEVER hard-code credentials in source code or commit them to Git**
- **Use environment variables / user-secrets per environment**
  - Dev, test and production each get their own values
  - Same code, different config — a 12-Factor App principle
- **Production: secrets come from the platform (Azure App Settings, AWS Secrets Manager)**

```
# .env  (git-ignored!)
DB_CONNECTION="Host=localhost;Port=5432;
  Database=lankamart;Username=app_user;
  Password=${DB_PASSWORD};SslMode=Require"
```

```
// Program.cs
var cs = builder.Configuration
    .GetConnectionString("Default");
builder.Services.AddDbContext<AppDbContext>(
    o => o.UseNpgsql(cs));
```

Checklist: .env in .gitignore · least-privilege DB user (no superuser!) · SSL required · different passwords per environment

CRUD Workflow — Repository / Service Layers



Why layers?

- Separation of concerns — each layer has one job
- Testability: mock the repository, unit-test the service
- Swap implementations (e.g. cache layer) without touching controllers
- CRUD = Create, Read, Update, Delete ↔ INSERT, SELECT, UPDATE, DELETE ↔ POST, GET, PUT, DELETE

ORM Concepts — Entity Framework Core + Npgsql

What is an ORM?

An **Object-Relational Mapper (ORM)** allows a C# application to communicate with a database using **classes and objects**.

In **Entity Framework Core**:

- **classes ↔ tables,**
- **properties ↔ columns,**
- **objects ↔ rows**

- We use **LINQ queries** to retrieve, filter, and sort data.
- We can write queries using LINQ, and **Entity Framework translates** them into parameterized SQL, which helps prevent SQL injection.

EF Core (Npgsql provider)

```
// Entity class ↔ table
public class Product {
  public long Id { get; set; }
  public string Name { get; set; }
  public decimal Price { get; set; }
}

// LINQ → parameterized SQL
var cheap = await db.Products
  .Where(p => p.Price < limit)
  .OrderBy(p => p.Name)
  .ToListAsync();

-- SELECT * FROM products
--  WHERE price < $1 ORDER BY name
```

ORM Concepts — Entity Framework Core + Npgsql

EF Core Features

- **Change tracking** detects modifications to objects, and SaveChanges() updates the database.
- **Migrations** help us manage database schema changes through code.

- **Npgsql** is the PostgreSQL database provider that allows Entity Framework Core to communicate specifically with PostgreSQL.

- **Benefits: productivity, type safety, built-in SQL-injection protection**
- **Costs & cautions:**
  - N+1 query problem — use .Include() deliberately
  - We must still understand SQL and check the generated queries. Problems such as the **N+1 query issue** can reduce application performance

EF Core (Npgsql provider)

```
// Entity class ↔ table
public class Product {
  public long Id { get; set; }
  public string Name { get; set; }
  public decimal Price { get; set; }
}

// LINQ → parameterized SQL
var cheap = await db.Products
  .Where(p => p.Price < limit)
  .OrderBy(p => p.Name)
  .ToListAsync();

-- SELECT * FROM products
--  WHERE price < $1 ORDER BY name
```

Configuring the Model: Constraints

Data Annotations vs Fluent API

EF Core provides two primary ways to configure constraints (Max Length, Required, Column Names, Default values):

- **Data Annotations:** Attributes applied directly to the entity classes. Quick and readable, but mixes domain logic with DB config.
- **Fluent API:** Configured inside the DbContext's OnModelCreating method. Keeps entity classes pure (POCOs) and offers more advanced configuration options not available via attributes.

Best Practice: Use Fluent API configurations separated into individual **`IEntityTypeConfiguration<T>`** classes for clean architecture.

Model Configuration Examples

```
// 1. Data Annotations
public class Product {
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Name { get; set; }
}

// 2. Fluent API (in DbContext)
protected override void OnModelCreating(
ModelBuilder builder) {
    builder.Entity<Product>()
    .Property(p => p.Name)
    .IsRequired()
    .HasMaxLength(100);
}

// 3. Usage
using var db = new AppDbContext();
db.Products.Add(new Product { Name = "Laptop" });
await db.SaveChangesAsync();
```

Database Performance: Creating Indexes

Why use Indexes?

Indexes are special lookup tables that the database search engine can use to speed up data retrieval. Without an index, the DB performs a "Full Table Scan".

- **Primary Keys** are automatically indexed by EF Core.
- **Foreign Keys** are usually indexed automatically, depending on the provider.
- **Custom Indexes:** Should be added to columns heavily used in WHERE, ORDER BY, or JOIN clauses (e.g., Emails, Usernames, Dates).
- **Composite Indexes:** Indexes spanning multiple columns, useful for frequent multi-column queries.
- **Unique Indexes:** Enforces data integrity (e.g., ensuring no duplicate emails).

Creating Indexes (Fluent API)

```
// Creating an Index using Fluent API
protected override void OnModelCreating(
   ModelBuilder mb) {
   // Simple Index
   mb.Entity<User>()
      .HasIndex(u => u.Email);
   // Unique Index
   mb.Entity<User>()
      .HasIndex(u => u.Username)
      .IsUnique();
   // Composite Index
   mb.Entity<Person>()
      .HasIndex(p => new {
         p.FirstName, p.LastName
      });
   // Index using Data Annotation (EF 5.0+)
   // [Index(nameof(Email), IsUnique = true)]
}
```

Relationships: One-to-One

Defining the Relationship

A One-to-One relationship exists when exactly one row in Table A corresponds to exactly one row in Table B. Often used to split wide tables or isolate sensitive data.

- **Navigation Properties:** Both entities contain a reference property to the other entity.
- **Foreign Key:** EF Core requires a Foreign Key on one side of the relationship to determine the dependent entity.
- **Dependent Entity:** In the example, UserProfile is the dependent entity because it holds the UserId FK.

Entity Models (One-to-One)

```
public class User {
   public int Id { get; set; }
   public string Username { get; set; }
   // Reference Navigation Property
   public UserProfile Profile { get; set; }
}

public class UserProfile {
   public int Id { get; set; }
   public string Bio { get; set; }
   // Foreign Key
   public int UserId { get; set; }
   // Reference Navigation Property
   public User User { get; set; }
}

// 3. Usage
using var db = new AppDbContext();
db.Users.Add(new User {
   Username = "alice",
   Profile = new UserProfile { Bio = "Developer" }
});
await db.SaveChangesAsync();
```

Relationships: One-to-Many

The Most Common Pattern

A One-to-Many relationship exists when one row in Table A can be associated with many rows in Table B. (e.g., A Blog has many Posts).

- **Principal Entity:** Contains a Collection navigation property (e.g., ICollection<Post>).
- **Dependent Entity:** Contains a Reference navigation property back to the principal, and a Foreign Key property.
- **Convention:** EF Core automatically discovers this relationship by convention if the naming patterns match.

Entity Models (One-to-Many)

```
public class Blog {
  public int BlogId { get; set; }
  public string Url { get; set; }
  // Collection Navigation Property
  public ICollection<Post> Posts { get; } = new
  List<Post>();
}

public class Post {
  public int PostId { get; set; }
  public string Title { get; set; }
  // Foreign Key
  public int BlogId { get; set; }
  // Reference Navigation Property
  public Blog Blog { get; set; }
}

// 3. Usage
using var db = new AppDbContext();
var blog = new Blog { Url = "dotnet.com" };
blog.Posts.Add(new Post { Title = "EF Core" });
db.Blogs.Add(blog);
await db.SaveChangesAsync();
```

Relationships: Many-to-Many

Implicit Join Tables (EF Core 5.0+)

A Many-to-Many relationship exists when multiple rows in Table A map to multiple rows in Table B. (e.g., Posts and Tags).

- **Modern EF Core:** Handles Many-to-Many implicitly. You only need to define Collection navigation properties on both sides.
- **Join Table:** EF Core automatically generates the intermediate "Join Table" in the database under the hood during migrations.
- **Extra Payload:** If you need to store extra payload data (like a "DateAdded" field) in the join table, you must manually define the join entity class.

Entity Models (Many-to-Many)

```
public class Post {
   public int Id { get; set; }
   public string Title { get; set; }
   // Collection Navigation
   public ICollection<Tag> Tags { get; set; }
}

public class Tag {
   public int Id { get; set; }
   public string Name { get; set; }
   // Collection Navigation
   public ICollection<Post> Posts { get; set; }
}

// EF Core automatically creates a PostTag
// join table with PostId and TagId.

// 3. Usage
using var db = new AppDbContext();
var post = new Post { Title = "Many-to-Many" };
post.Tags = new List<Tag> { new Tag { Name = "EF
Core" } };
db.Posts.Add(post);
await db.SaveChangesAsync();
```

Database Errors & Connection Pooling

Handling database errors

- Database operations may fail due to **timeouts, duplicate values, foreign-key violations, or connection issues.**
- We should catch the **specific database error** and return a meaningful API response. *For example*, PostgreSQL error **23505** means a unique value already exists. The API can return **409 Conflict** with a safe message such as 'Email already registered.
- We should retry only **temporary connection failures**. We must not automatically retry write operations because this could create duplicate records.
- The complete error should be logged on the **server**, but technical or sensitive details should not be shown to the client.

Connection pooling

- A **connection pool** keeps a set of database connections ready for reuse. This improves performance because creating a new connection for every request is expensive.
- **Npgsql manages connection pooling automatically**. When we close a connection, it normally returns to the pool for another request.
- Finally, always dispose of database contexts and connections correctly. Otherwise, connection leaks can exhaust the pool and prevent new requests from accessing the database

PART 4

Authentication, Authorization & JWT

- Authentication vs authorization
- Public vs protected routes; sessions vs tokens
- JWT structure, login flow, access & refresh tokens
- Token storage and common JWT security risks

Authentication vs Authorization

Authentication (AuthN)

"Who are you?"

- Verifying identity: login with email + password
- Also: OAuth (Google login), MFA, biometrics
- Happens first, typically once per session
- Failure → 401 Unauthorized

Authorization (AuthZ)

"What may you do?"

- Checking permissions for an action/resource
- Roles (Admin, Staff, Customer) & policies
- Happens on every protected request
- Failure → 403 Forbidden

Airport analogy: passport control checks WHO you are (authentication); your boarding pass decides WHICH flight and seat you may board (authorization). Both are required — in order.

Protecting Routes: Sessions vs Token-Based Auth

Public vs protected routes

Public: GET /products · POST /auth/register · POST /auth/login
Protected: GET /orders/my · POST /orders · admin endpoints — **require proof of identity on EVERY request**

HTTP is stateless

The server forgets you between requests. After login, every later request must carry proof of identity. Two strategies exist for that proof →

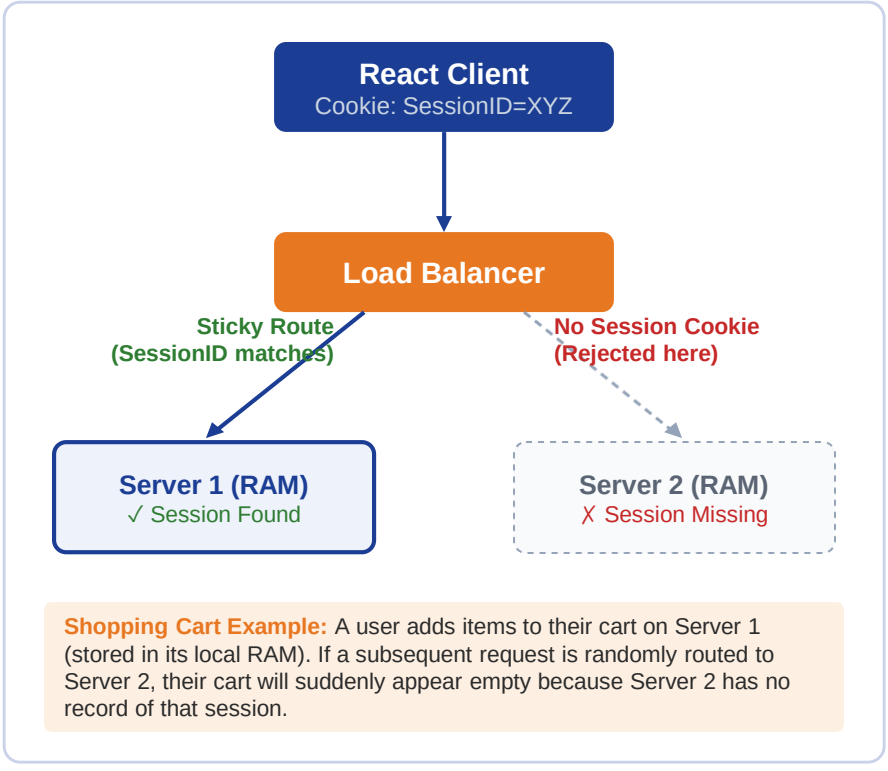
	Server-side sessions	Tokens (JWT)
Where state lives	Server memory/store; client holds a session-id cookie	Inside the signed token on the client; server stores nothing
Scaling	Needs sticky sessions or shared store (Redis)	Any API instance can verify — ideal for multiple servers, mobile, microservices
Revocation	Easy — delete the session	Hard — valid until expiry (hence short lifetimes + refresh tokens)
Typical use	Classic server-rendered web apps	SPAs (React) + REST APIs — our stack

Server-Side Sessions: Sticky Sessions (Session Affinity)

How Session Affinity Works

HTTP is stateless. When scaling horizontally across multiple servers, traditional session state lives in a single server's local RAM. If a client's requests are routed randomly, their session is lost.

- **Affinity Cookie:** The load balancer inspects the response to inject a unique cookie (or tracks the session cookie). On later requests, it routes the client back to the exact same server.
- **The Vulnerability:** If **Server 1** goes down, all sessions stored in its RAM are permanently lost, forcing users to re-login.
- **The Solution:** Use a centralized store like Redis instead of local RAM so any server can verify any session.



Shopping Cart Example: A user adds items to their cart on Server 1 (stored in its local RAM). If a subsequent request is randomly routed to Server 2, their cart will suddenly appear empty because Server 2 has no record of that session.

JSON Web Token (JWT) — Structure

A JSON Web Token, or **JWT**, contains three parts: the **header, payload, and signature.**

HEADER

```
eyJhbGciOiJIUzI1NiIs...
```

```
{ "alg": "HS256",
  "typ": "JWT" }
```

The **header** identifies the token *type* and *signing* algorithm.

PAYLOAD

```
eyJzdWIiOiI0MiIsInJv...
```

```
{ "sub": "42",
  "role": "Customer",
  "exp": 1718200000 }
```

The **payload** contains claims such as the user ID, role, and expiry time. This information is only **Base64-encoded**, not encrypted, so anyone can read it.

SIGNATURE

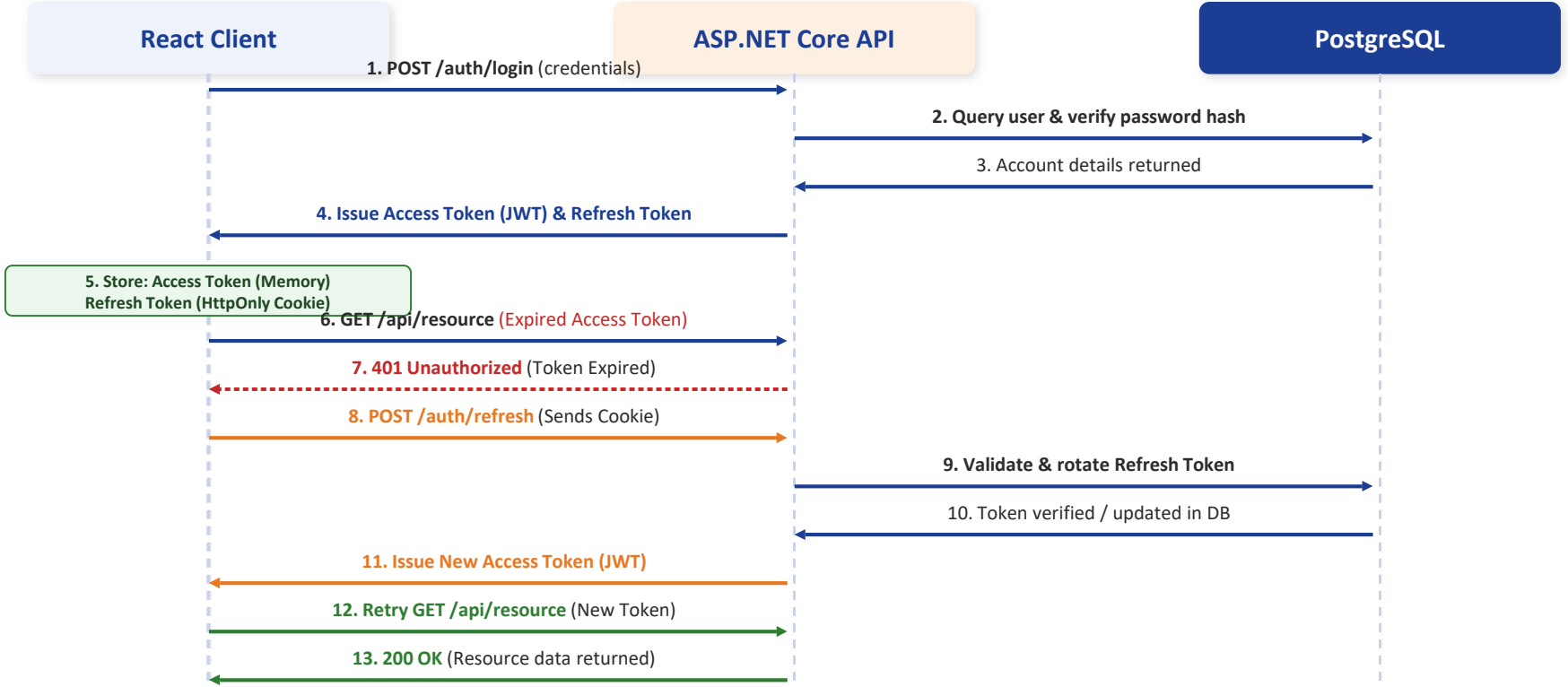
```
SflKxwRJSMeKKF2QT4f...
```

```
HMACSHA256(
  base64(header) + "." +
  base64(payload), SECRET)
```

The **signature** allows the server to verify that the token is authentic and has not been modified.

Golden rule: Never store **passwords or sensitive information** inside a JWT payload

JWT Authentication Flow



Access Tokens, Refresh Tokens & Secure Storage

	Access token	Refresh token
Purpose	Sent with every API request to prove identity	Used only to obtain a new access token
Lifetime	Short: 15–60 minutes	Long: days–weeks
Server state	None (stateless JWT)	Stored hashed in DB → can be revoked (logout, ban, rotation)
If stolen	Damage window limited by expiry	Serious — hence rotation & revocation list

Where should the React app keep tokens?

- localStorage — convenient but readable by ANY JavaScript → exposed if an XSS attack succeeds
- Recommended: refresh token in an HttpOnly, Secure, SameSite cookie (JS cannot read it); access token in memory only
- Whatever you choose: HTTPS always, short expiry, and never log tokens

Common JWT Security Risks & Mitigations

Risk	What goes wrong	Mitigation
alg: none / weak alg	Attacker strips or downgrades the signature algorithm	Server pins accepted algorithms; reject 'none'; use vetted libraries
Weak / leaked secret	Short secrets brute-forced offline → forged tokens	Long random key (256-bit+), env vars, rotation plan
No expiry validation	Stolen tokens work forever	Always set & validate exp; keep lifetimes short
Sensitive data in payload	Payload is Base64 — readable by anyone	Only ids, roles, expiry — never passwords/PII
Token theft via XSS	Injected script reads localStorage tokens	HttpOnly cookies, CSP, input/output sanitization
Trusting client claims	Role checked in React only — attacker calls API directly	Authorization enforced server-side on EVERY endpoint

The client is enemy territory: hiding a button in React is UX, not security. Every rule must also exist in the API.

Checkpoint 02 – Discussion

Discuss in pairs — 5 minutes, then we debate

D1 Your API returns 401 for an expired token AND for a wrong password. A teammate says both should be 403. Who is right, and why does it matter?

D2 A friend proposes: "Make tokens valid for 30 days so users never get logged out." Argue for and against. What design gets both convenience and safety?

D3 You can read your lecturer's JWT payload with a Base64 decoder. Is that a vulnerability? Justify using the signature mechanism.

D4 What is the WORST place mentioned today to store: (a) the JWT signing key, (b) user passwords, (c) the DB connection string?

PART 5

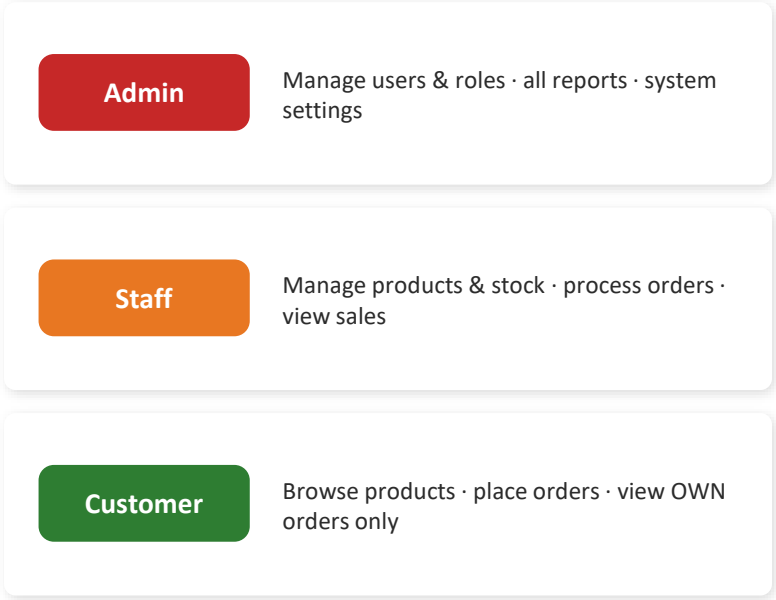
Authorization, Secure Integration & Practice

- Role-based access control & schema
- Protecting endpoints
- SQL injection & validation
- Safe errors • case study • activity • quiz



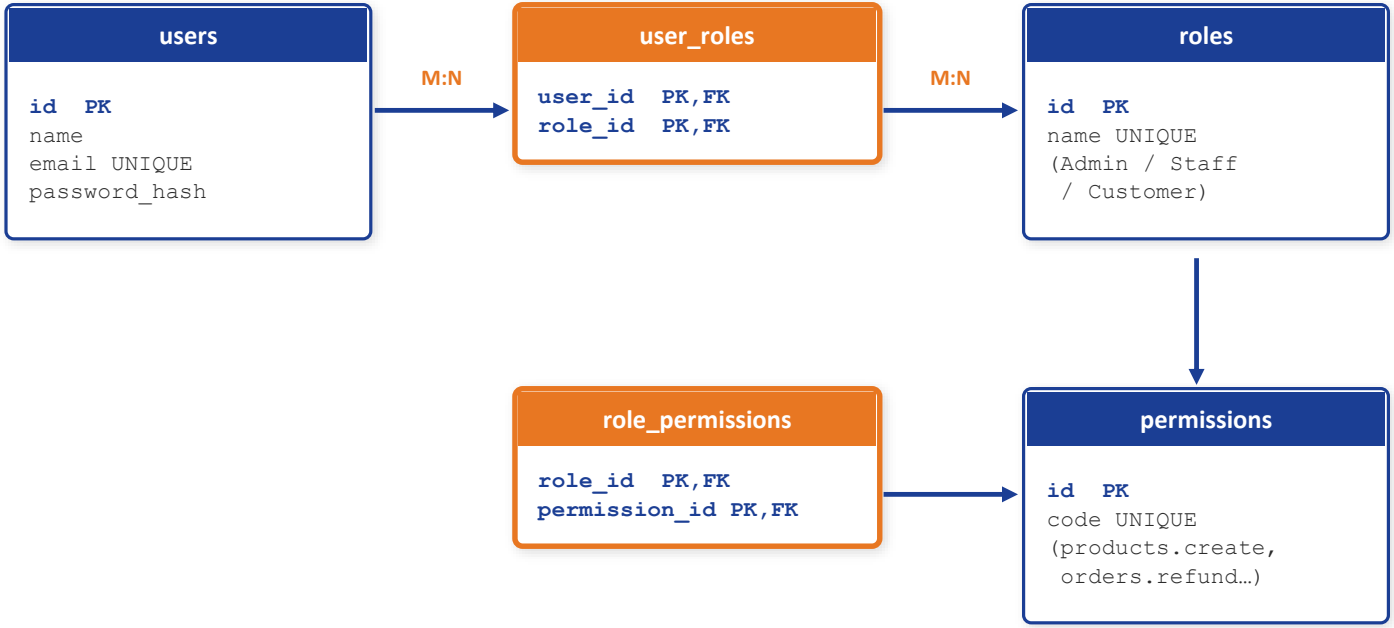
Role-Based Access Control (RBAC)

- **Permissions attach to roles, not to individual users**
  - Users gain permissions by being assigned roles
  - Manage 3 roles instead of 3,000 users' individual rights
- **Principle of least privilege: each role gets the minimum it needs**
- **It is easier to manage because a new employee can simply be assigned the appropriate role.**
- **For more control, roles can contain specific permissions such as products.create or orders.refund.**



Note "own orders only": Roles alone cannot enforce **"own orders only."** The API must compare the order's **customer ID** with the **authenticated user's ID in the JWT.**

Mapping RBAC to PostgreSQL Tables



Same pattern twice!

users↔roles and roles↔permissions are both M:N — resolved with junction tables, exactly like order_items.

At login: load the user's roles → embed as claims in the JWT → no DB lookup on later requests (until refresh).

Protecting API Endpoints by Role — ASP.NET Core

ProductsController.cs

```
[ApiController]
[Route("api/products")]
public class ProductsController : ControllerBase {

  [HttpGet]                      // public route
  [AllowAnonymous]
  public Task<IEnumerable<ProductDto>> GetAll() ...

  [HttpPost]                     // Staff or Admin
  [Authorize(Roles = "Staff,Admin")]
  public Task<IActionResult> Create(CreateProductDto d) ...

  [HttpDelete("{id}")]           // Admin only
  [Authorize(Roles = "Admin")]
  public Task<IActionResult> Delete(long id) ...
}
```

Request pipeline

1 JWT middleware: signature & expiry valid? → else 401

2 [Authorize]: authenticated? → else 401

3 Role/policy check passed? → else 403

4 Object-level check in service: owns this resource?

5 Controller action runs

Policies > raw roles for complex rules: AddPolicy("CanRefund", p => p.RequireClaim("perm", "orders.refund"))

Secure Request Flow — Frontend → Backend → Database

React (browser)

- HTTPS only
- Bearer token via interceptor
- Client-side validation (UX)
- Holds no secrets, no SQL

ASP.NET Core API

- CORS allow-list (your origin only)
- JWT validation → roles + ownership
- Server-side validation & sanitization
- Parameterized queries via EF Core

PostgreSQL

- SSL connection, private network
- Least-privilege app_user
- Constraints as last line of defence
- Only hashed passwords stored

Defence in depth: every layer assumes the previous one failed. Browser checks can be bypassed; the API re-validates; the database constrains anyway.

Validation, Sanitization & Preventing SQL Injection

- **Validate on BOTH sides — for different reasons**
  - Frontend validation provides quick feedback but can be **bypassed**.
  - Backend validation is the real security layer; check the type, format, range, and business rules.

- **ASP.NET Core supports validation through data annotations and FluentValidation.**
  - [Required], [EmailAddress], [Range(0, 1000000)]

- **React normally encodes displayed content, helping to prevent XSS attacks.**

- **SQL injection occurs when untrusted input is joined directly with an SQL command.**
  - Never build SQL queries using **string concatenation**.
  - Use **parameterized** queries or **EF Core LINQ queries**.

SQL injection

```
-- VULNERABLE: string concatenation
var sql = "SELECT * FROM users"
  + " WHERE email='" + input + "'";
-- input:  ' OR '1'='1   → returns ALL users

-- SAFE: parameterized raw SQL
db.Users.FromSqlInterpolated(
  $"SELECT * FROM users WHERE email={input}");

-- SAFE: LINQ (parameterized by EF Core)
db.Users.Where(u => u.Email == input);
```

Parameters travel SEPARATELY from the SQL text — the database can never mistake data for commands. Hand-escaping is not a substitute.

Error Handling Without Leaking Secrets

WRONG — what the client should never see

```
HTTP 500
Npgsql.PostgresException (0x80004005):
23505: duplicate key violates unique
constraint "users_email_key"
at LankaMart.Repos.UserRepo.Insert()
in C:\src\api\UserRepo.cs:line 42
```

Leaks table & constraint names, file paths, stack frames, driver versions — free reconnaissance for attackers.

RIGHT — safe, structured error response

```
HTTP 409 Conflict
{
  "type":  ".../errors/email-taken",
  "title": "Email already registered",
  "status": 409,
  "traceId": "00-8f3a...-01"
}
```

Problem Details format (RFC 9457) — built into ASP.NET Core. traceId links a user report to the full server-side log entry.

The pattern

- Global exception middleware: log full detail server-side → return generic Problem Details to the client
- Map known cases to proper codes: validation → 400 · auth → 401/403 · not found → 404 · conflict → 409 · unknown → 500

Case Study: LankaMart — Putting It All Together

Requirements → design decisions, with justification (LO4)

Requirement	Design decision	Justification
Customers register & log in	users + bcrypt hash; JWT access (15 min) + rotating refresh token	Stateless scaling for the SPA; revocation via refresh store
Orders contain many products	orders + order_items junction, composite PK, snapshot unit_price	M:N resolved; invoices immune to later price changes
Staff manage catalogue, admins manage staff	roles + user_roles; [Authorize(Roles=…)] on endpoints	Least privilege; policy change = edit role, not 3,000 users
Customers see only their own orders	Ownership check in service: order.customer_id == token sub	Role checks alone can't prevent object-level access (OWASP #1)
Stock never negative; emails unique	CHECK (stock_qty >= 0); UNIQUE(email)	Constraints enforce truth even past application bugs

Same method applies to your assignment domain: property management, student management, clinic booking …

Group Activity: Schema & Auth Flow (20 min)

Groups of 3–4 · paper or laptop · present in 60 seconds

Scenario: University Library System

Students borrow books; librarians manage the catalogue and loans; admins manage user accounts. A student may borrow up to 5 books; a book can have many copies.

- **Task 1 — ER diagram: entities, keys, cardinalities (resolve any M:N)**
- **Task 2 — RBAC: roles table + 2 protected endpoints with required roles**
- **Task 3 — Auth flow: sketch login → JWT → borrow-a-book request**
- **Task 4 — One constraint that enforces a business rule in PostgreSQL**

Marking lens (same as the assignment)

- Junction tables where needed?
- FKs on the correct side?
- 3NF — or justified exceptions?
- 401 vs 403 used correctly?
- Object-level check identified? (a student sees only own loans)

Quick Quiz — 5 Questions

1 Which normal form is violated when module_name sits in a table keyed by (student_id, module_id)?

A) 1NF B) 2NF C) 3NF

2 An order_items table linking orders and products is an example of resolving a …

A) 1:1 B) 1:N C) M:N relationship

3 A valid but expired JWT arrives at a protected endpoint. The correct response is …

A) 400 B) 401 C) 403

4 Which part of a JWT prevents tampering?

A) Header B) Payload C) Signature

5 The single most effective defence against SQL injection is …

A) Input length limits B) Parameterized queries C) Hiding error messages

Lecture Summary — Key Takeaways

Design the data first	3NF schemas with proper keys, junction tables for M:N, constraints and FK indexes — in PostgreSQL, with migrations.
Integrate through layers	Controller → service → repository → EF Core/Npgsql; env vars for connection strings; pooled connections; typed error handling.
Authenticate with JWT	Stateless signed tokens: short-lived access + revocable rotating refresh; bcrypt-hashed passwords; secure token storage.
Authorize everywhere	RBAC via roles/junction tables, [Authorize] policies on every endpoint, PLUS object-level ownership checks.
Defend in depth	Validate both sides, parameterized queries only, safe Problem-Details errors, least-privilege DB users, secrets in env config.

Learning Outcomes, Next Lecture & Preparation

LO	Covered today by…
LO2	Designing PostgreSQL schemas and integrating them with ASP.NET Core (EF Core + Npgsql) to build full-stack applications
LO3	Best practices for secure integration: layered architecture, env configuration, validation, parameterized queries, safe error handling
LO4	Selecting and justifying PostgreSQL, JWT-based authentication and RBAC against project requirements (case study & activity)



Next: Agentic Software Development (1 of 3)

AI agents in engineering workflows, AI-assisted design and coding — you will use today's schema and auth design as the playground for AI-assisted implementation.



Before the lab / next week

- Install PostgreSQL 16 + pgAdmin (or Docker image)
- Sketch the ER diagram for YOUR assignment domain
- Read: EF Core 'Getting Started with PostgreSQL' (Npgsql docs)
- Skim OWASP Top 10 — A01 & A07

References & Further Reading

- **Module-recommended texts**
  - Freeman, E. & Robson, E. (2020) Head First Design Patterns, 2nd edn. O'Reilly Media.
  - Ford, N., Richards, M., Sadalage, P. & Dehghani, Z. (2021) Software Architecture: The Hard Parts. O'Reilly Media.
  - Biswas, A. & Talukdar, W. (2025) Building Agentic AI Systems. Packt Publishing.
- **Documentation (primary sources)**
  - PostgreSQL 16 Documentation — postgresql.org/docs (DDL, constraints, indexes)
  - Npgsql & EF Core PostgreSQL provider — npgsql.org
  - Microsoft Learn — ASP.NET Core Security: Authentication & Authorization
- **Security references**
  - OWASP Top 10 (2021) & OWASP API Security Top 10 (2023) — owasp.org
  - RFC 7519 (JSON Web Token) · RFC 9457 (Problem Details for HTTP APIs)
  - OWASP Cheat Sheet Series: SQL Injection Prevention · Password Storage

SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 05

Agentic AI Fundamentals – Part 1

Year 3 • Semester 1 • 4 Credits

Lecturer: Lasal Hettiarachchi | lasal.h.visiting@sliit.lk | Course web: courseweb.sliit.lk

Today's Roadmap – 3 Hours

Hour 1	**Frameworks & Fundamentals** Introduction to Agentic AI | Frameworks | Fundamental knowledge about Models
Hour 2	**MODEL + TOOLS + LOOP + GOALS** Effective tool usage in Agentic AI
Hour 3	**Composition** Quick intro to the LABS

Short breaks after each hour • Questions welcome at any point

PART 1 OF 4

Fundamentals Of LLMs

- How LLMs work
- Limitations of LLM workflows
- What is agentic AI
- Agentic AI spectrum



Agent vs Model

Prompt

"Find me a flight to Singapore next Friday under $400 and hold a seat."

smart_toy

Traditional LLM Model

Provides information and static advice, but cannot take action.

"Here are some tips for finding cheap flights..."

task_alt

AI Agent

Executes tasks, calls APIs, and performs real-world actions to achieve goals.

"Held seat 34C on TR 561, Confirmation #8842"

Small pieces of words combine to form sentences



Tokens

Tokens are the foundational units of text processing in NLP, created by breaking down raw text into smaller components like words, subwords, or even characters.



Embeddings

Embeddings are the result of training vectorized tokens to capture semantic relationships between words. Unlike raw vectors, embeddings position similar words (e.g., "king" and "queen") closer in vector space, allowing models to understand context and meaning.

https://medium.com/@saschametzger/what-are-tokens-vectors-and-embeddings-how-do-you-create-them-e2a3e698e037

Small pieces of words combine to form sentences

Tokens

Example sentence: **“The cat eats fish.”**

1. Tokens split the sentence

["The", "cat", "eats", "fish"]

Each token receives an ID:

The → 10 | eats → 40
cat → 25 | fish → 61

However, the number **25** does not explain what “cat” means.

Embeddings

The **model** converts each **token ID** into a **vector:**

cat → represents an animal
eats → represents an action
fish → represents food in this context

The embedding for **“cat”** will be more similar to **“dog”** than to **“car.”**

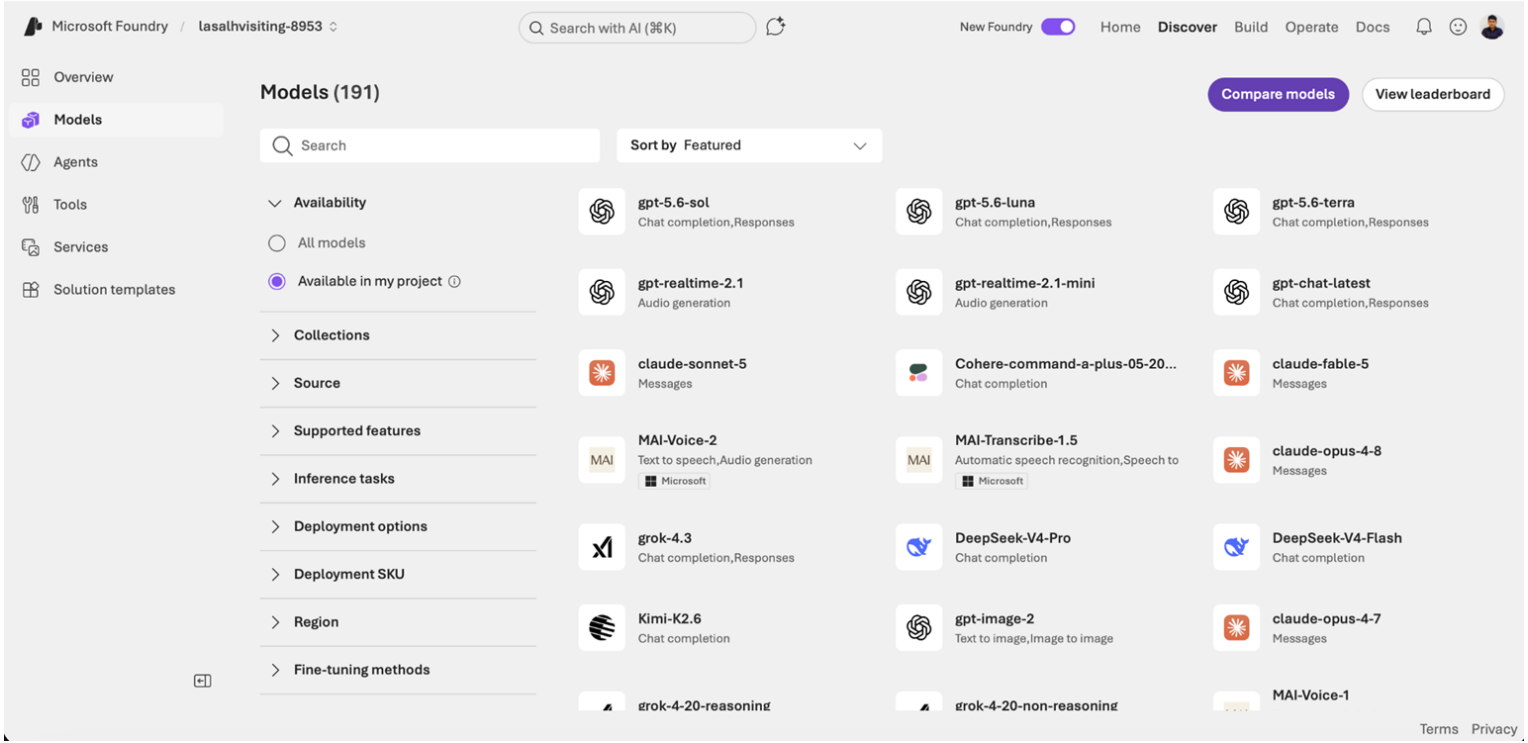
- **Tokens** help the model separate and identify the words.
- **Embeddings** help the model understand their meanings and relationships.

What is an LLM, really?

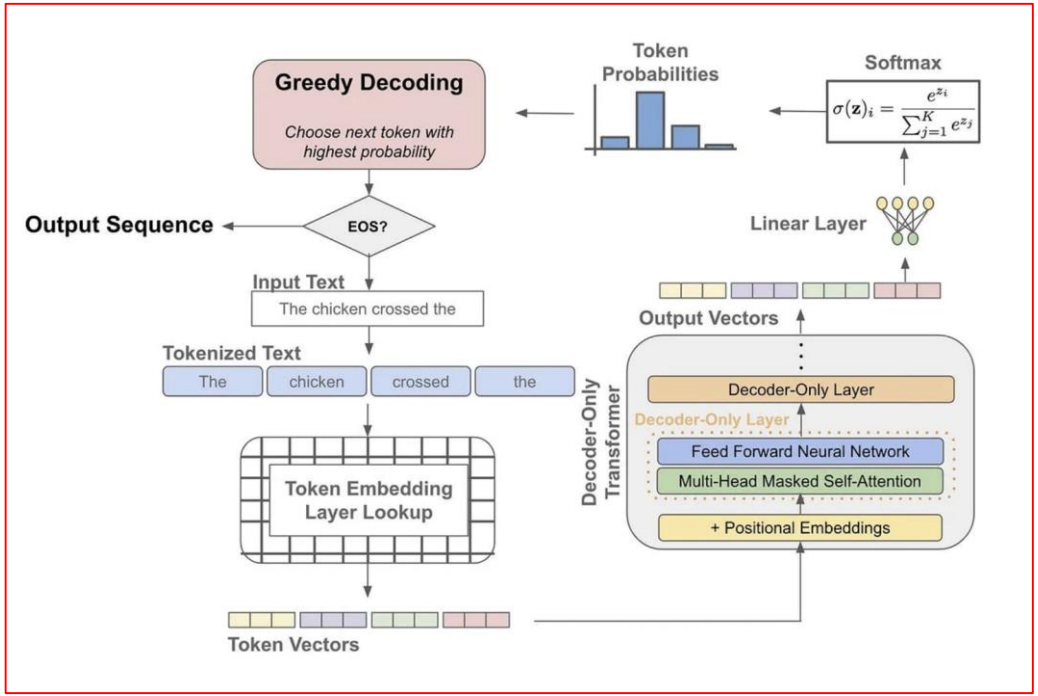
Definition: A neural network trained to predict the next token given the previous tokens. Usually the first set of seed text (user prompt) and the system message is provided.

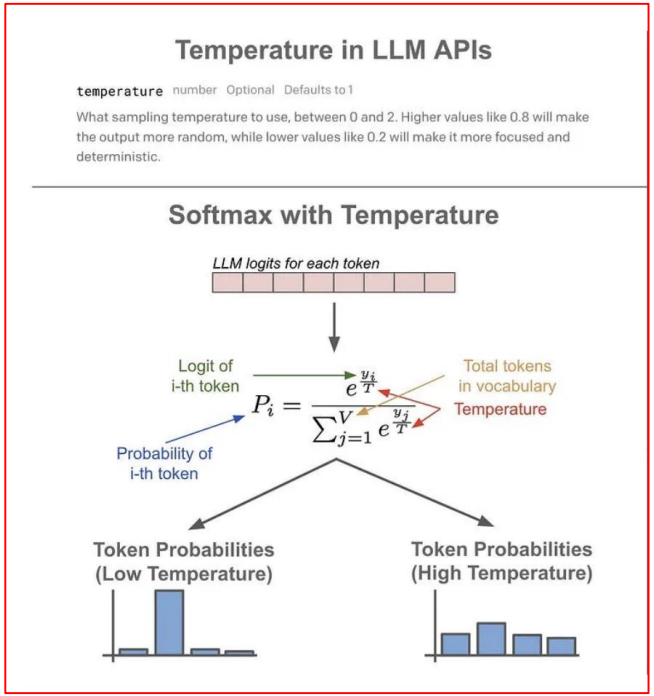
- "Token" ≈ a chunk of text (~¾ of a word). Text is split into tokens first.
- Trained in two broad stages (high level):
  - **Pre-training** — read enormous amounts of text → learn language & world patterns.
  - **Alignment / instruction-tuning** — learn to *follow instructions* and be helpful.
- At inference it is, fundamentally, a **function**: `tokens in → next token out`, applied repeatedly to generate text.

Model Catalog



What is an LLM, really?





What is an LLM, really?

“The chicken crossed the… ? ”

1. The sentence is first divided into **tokens**.
2. Tokens are converted into **embeddings** so the model can process their meanings.
3. **Transformer layers** examine the context and relationships between tokens.
4. The model gives every possible next tokens raw scores called **logits**.
5. A **higher logit** means the ***token is more suitable***.
6. **Softmax** converts the logits into **probabilities** that total **100%**.. (road → 70% , street → 20% , river → 10%)
7. The model selects **“road,”** adds it to the sentence, and repeats the process.

Simple flow: **Text → Tokens → Embeddings → Probabilities → Select next token → Repeat**

From hidden state to a probability

At each step the network emits a vector of logits — one raw score zi for every token i in the vocabulary V.

- **Hidden state** is the model's internal understanding of the text so far.
- Logits become a probability distribution via the **softmax**:

$$p_i = \frac{e^{z_i}}{\sum_{j \in V} e^{z_j}}$$

- Each $p_i \in (0, 1)$ and $\sum_i p_i = 1$ — a valid distribution.
- Exponentiation makes bigger logits dominate; it's monotonic, so ranking is preserved.

Temperature sharpening vs flattening

- Temperature T rescales the logits **before** softmax:

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- $T \to 0$: the largest logit dominates → distribution collapses to **argmax** (greedy, deterministic).
- $T = 1$: the model's "natural" distribution.
- $T \to \infty$: logits flatten → **uniform** (maximum randomness).

Temperature sharpening vs flattening

- Temperature (T) changes the logits **before softmax**.
- It **controls** how predictable or varied the output will be.
  - **Low temperature ((T<1))**: *probability differences become larger*. The highest-scoring token dominates.
  - **(T=1)**: uses the model's original probability distribution.
  - **High temperature ((T>1))**: *probability differences become smaller*. More tokens have a chance of being selected.
  - **Argmax** means selecting the token with the highest score.
  - **Uniform** means tokens become almost equally likely.

Example: "The chicken crossed the…"

Low (T): almost always selects **"road." |** Normal (T): probably selects **"road." |** High (T): may select **"street," "river,"etc**

Temperature sharpening vs flattening

Sharpening: Makes the highest-probability token much more dominant. The output becomes predictable.

Flattening: Distributes probability more evenly among tokens. The output becomes varied or creative.

Distribution	Road	Street	River
Original	70%	20%	10%
Sharpened	90%	8%	2%
Flattened	45%	32%	23%

Low temperature → sharpening

High temperature → flattening

Let's go for a quick playthrough

1. Temperature (top-p pinned at 1.0)

```
for t in (0.0, 1.0, 2.0):
    ask(t, 1.0)
```

[22] ✓ 47.4s

```
temperature=0.0  top_p=1.0
    Guiding ships home, he stands alone.
    Guiding ships home, he stands alone.
    Guiding ships home, he stands alone.

temperature=1.0  top_p=1.0
    Light shines bright; they sail home.
    Light guides ships; I remain alone.
    Lonely light keeps ships from rocks.

temperature=2.0  top_p=1.0
    Light beacons cut through dark loneliness.
    Lost sailor signals home, beams died.
    Silent stone lantern guides lost souls.
```

What is Instruction Tuning?

01

Definition

A technique for fine-tuning Large Language Models (LLMs) using a labeled dataset of instructional prompts and their corresponding desired outputs.

02

The Goal

It teaches a pre-trained model how to follow specific user instructions rather than just generating linguistically coherent text.

03

Practical Impact

It is the critical step that transforms a basic text-generator into a useful, interactive assistant (e.g., turning a base model into ChatGPT).

Why Do We Need It?

01

The Problem with Base Models

Pre-trained models are optimized only to predict the next word.

Example: If you prompt "Teach me how to bake bread," it might just append "in a home oven" rather than providing a recipe.

02

Bridging the Gap

Instruction tuning aligns the model's fundamental objective (next-word prediction) with the user's actual intent.

03

Efficiency

It requires a fraction of the computational power, time, and data compared to training an massive model from scratch.

How It Works & Training Data

01

The Mechanism

Models learn from Supervised Fine-Tuning (SFT) using datasets with three elements:

- **Instruction: The task (e.g. "Translate")**
- **Context: Optional info**
- **Target: Correct response**

02

Human-Created

High-quality but labor-intensive datasets curated by humans.

Examples: Flan, Dolly, OpenAssistant

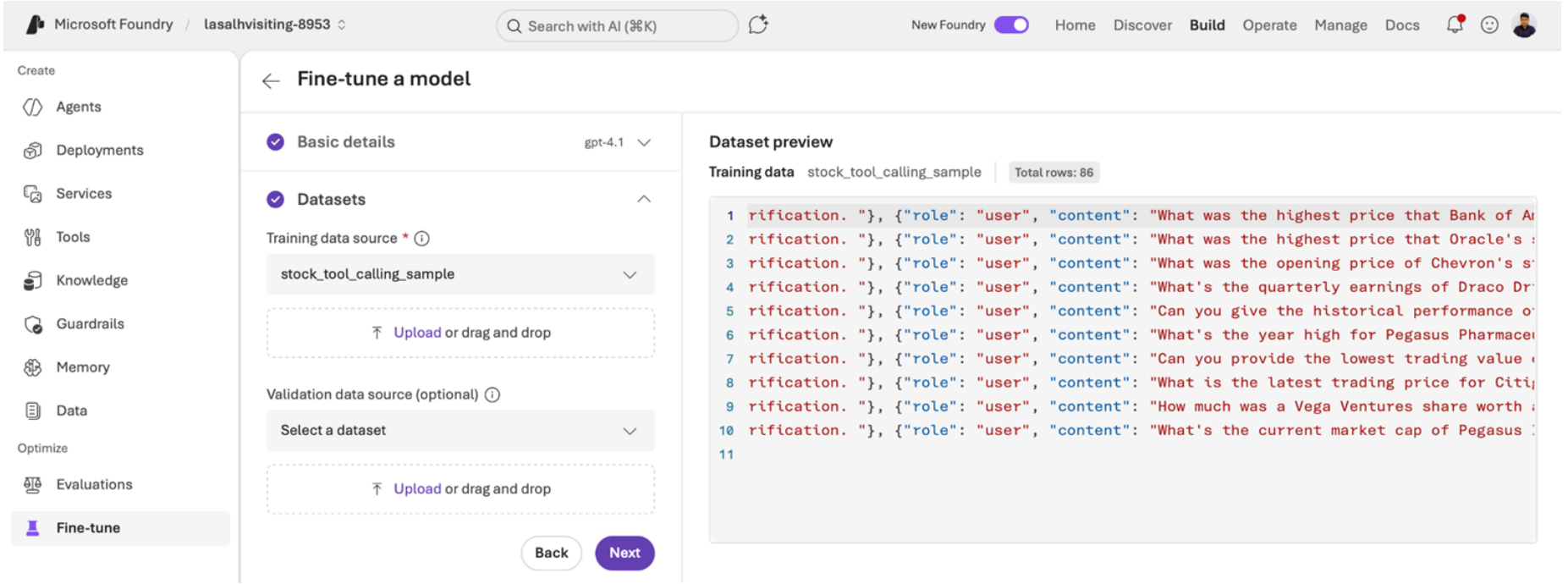
03

LLM-Generated

Using outputs from larger models (like GPT-4) to cheaply train smaller models.

Examples: Alpaca, ShareGPT

Quick Demo


Microsoft Foundry / lasalhvisiting-8953
Search with AI (⌘K)
New Foundry
Home
Discover
Build
Operate
Manage
Docs
Create
Agents
Deployments
Services
Tools
Knowledge
Guardrails
Memory
Data
Optimize
Evaluations
Fine-tune
Fine-tune a model
Basic details
gpt-4.1
Datasets
Training data source *
stock_tool_calling_sample
Upload or drag and drop
Validation data source (optional)
Select a dataset
Upload or drag and drop
Back
Next
Dataset preview
Training data
stock_tool_calling_sample
Total rows: 86
1 rification. "}, {"role": "user", "content": "What was the highest price that Bank of A
2 rification. "}, {"role": "user", "content": "What was the highest price that Oracle's
3 rification. "}, {"role": "user", "content": "What was the opening price of Chevron's s
4 rification. "}, {"role": "user", "content": "What's the quarterly earnings of Draco Dr
5 rification. "}, {"role": "user", "content": "Can you give the historical performance o
6 rification. "}, {"role": "user", "content": "What's the year high for Pegasus Pharmace
7 rification. "}, {"role": "user", "content": "Can you provide the lowest trading value
8 rification. "}, {"role": "user", "content": "What is the latest trading price for Citi
9 rification. "}, {"role": "user", "content": "How much was a Vega Ventures share worth
10 rification. "}, {"role": "user", "content": "What's the current market cap of Pegasus
11

The three defining limitations

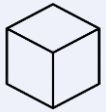
- There are 3 defining limitations when it comes to using models alone
- Almost everything we build this term compensates for these:

Limitation	What happens there	
Stateless	No memory between calls	If you want the model to "remember," you must store and replay it.
Ungrounded	No live data; only training + your input	To answer about your documents or current facts, you must bring the data to the model (retrieval) or give it tools to fetch it.
Probabilistic	Improvises; "best" not guaranteed	We control this with the **temperature**

Evolution from standalone models to Agentic AI

AI that can

Create for you

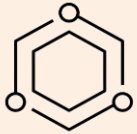


Models

- Text summarization
- Text synthesis
- Pattern matching

AI that can

Retrieve for you



Assistants

- Information retrieval
- Prescriptive tasks
- Single step or rule based processes

AI that can

Do for you



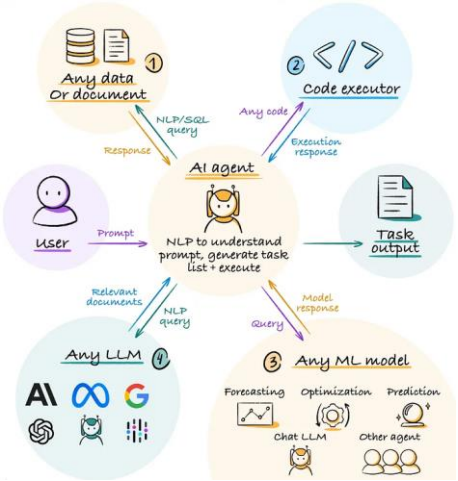
Agents

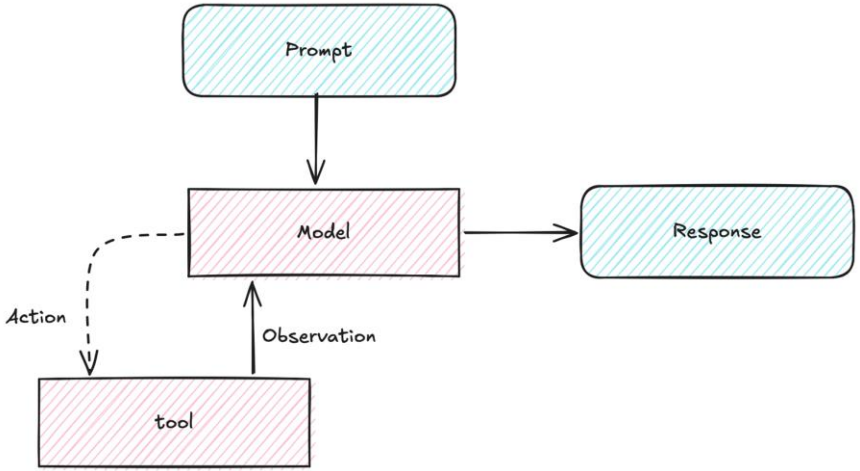
- Multi step processes
- Autonomous actions
- Self-corrections

- The evolution from *probabilistic* to *agentic* and how we overcome the limitations of LLMs
- A **model** gives an answer, an **assistant** finds information, but an **agent** completes a task.

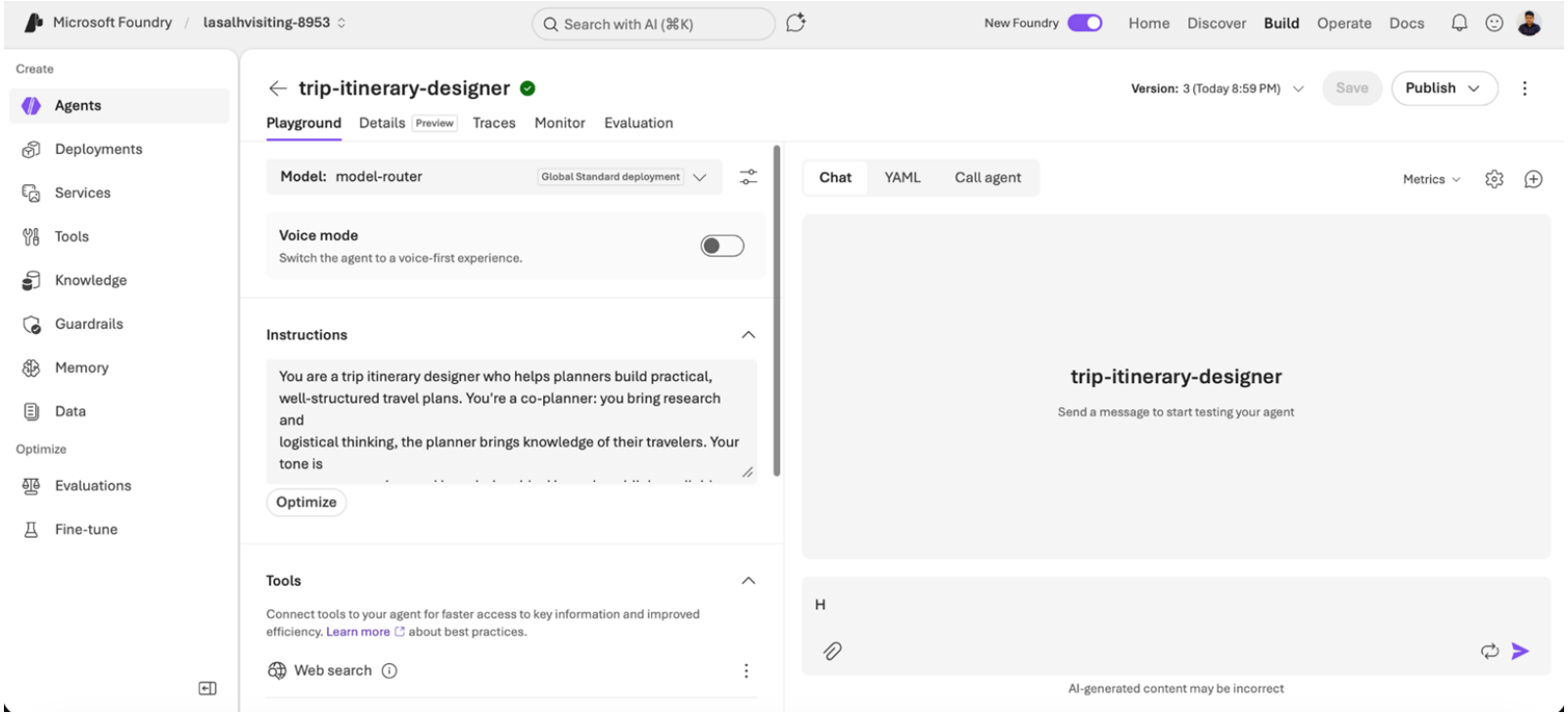
What is an Agent?

Definition: An LLM directing a loop: it observes state, decides on actions (tool calls), a runtime executes them, and results feedback.





Quick demo on playground



The Agency Spectrum

Definition: Agency means how much control the AI has over completing a task.

Level	Who controls the process?	Example
LLM call	The user provides one task; AI creates one response.	Summarize this email
Workflow / chain	The developer fixes the sequence of steps. The model follows this predefined path.	Translate → summarize → format
Router	The model selects one path from several predefined paths.	Send a customer request to billing support or technical support.
Autonomous agent	The model decides which tools to use, their order and when to stop.	Book me a flight to London tomorrow It should request human approval before making the payment.

Do You Actually Need an Agent?

- Agents provide flexibility and but they can have: latency, cost, and non-determinism
- Every LLM decision is a chance to be wrong

Ask yourself…	Then reach for…
Can we define all the steps in advance?	Use a fixed **workflow**. *Example: Validate a form → save it → send a confirmation email.*
Are there only a few known outcomes?	Use a **router** or **classifier**. *Example: Route a request to Billing, Sales or Technical Support.*
Does the task require changing multi-step decisions?	Use an **agent** with only the required level of autonomy. *Example: Create a travel plan by checking weather, budget and availability.*

Always select the simplest and most reliable solution that can complete the task.

Checkpoint 1 – Justify which pattern you would use



1. You are building an application that takes a daily news article in Spanish, translates it into English, summarizes it into three bullet points, and finally formats the output into a specific JSON structure for your database.

2. A user prompts your application with: "Find me a flight to London tomorrow, book a 4-star hotel near the city center, and check if I need an umbrella based on the forecast." The system must interact with external booking APIs and weather services to fulfill the request.

3. Your company receives thousands of customer emails daily. You need a system that reads each incoming email and simply categorizes it into one of three buckets: "Billing Issue," "Technical Support," or "General Inquiry."

Pair up — 3 minutes — then we discuss answers together.

Checkpoint 1 – Answers

1. Workflow / Chain

You can write the steps down in advance (Translate → Summarize → Format). It relies on a fixed code path with specific LLM steps rather than autonomous decision-making.

2. Autonomous Agent

This requires dynamic multi-step decisions against an unpredictable environment (flight/hotel availability). The LLM must observe state, decide which tools to use and in what order, and execute them.

3. Router / Classifier

The output space is small and known (only three categories). The LLM just needs to decide which of the N fixed paths the input belongs to.

Rule of thumb — choose the least autonomy that still solves the problem.

PART 2 OF 4

ReAct & Tool Calling

- ReAct tool calling
- Tool execution lifecycle



ReAct: why interleaving beats planning ahead

PROMPT: "Who is the current CEO of the company that owns LinkedIn, and what year did they become CEO?":

- **Plan-then-execute**: write the full plan, run it. Fragile — the plan can't react to what search actually returns (what if the owner changed? what if search returns a disambiguation page?).
- **ReAct (Reasoning + Acting):** alternate Thought → Action → Observation, deciding each action after seeing the last observation:

```
Thought: I need the owner of LinkedIn first.          | reasoning (tokens)
Action:  search("who owns LinkedIn")                  | tool call
Observation: "Microsoft acquired LinkedIn in 2016"    | runtime result
Thought: Now I need Microsoft's current CEO…          | reasoning
Action:  search("Microsoft current CEO start year")   | tool call
Observation: "Satya Nadella, CEO since 2014"          | result
Thought: I can answer now.
Answer:  Satya Nadella, CEO since 2014.
```

The Messages

Definition: Messages are the fundamental unit of context for models in LangChain. They represent the input and output of models, carrying both the content and metadata needed to represent the state of a conversation when interacting with an LLM..

Messages are objects that contain:

- **Role** - Identifies the message type (e.g. `system`, `user`)
- **Content** - Represents the actual content of the message (like text, images, audio, documents, etc.)
- **Metadata** - Optional fields such as response information, message IDs, and token usage

Message types

- System message - Tells the model how to behave and provide context for interactions
- Human message - Represents user input and interactions with the model
- AI message - Responses generated by the model, including text content, tool calls, and metadata
- Tool message - Represents the outputs of tool calls

https://docs.langchain.com/oss/python/langchain/messages
https://developers.openai.com/api/docs/guides/function-calling

The Bridge problem

The problem: LLMs produce text

Software needs structured commands. **Parsing** free-form text or model-written JSON can fail because of extra text, invalid syntax, missing fields, or wrong data types.

The solution: Native tool calling

The model returns a structured request such as

get_weather({"city":"Kandy"}).

The application validates the tool name, inputs, permissions, and business rules before execution.

User request → LLM → Structured tool call → Validation → Tool → Result → Final answer

The LLM requests the action; the application validates and executes it.

Frame by Frame: Tool calling cycle



- **Tools** extend what agents can do letting them fetch real time data, execute code, query external databases, and take actions in the world.
- Under the hood, tools are callable functions with well defined inputs and outputs that get passed to a chat model. The model decides when to invoke a tool based on the conversation context, and what input arguments to provide.

```python
from langchain.tools import tool

@tool
def search_database(query: str, limit: int = 10) -> str:
    """Search the customer database for records matching the query.

    Args:
        query: Search terms to look for
        limit: Maximum number of results to return
    """
    return f"Found {limit} results for '{query}'"
```

Checkpoint 2 – Quick Activity

1. The Scenario: Brainstorm one specific real-world task where an AI agent cannot rely on its internal knowledge and must interact with an external system (e.g., checking live sports scores, sending an email, booking a meeting room).

2. Design the Signature: Give your tool a clear function name and list the exact input parameters (arguments) the LLM would need to extract from a user's prompt (e.g., book_room(room_id, start_time, duration)).

3. Determine the Return: What specific data type or confirmation must this tool return back to the LLM so it can formulate a helpful final response to the user?

Pair up — 3 minutes — then we discuss answers together.

PART 3 OF 4

Agentic Frameworks

- Agentic frameworks
- How to choose the correct framework



Agentic frameworks

Definition: An agentic framework is a software architecture designed to build, manage, and orchestrate autonomous AI agents.

Overcoming the "Stateless" Limitation

A raw LLM cannot pause a task, wait for an external system to process a job, and come back to it later. Agentic frameworks manage the "state" of a task. If a process takes hours or requires waiting for a human to approve an action, the framework parks the state and resumes it seamlessly.

Deterministic Control in a Probabilistic System

LLMs are probabilistic. Frameworks enforce structure. They ensure that if an LLM decides to delete a file, the framework intercepts that decision, validates the parameters, and can enforce a "human-in-the-loop" approval process before the action actually happens.

Cyclical Execution and Error Correction

The agent can execute a tool (like running code), read the error message, realize its mistake, and try a different approach without the user having to re-prompt it.

Multi-Agent Orchestration

Complex problems often require different skill sets. Instead of using one massive prompt to do everything, frameworks allow developers to build multi-agent systems.

Key Components of an Agentic Framework

Models

The brain of the system. While the LLM is the most famous part, an agentic framework often uses a mix of models.

Tools

Tools give the LLM "hands" to interact with the outside world. The framework translates the LLM's text output into functional calls.

Memory

Memory provides the context necessary for the agent to maintain continuity across tasks and sessions.

Monitoring

This is the monitoring backend for the agents execution.

Prompts

The instructions and the response objects that govern the HUMAN, SYSTEM, TOOL, AI lifecycle

Middleware / Orchestration

These include the structured outputs, guardrails, HIL, Orchestration etc.

The prompt guides the model, memory provides context, tools perform actions, orchestration controls the process, and monitoring records everything.

Popular Agentic Frameworks

Framework	Best at	Reach for it when…
LangChain + LangGraph *(our stack)*	biggest ecosystem of integrations; LangGraph: explicit, durable, resumable control flow	you want maximum transferability and production-grade control
Microsoft Agent Framework (succeeds AutoGen + Semantic Kernel)	multi-agent patterns with first class citizenship	deep Azure ecosystem alignment
LlamaIndex	data first: ingestion, indexing, retrieval quality knobs	RAG is the product and agents are a feature (inverse of LangChain's emphasis)
Google ADK	thorough multi agent orchestration with sequential pipelines, parallel workflows and feedback loops	quick prototyping and deep orchestration flows and handoffs
DSPy	*optimizing* prompts/pipelines programmatically (adjacent to agents)	you have evals and want the machine to tune the prompts

Popular agentic frameworks

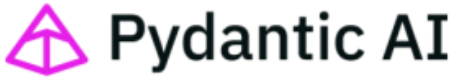


















Best Practices

- Start with Small-Scale Prototypes
- Incorporate Human Oversight
- Optimize Memory Usage
- Ensure Robust Monitoring and Observability
- Ensure security is not an afterthought

https://blog.jetbrains.com/pycharm/2026/06/top-agentic-frameworks-for-building-applications-2026/#best-agentic-frameworks-for-your-projects
https://www.exabeam.com/explainers/agentic-ai/agentic-ai-frameworks-key-components-top-8-options/

Next Week & How to Prepare

Lecture 06 — Knowledge & memory of AI agents: RAG, embeddings, retrieval, memory & the idea of tools

- Go through the LABs
  - Go through background material on the above topics
  - Play around on Azure AI Foundry

SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 06 • Agentic Software Development 2

Agentic RAG

Year 3 • Semester 1 • 4 Credits

Lecturer: Lasal Hettiarachchi | lasal.h.visiting@sliit.lk | Course web: courseweb.sliit.lk

Today's Roadmap – 3 Hours

Block A 0–35 mins	**The grounding problem & the RAG pipeline** Why models hallucinate, fine-tuning vs RAG, loading, splitting and overlap
Block B 35–85 mins	**Embeddings, vector search & RAG quality** Semantic search, similarity metrics, vector stores and retrieval evaluation
Block C 95–140 mins	**Production-ready RAG** Query transformation, hybrid retrieval, reranking, evaluation and security
Block D 145–180 mins	**Memory & the idea of tools** Short-term and long-term memory, conversational RAG, tools versus retrieval

Short breaks at 85–95 min and 140–145 min • Questions welcome at any point

PART 1 OF 4

The Grounding Problem & the RAG Pipeline

- The grounding problem: why models invent answers
- Fine-tuning vs RAG — when to use each
- The two-phase pipeline: indexing and querying
- Loading, splitting, and why overlap matters



The grounding problem

Ask a base model about **your** company's refund policy. It will **hallucinate** a plausible answer or say it doesn't know — it was never trained on your PDF, and it has no live access.

Fine-tuning

Idea: *Re-train or adjust the model using domain-specific data.*

Trade-offs: Costly, slow, **static**, Less transparent

RAG

Idea: *Retrieve relevant information and include it in the model's prompt.*

Trade-offs: Cheap, **live**, citeable ✓

RAG changes the input, *not the model. That's why it's the default for "answer from my documents."*

Fine-tuning vs RAG — when each?

Use fine-tuning when…

- You need a consistent **style, format, or behaviour**
- The "knowledge" is a ***skill***, not a fact set
- You can afford to retrain on updates
- Teach the AI **how to answer** professionally.

Use RAG when…

- You need **current or private facts**
- The knowledge **changes** over time
- You need **citations / auditability**
- Give the AI **the correct information** before answering.

Often you use both: fine-tune the *behaviour*, RAG the *facts*.

Heuristic: **RAG for what it should *know*, fine-tuning for how it should *act*.**

RAG as a mental model: two phases

Phase 1: INDEXING (done once, offline)

docs → load → split → embed → **store in vector DB**

Phase 2: QUERYING (per question, online)

question → embed → retrieve top-k chunks → put chunks in prompt → model → **answer (+ citations)**

Hold these two phases separately.

Most RAG confusion comes from mixing "prepare the knowledge" with "answer a question."

Step 1 — Loading (get data in)

First , collect data from Any source — PDF, web page, database, Markdown, Slack export.

Convert every sources into a common **Document** format**.**

A **document** contains: **content** (The actual text.) + **metadata** (source, page, url, author, date…).

- One **loader interface**, hundreds of source types.
- **Metadata is precious** — it's helps with **filtering** and providing **citations**.

Design point: Always keep the content and its metadata together.
You can't cite a source you threw away.

Step 2 — Splitting (chunking)

We break documents into passages because:

- **Focused pieces:** Retrieval works best on focused pieces, not whole books.
- **Token budget:** Chunks must fit the budget alongside question and answer.

The craft: *chunks big enough to hold a complete idea, small enough to be precise.*

Too big

Retrieval returns unnecessary information, wastes tokens and makes the correct answer difficult to find.

Too small

Ideas are split across chunks, key context is lost, and retrieval misses them.

Chunk size and overlap are two important factors that affect RAG quality.

We'll tune it in the lab.

Choosing the Right Chunk Size

A good chunk keeps one complete idea together:

- **Keep related text:** Keep sentences about the same topic together.
- **Stay focused:** Avoid mixing several unrelated topics in one chunk.

The goal: *enough context to answer, with little unnecessary text.*

Start here

Try chunk_size = 1000 characters.

Use chunk_overlap = 100–200 characters.

Test and adjust

Ask real questions. Check the retrieved chunk, then increase or decrease the size.

There is no universal best chunk size.

Start with a sensible value, then test it using your own documents and questions.

Why Overlap Matters

Sometimes important information is divided between two chunks.

Chunk A ends here ┐
... annual plans is 30 | days. Monthly...
└ Chunk B starts here

Without overlap, **"30"** and **"days"** land in completely different chunks.

Then the system may fail to understand the complete meaning: "**30 days.**"

The Search Failure

A user query for **"how many days for a refund?"** may match neither chunk well, as the key semantic connection is completely broken.

The Solution: Overlap

Overlap repeats a small amount of text between neighbouring chunks. By keeping "30 days" together in both chunks, the context remains fully intact.

Trade-off: Safety vs. Efficiency

More overlap guarantees better boundary safety but generates more chunks, increasing storage costs and token consumption.

Typical overlap is ≈ 10–20% of chunk size.

Discussion Checkpoint #1 (5 min)

1. Your RAG bot gives a vague, half-right answer about pricing. Before blaming the model, **where do you look first**, and why?

2. You set `chunk_size = 50` characters to be "precise." What likely goes wrong?

With a neighbour — 5 minutes — then we discuss answers together.

Discussion Checkpoint #1 — Model Answers

Scenario 1: Vague Pricing Answers

Your RAG bot gives a vague, half-right answer about pricing. Before blaming the model, **where do you look first**, and why?

Checkpoint 1 Answer

Inspect the retrieved chunks first. Most failures in RAG are retrieval issues, not generation. Always audit the raw context fed to the model.

Scenario 2: Ultra-Precise Chunking

You set `chunk_size = 50` characters to be ultra-precise. What likely goes wrong?

Checkpoint 2 Answer

50 characters shreds ideas. Retrieval returns micro-fragments with no context. The model cannot answer from "confetti".

With a neighbour — 5 minutes — then we discuss answers together.

PART 2 OF 4

Embedding's, Vector search & RAG quality

- Keyword search vs semantic search
- Embeddings and how similarity is measured
- Vector stores and approximate nearest neighbours
- Evaluating retrieval and tuning RAG quality



The search problem

How do we find the chunks "relevant" to a question?

Keyword search

Matches **letters/words** directly.

For example, searching for `"car"` won't match `"automobile."`

It can miss synonyms, paraphrases and differently worded questions.

Keyword search is still useful for exact values such as error codes, product IDs and names.

Semantic search

Matches by conceptual **meaning**.

For example, Therefore *"*`I forgot my login`*"* can match a passage about `"password reset`*“*

Finds relevant text even when the words are different.

Semantic search is useful when users express the same idea using different words.

Embeddings make semantic search possible.

> ***Strong RAG** often combines both methods: keyword search for exact terms and semantic search for meaning.*

Embedding's — Turning Meaning into Numbers

An embedding converts text into a vector—a numerical representation of its meaning.

Vector Closeness

Similar meaning → nearby vectors

Different meanings → distant vectors

`"reset my password"` and
`"I forgot my login”` = close

`"reset my password"` and
`"pizza toppings“`= far

How RAG Uses Embedding's

1. *Convert the user’s question into a vector.*
2. *Compare it with stored chunk vectors*
3. *Retrieve the nearest and most relevant chunks.*

The mathematical distance or angle between vectors directly measures **semantic similarity**.

Embeddings help RAG find relevant information even when the question and document use different words.

Why Are Embeddings High-Dimensional?

Embeddings use many values together to represent subtle differences in meaning.

More Dimensions = More Detail

An embedding may contain hundreds or thousands of values. Together, they represent topic, tone, context, and entities.

More dimensions help separate subtle meanings. **One value alone is not meaningful**; the full pattern carries the meaning.

Compare with Cosine Similarity

First, convert both texts into vectors. Then compare their **direction** using cosine similarity.

A **higher similarity** means closer meaning; a lower score means different meaning.

> *High dimensions provide space to **represent detailed meaning**; cosine similarity helps RAG find the closest information.*

Measuring Closeness — Three metrics

Here, **a** and **b** represent two embedding vectors.

1. Dot Product

$$a \cdot b = \sum a_i b_i$$

Multiply matching values and add them together.

A larger result can indicate greater similarity.

However, vector size or magnitude affects the result.

2. Euclidean Distance

$$||a - b|| = \sqrt{\sum (a_i - b_i)^2}$$

Measures the straight-line distance between two vectors.

Smaller distance means greater similarity.

It is also sensitive to vector scale.

3. Cosine Similarity

$$\cos \theta = a \cdot b / (||a||b||)$$

Compares the direction or angle of two vectors.

It ignores vector magnitude.

Therefore, it is commonly used with text embedding's.

> *A Cosine score : **cos θ = 1** means highly similar, **0** means unrelated, and **-1 (negative)** means different directions.*

Example: "Forgot my password" and "Reset my login" should have a higher cosine similarity than "Forgot my password" and "Pizza toppings."

L2 Normalization — Make Every Vector Length 1

If vectors are L2-normalized (||a|| = ||b|| = 1), then all three metrics are mathematically linked:

The Mathematical Equivalence

L2 normalization changes a vector so its total length becomes 1, without changing its direction.

$$||a - b||_2^2 = 2 - 2(a \cdot b) = 2 - 2\cos\theta$$

On L2-normalized vectors, the squared Euclidean distance is directly proportional to the dot product and the cosine similarity.

After Normalization

- **Same score:** dot product and cosine similarity give the same ranking. Euclidean distance is also mathematically connected to them.
- **Same ranking:** Euclidean distance finds the same nearest vectors; smaller distance means closer.

Normalizing makes "similarity" about meaning, not text length or vector scale.

In Practical: **Normalize** document embedding's when storing them, normalize each query once, and use a fast dot product to find the most similar chunks.

How Embedding's Learn Meaning

Embedding models are trained with **contrastive learning** (a dual encoder) to map meaning to distance: Contrastive learning **pulls related text closer** and **pushes unrelated text** farther **apart**

The model receives

$$\mathcal{L} = -\log \left[e^{(sim(q,d^+)/\tau)} / \sum e^{(sim(q,d)/\tau)} \right]$$

Anchor (a): the query

Positive (p): the correct or related passage

Negative (n): unrelated passages

• **Query**: "I forgot my login"

• **Positive**: "Reset your password“ • **Negative**: "Best pizza toppings"

The Dual Encoder Mechanism

A dual encoder converts the query and passage into separate vectors

- **Pull Related Pairs:** Training actively pulls the embedding's of related queries and positive passages closer together.
- **Push Negatives Apart:** Simultaneously pushes unrelated pairs far apart in the vector space.

Why it works: Similar meanings become nearby vectors—so "I forgot my login" ends up near "reset your password." **Distance now represents meaning.**

Searching fast: exact vs approximate NN

NN means Nearest Neighbor —finding the vectors most similar to the query.

Brute Force (Exact)

`O(n · d)`

compares the query with every stored vector.

It gives the exact result.

However, it becomes slow with millions of vectors.

IVF Index

`Inverted File`

Groups similar vectors into clusters and searches only the nearest clusters.

HNSW Graph

`~O(log n) search`

creates a graph and quickly moves through connected vectors to find close matches.

Multi-layer graph of navigable small worlds.

PQ Compression

`Product Quantization`

Compress vectors into compact codes. Uses far less memory and boosts speed.

Managed Search: Vector databases like Pinecone, pgvector, and FAISS implement these complex indices. You get fast nearest-neighbour searches out of the box without building algorithms from scratch.

MMR — Relevant Results Without Repetition

Plain top-k can return near-duplicates. **Maximal Marginal Relevance** MMR re-ranks retrieved chunks to **balance relevance** and diversity.

How MMR Scores Each Chunk

$$\text{Score} = \lambda \times \text{relevance} - (1-\lambda) \times \text{repetition}$$

- **Relevance**: How well does this chunk match the query?
- **Repetition**: How similar is it to chunks already selected?

What λ (Lambda) Controls

- **λ close to 1:** Prioritize the most relevant chunks.
- **λ around 0.5:** Balance relevance with new information.
- **Lower λ** prioritizes diverse, less repetitive chunks.

Example: Plain top-k may return three password-reset chunks. MMR can return password reset, account recovery, and login security—more useful context for the LLM.

Evaluating retrieval — the metrics

How do you know retrieval is *good*? Score it against known-relevant passages.

@k means we evaluate only the first k results, such as the top 5.

1. Recall@k

`(relevant top-k) / (all relevant)`

Captures:
Out of all the relevant chunks, how many did we retrieve?

2. Precision@k

`(relevant top-k) / k`

Captures:
Out of the retrieved chunks, how many are actually relevant?

3. MRR

`mean of 1/rank of first hit`

Captures:
How high is the first correct result ranked?

4. nDCG@k

`DCG / IDCG`

$DCG = \sum rel_i / \log_2(i+1)$

Captures:
Are the most useful results placed near the top?

For RAG, Recall@k matters most — if the right chunk isn't retrieved, the model **cannot** answer correctly, no matter how good it is.

Vector stores — searching the galaxy

A **vector store** indexes embeddings so we can find nearest neighbours ***fast*** (even over millions of chunks).

Core Responsibilities

- **A vector store keeps**: The chunk's **embedding** + its **text** + its **metadata**.
- Given a query embedding, return the **top-k** closest chunks dynamically.

Where it Runs

Local / In-Process:
FAISS, Chroma

Managed / Production:
Pinecone, pgvector, Weaviate

The store is an implementation detail. The **concept** is "approximate nearest-neighbour search over meaning."

Assembling the answer — "stuffing" context

The simplest, most common RAG strategy: retrieve, insert, and instruct. It means placing all selected document chunks directly into the LLM prompt.

The process has three steps:

1. Retrieve: Find the top-k relevant document chunks dynamically.

2. Stuff: Insert them into the model's prompt *as rich context.*

3. Instruct: Tell the model to answer strictly using only that context.

Anti-Hallucination - Guardrail

The final instruction forms the system's core defense:

"Answer only from this context; if it's not here, say you don't know."

This changes the task from risky recall to reliable reading comprehension.

The model's job shifts from recall (risky) to reading comprehension over supplied text (reliable). That shift is RAG.

Grounding & citations

Trustworthy AI isn't just "right" — it **shows its work**.

Verifiable Metadata

Because each chunk carries **metadata**, the model can say *where* an answer came from:

"Refunds take 5–7 days [handbook, Billing]."

Structured Outputs

We can also return the result as a **structured output** containing

```
answer + sources + confidence
```

Citations turn an opinion into a verifiable claim, allowing users and auditors to trace information back to its source.

What makes RAG good or bad — the retrieval

RAG quality is **mostly retrieval quality**. The dials:

The Retrieval Levers

Chunk size / overlap
Smaller chunks are more focused. **Larger chunks** provide more context. **Overlap** prevents information from being split between chunks.

k (chunks retrieved)
Controls how many chunks are retrieved. A larger k may improve recall, but it can also add noise and increase token usage.

Diversity (MMR)
Reduces duplicate results and provides more diverse information.

Filtering & Representation

Metadata filtering
Limits the search by document, section, date or category.

Embedding model
A better model can represent meaning more accurately and improve matching.

Debugging maxim: **"The LLM is wrong" is usually "retrieval gave it junk."** Inspect the retrieved chunks *first*.

RAG failure modes (and the fix)

Symptoms map to causes — and **almost every cause is retrieval**:

SYMPTOM	LIKELY CAUSE	THE FIX
Vague / generic answer	Chunks too big or irrelevant	**Tune chunk size; raise relevance**
"I don't know" when it should know	Retrieval missed it	**Smaller chunks, higher k, better query**
Confident but wrong	Model used memory, not context	**Strengthen the "only from context" guardrail**
Repetitive context	Near-duplicate chunks	**Use MMR for diversity**
Slow / expensive	k too high, chunks too big	**Lower k; right-size chunks**

> **Almost every row is retrieval, not the model.** Internalise that.

PART 3 OF 4

Production-ready RAG

- From demo RAG to production RAG
- Query transformation, hybrid retrieval and reranking
- Pipeline patterns: routing, multi-hop and self-correction
- Evaluation, freshness, cost and security



From demo RAG to production RAG

A **basic RAG demo** usually uses one user, one document and simple questions.

Production RAG is the basic RAG pipeline strengthened for real users, real data and real risks:

Demo assumption

- One user, one PDF
- Clean, self-contained questions
- Documents never change
- "Looks right" is enough
- Everyone may read everything

Production reality

- Millions of chunks, many users, permissions
- Vague, conversational, multi-part queries
- Docs update daily → **stale index**
- Must **measure** quality, cost, latency
- Access control, PII, prompt injection

Production RAG still has the same main phases:

1. **Ingestion**: Prepare, chunk, embed and index documents.
2. **Query time**: Retrieve relevant chunks and generate the answer.

The RAG Upgrade Map

Improvements are required on both the query side and the index side.

Query side – happens when the user asks a question:

Understand: Rewrite unclear queries or create multiple query

Retrieve: Combine semantic vector search with keyword search

Filter: Apply metadata filters and access-control rules.

Refine: Rerank, remove weak results and compress unnecessary content.

HyDE creates a possible answer first and uses it to search for similar documents.
ACL ensures that users retrieve only the documents they are permitted to access.

Index side – happens when preparing the documents:

- **Split documents** using smarter or semantic chunking.
- **Use an embedding** model suitable for the domain.
- **Store embeddings** together with text and metadata.
- **Update the index** when documents change and maintain versions.

Topology: routing · multi-hop · self-correcting · agentic composition
Around It: evaluation · caching · tracing · user feedback · security guardrails

Fix the question first — query transformations

Users don't ask "retrievable" questions. Transform the query **before** embedding it:

Rewrite / condense

- Converts a follow-up question into a complete standalone question.
- Example: “Is it cheaper?” becomes “Is Pinecone cheaper than Weaviate?”

Step-back

- Creates a more general version of an overly specific question.
- This helps retrieve broader background information.

Multi-query

- Generates several versions of the same question.
- Retrieves results for every version and combines them.
- This improves recall when documents use different wording.

HyDE (Hypothetical Document Embeddings)

- The LLM first generates a possible or hypothetical answer.
- That answer is embedded and used to search for similar documents.

LangChain: Improve the question before searching to improve the retrieved context.

Hybrid Retrieval — Meaning(dense) + Exact Words(keyword)

Embeddings miss **exact strings** (error codes, SKUs, names, acronyms); keyword search (**BM25**) misses **paraphrases**. Production systems run **both** and fuse rankings.

Reciprocal Rank Fusion (RRF)

RRF Formula:

```
RRF(d) = ∑r ∈ rankers 1 / (k + rankr(d))
```

- A document ranked high by **either** ranker scores well.
- Parameter **k** (≈ 60) damps the long tail effect.
- Strictly rank-based: **no score normalization** is needed.

Implementation & Frameworks

LangChain Integration:

```
EnsembleRetriever(
  retrievers=[bm25, vector],
  weights=[0.4, 0.6]
)
```

- Combines sparse and dense retrievers seamlessly.
- Configurable weights for fine-tuned hybrid fusion.

Rule of thumb: Hybrid retrieval consistently beats either model alone — especially on **IDs, jargon, and names**.

Hybrid Retrieval — Meaning(dense) + Exact Words(keyword)

combines vector search with keyword search. Vector search understands meaning; BM25 catches exact terms. RRF combines both ranked lists.

1 & 2 — Search in Two Ways

1. Vector Search (Dense)

- Search by Finds meaning, synonyms and paraphrases
- May miss exact IDs, codes, names and SKUs.

2. Keyword search or BM25:

- May miss synonyms or different wording.

How They Work Together

Three Simple Steps

1. Run vector search
2. Run BM25 search
3. Merge both rankings with RRF

- RRF gives more points to results ranked near the top.
- It uses ranks, so raw scores do not need normalization.

Example: Query “ERR-401 login issue” → BM25 finds the exact code; vector search finds **authentication-failure documents**. The combined list contains both.

Retrieve wide, Re-rank narrow

Production RAG commonly uses a **two-stage retrieval process**.

Stage 1: Hybrid / Vector Search (Wide)

Retrieves the top ~50 candidates. Fast and approximate to ensure high recall.

Stage 2: Cross-Encoder Re-ranker (Narrow)

Filters down to the best 3–5 documents. Slow but highly precise context for the prompt.

A **bi-encoder** creates the query and document embedding's separately.

- Comparison uses a fast dot product.
- It is suitable for searching millions of documents.

A **cross-encoder** reads the query and document together.

- It provides more accurate relevance scores.
- However, it requires a model call for every query-document pair, so it is slower and more expensive.

> **Cheap-then-precise beats precise-everywhere.** LangChain: `ContextualCompressionRetriever` wrapping a reranker (e.g. Cohere Rerank, BGE cross-encoder).

Smarter indexing — beyond flat chunks

Basic RAG often splits documents after a fixed number of characters.

This may break sentences, sections or code blocks and remove important context.

Smarter indexing creates chunks based on the document's meaning and structure.

Structure & Parents

- **Structure-aware splitting:**

split using headings, paragraphs, tables or code blocks. This keeps related information together.

- **Parent-document retrieval:**

Search using small, focused chunks. When a small chunk matches, return its larger parent section. This combines accurate matching with enough surrounding context.

Summaries & Context

- **Multi-vector / summaries indexing:**

Create summaries or possible questions for each chunk. Search using these additional representations. Return the original document content to the LLM

- **Contextual chunk headers:**

Add the document title, section name or short description before embedding. This helps the embedding model understand what the chunk is about.

LangChain integration: *`ParentDocumentRetriever · MultiVectorRetriever`*

Measured gains: Better document preparation can improve retrieval without changing the LLM.

After retrieval, before the prompt

Retrieval may return useful chunks together with weak, repeated or unnecessary content.

Before sending them to the LLM, we should clean and organize the context.

Filtering & Compression

- **Score threshold:**

  Remove chunks with a relevance score below a selected threshold. If every result is weak, the system should say "I don't know" instead of using unrelated information.

- **Contextual compression:**

  Remove irrelevant sentences from retrieved chunks. Keep only the parts that help answer the user's question. This reduces token usage and noise.

Ordering & Deduplication

- **Lost in the middle:**

  LLMs may pay more attention to information at the beginning and end of a long prompt. Place the strongest chunks in important positions. LangChain provides ***LongContextReorder*** for this purpose.

- **Deduplicate:**

  Remove chunks that contain almost identical information. MMR can help retrieve more diverse chunks.

The Gold Standard: The prompt's token budget is prime real estate — spend it strictly on **signal**, not noise.

Reference: Liu et al. (2023) "Lost in the Middle: How Language Models Use Long Contexts"

Evaluating RAG end-to-end — the triad

End-to-end RAG evaluation checks the relationship between three elements: The user's **question** , The **retrieved context**, The **generated answer**:

The Core Triad Metrics

Faithfulness: Is every statement in the answer supported by the retrieved context? Unsupported claims indicate hallucination

Context Relevance: Does the retrieved context contain useful information for answering the question? Irrelevant chunks indicate a retrieval problem.

Answer Relevance: Does the final answer directly address what the user asked? An answer may be factually correct but still be off-topic.

Continuous Quality (CI/CD)

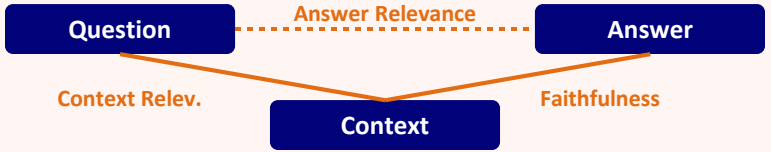
Golden Set: Create a golden test set containing sample questions, expected answers and correct source documents.

CI for Quality: Run this test set whenever chunking, embeddings, retrieval settings or prompts are changed.

LLM-as-Judge: Use Frameworks such as Ragas and LangSmith can help automate evaluation using an LLM as a judge.

Evaluation Mantra

"If you can't measure it, you can't tune it." A system that merely "feels better" will not survive in production.



RAG Architectures — the maturity ladder

How the research literature classifies RAG systems (Gao et al., 2023 — the RAG survey):three levels of RAG system maturity.

Naive RAG

Shape:
Uses a simple linear pipeline:
index → retrieve → generate

Adds:
The basic laboratory pipeline

Example : Simple FAQ chatbot

Advanced RAG

Shape:

Keeps the main pipeline but improves its stages.
+ Pre-retrieval & post-retrieval stages

Adds:
Transforms, hybrid, re-ranking, context compression

Example : Enterprise document assistant

Modular RAG

Shape:

Changes the pipeline into a collection of reusable modules. Different queries can follow different paths. Swappable modules, **non-linear** flows

Adds:
Routing, loops, memory, external tool use

Example : Complex assistant using documents, databases and tools

Key Difference: Everything up to Advanced RAG upgraded the **stages**, but **Modular RAG** changes the **shape**—the pipeline becomes a **graph** of modules composed per use case.

Core Rule: An architecture = **which modules** + **how data flows** between them.

Routing & Branching — Choose the Right Path

The router uses structured output to choose a source, tool, or skip-retrieval path.

How the Router Chooses a Path

1. Route — first examines the question and choose the correct source, retriever, or tool.

2. Decompose — Split a complex question into smaller sub-questions. Route each sub-question to the correct source. Retrieve the answers and merge them.

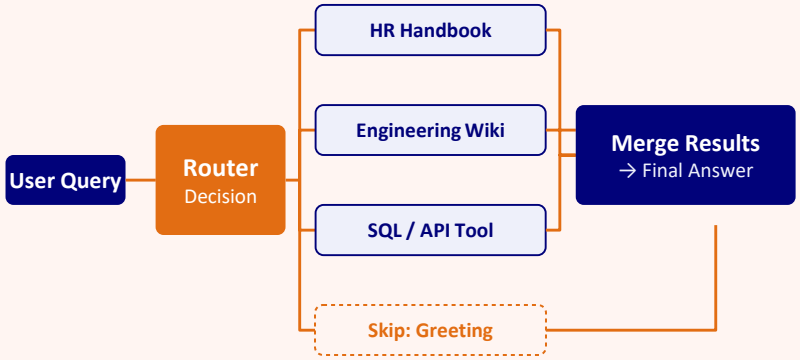
Ex. **"What is my leave balance, and how do I deploy the API?"**

Leave balance → HR or employee database

API deployment → Engineering Wiki

3. Skip-retrieval branch: —Simple greetings do not need document retrieval.

4. Merge + Re-rank : — After retrieval, results can be **merged** and **re-ranked** before generating the final answer.



Key Idea : Routing uses an LLM's structured output to control which path the RAG system follows.

Pipeline Patterns — Iterative & Self-correcting

Basic linear RAG retrieves information only once.

Complex questions may require the system to search, inspect the results and search again.

Multi-hop

Retrieve information for the first part of the question.

Read the result and create the next query.

Continue until all required information is collected.

Self-RAG

The model checks its own draft answer.

It asks: Is the answer supported? Is the context relevant? Do I need more information?

If necessary, it retrieves again.

Corrective RAG

First evaluate the quality of the retrieved documents.

If the documents are weak, rewrite the query or search another source, such as the web.

GraphRAG

Organizes entities and their relationships as a knowledge graph.

Useful for questions involving connections, trends or information across many documents.

Key Takeaway: When a RAG pipeline decides whether to retrieve again, it begins to behave like an agent.

Agentic RAG — choosing an architecture

Start with simple RAG. Add routing, loops or tools only when evaluation shows a clear failure.

Agentic RAG

An agent can decide:

- **Should** I retrieve information?
- **Which** source or tool should I use?
- Do I need to search again?
- Is the evidence strong enough to answer?
- When should I stop?

Observed Problem	Recommended Pattern
Multi-part question is only partly answered	**Decomposition / RAG-Fusion**
Correct information exists in another source	**Routing**
Answer requires several connected facts	**Multi-hop / Iterative RAG**
Retrieved evidence is weak or conflicting	**Corrective RAG + Thresholds**
"Question asks for a whole-corpus summary or trend	**GraphRAG / Hierarchical Summaries**

Start simple → Evaluate → Identify the failure → Add the smallest useful pattern → Test again.

Track **answer quality, latency and cost** after every architectural change**.**

Key message: Complexity is justified only when it solves a measured problem.

Keeping RAG Reliable — Freshness, Cost & Speed

A RAG system needs continuous maintenance after deployment.

Keep the Index Fresh

- Embed only new or updated documents.
- Remove deleted information.
- Run synchronization regularly.

Control Cost

- Cache existing embedding's.
- Do not reprocess unchanged documents.
- Process document updates in batches.

Changing the Embedding Model

- Old and new embedding's cannot be mixed.
- Create a new index → re-embed everything → test → switch safely.

Reduce Latency

- Measure each step: rewrite → retrieve → rerank → generate.
- Run independent searches in parallel.
- Stream the answer to the user.

Key Takeaway: Production RAG is a continuous pipeline—synchronize, monitor, test and improve it regularly

RAG Security — Retrieved Content Can Be Unsafe

Documents retrieved for the LLM must be treated as untrusted input.

RAG Security

The **Retriever** as an Entry Point

Retriever = Input Channel

Anything stored in retrieved documents may enter the LLM prompt. Therefore, retrieved content should not be trusted automatically..

Risk	Defence
Indirect prompt injection - A document contains hidden instructions	Treat retrieved text as **data**, not instructions. Separate it clearly and validate the output.
Data leakage across users - Unauthorized data is retrieved	**ACL-aware retrieval -** Filter documents using the user's role and permissions.
PII in chunks - Personal data enters the prompt	Remove or mask PII before indexing. Audit what enters the prompt.
Citations as cover - A citation does not support the answer	Check that the cited passage directly supports the claim

Key Takeaway: Filter, restrict and validate retrieved content before allowing the LLM to use.

Discussion Checkpoint #2 (5 min)

1. Why must the **same embedding model** be used for indexing and querying?

2. A legal-search RAG returns 5 chunks that are basically the same paragraph. **Which lever fixes this**, and what does it trade away?

With a neighbour — 5 minutes — then we discuss answers together.

PART 4 OF 4

Memory & the idea of tools

- Why every model call is stateless
- Short-term memory: windows, summaries and budgets
- Long-term memory: semantic, episodic and procedural
- Conversational RAG and the idea of tools



Memory — the forgetting problem

LLM calls are stateless: Each request starts without the previous conversation. The application must provide the relevant history again.

Where Is Memory Stored?

- Stored by the application or a database.
- Use a **user ID, session ID** or **thread ID.**
- Keep each user's conversation separate.

How Much Should We Keep?

- Keep recent messages and important facts.
- **Summaries** or **remove** older messages.
- This reduces token cost, latency and context overflow.

How It Works: Load relevant history → Add the new message → Send both to the LLM → Save the response

Key Takeaway: The model does not remember by itself—the application stores and re-sends the relevant context.

Four Types of Agent Memory

Different memory types help an agent handle the current conversation, past events, known facts and instructions.

Memory Type	Simple Meaning	*Example in an AI Agent*
Working / Short-Term	*the current exchange*	Recent messages inside the context window
Episodic — events	*What happened before*	stored past sessions & interactions, summarised
Semantic — facts	*what I know*	User preferences, business facts and RAG knowledge
Procedural — skills	*how to act*	System prompts, business rules, workflows and tool instructions

Short-term memory **is included directly in the prompt.**

Long-term memory **is stored outside the model and must be saved, searched and added back when relevant.**

Short-Term memory — the loop

The LLM does ***not remember automatically***. The application repeats a memory loop for every user message

State Management

Before each call:

- Read the user's session or thread ID.
- Load that session's conversation history.
- Combine: system instructions + history + new message.

After each call:

- Return the generated answer.
- Save the new message and answer to the same session.

The Loop (Turn N)

```
load history(session)
  └──► [system + history + new msg]
         └──► model ──► reply
save(new msg, reply) ◄──┘
```

Storage Options

- **In-memory**: Simple, but lost when the application restarts.
- **Redis**: Fast and useful across multiple servers.
- **Database**: Persistent and suitable for long-term storage.

Key Takeaway: Every turn must load from and save to the correct session.

Short-Term memory— strategies & trade-offs

Conversation history grows after every message. More history improves memory, but also increases tokens, **cost**, latency and **context-overflow** risk.

Strategy	How It Works	Trade-off - (Benefit and Limitation)
Keep Everything	Send the complete conversation every time	**Remembers more details, but becomes expensive and may exceed the context limit**
Sliding Windowing	Keep only the most recent N messages	Fast and affordable, but may forget earlier information
Summaries Older Messages	*Replace older messages with a short summary*	Keeps the main ideas, but may lose important details
Trim by tokens	Remove the oldest messages until the history fits	Keeps cost predictable, but may remove useful context

Recent messages + summary of older messages + important facts stored separately

Key Takeaway : Choose a strategy based on what the application must remember, how long it must remember it and the available cost.

Short-Term memory in LangChain

The lab pattern — wrap **any** chain; history is loaded/saved per **session_id**:

```python
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

sessions: dict[str, ChatMessageHistory] = {}
def get_history(session_id: str) -> ChatMessageHistory:
    return sessions.setdefault(session_id, ChatMessageHistory())

chatbot = RunnableWithMessageHistory(chain, get_history,
    input_messages_key="input", history_messages_key="history")

chatbot.invoke({"input": "Hi, I'm Lasal!"},
    config={"configurable": {"session_id": "user-42"}})
```

Windowing & Budgets

Pipe `trim_messages(strategy="last", max_tokens=…)` in front of the prompt to avoid context overflow.

Durability

Swap the store seamlessly: use `SQLChatMessageHistory` (SQLite/Postgres) or Redis with **zero chain changes**.

RAG changes the input, *not the model. RunnableWithMessageHistory handles state transition dynamically under the hood.*

Long-term memory — remembering across sessions

Long-term memory allows an assistant to remember selected information after the current conversation ends.

Should We Avoid?

- Do not resend every past conversation.
- Do not insert months of raw chat logs into one prompt.
- This increases cost, latency and context-overflow risk.
- More history does not always produce a better answer.

Better Long-Term Memory Approach

- Identify useful facts, preferences or events.
- Store them outside the model using the correct user ID.
- Search for relevant memories when needed.
- Add only the selected memories to the new prompt.

Long-term memory = select useful information → store it outside the model → retrieve it when needed. *It follows the same pattern as RAG, but applies it to user facts, past events and behavior.*

Semantic Memory — remembering useful facts

Semantic memory stores stable facts about a user or the world—not complete conversations: ***extract → store → retrieve → inject***

```
class UserFact(BaseModel): # Lecture structured output!
  fact: str # e.g. "prefers concise answers; codes in Python"
  category: Literal["preference", "biography", "project"]

extract = extract_prompt | model.with_structured_output(UserFact)
fact = extract.invoke({"turn": last_exchange}) # after turn or in
bg

memory_vs.add_texts([fact.fact], metadatas=[{"user": uid, "cat":
fact.category}])
memories = memory_vs.similarity_search(question, k=3,
filter={"user": uid})

# → format into system prompt: "Known about this user: …"
```

How Semantic Memory Works

1. **Extract and validate :** *Identify a clear and useful fact from the conversation.*
2. **Store :** *Save it with the user ID, category, source and timestamp.*
3. **Retrieve :** *Search for facts related to the user's current question.*
4. **Use and maintain :** *Add relevant facts to the prompt. Update or delete outdated facts.*

Memory Flow: Extract → Validate → Store → Retrieve → Use

Key Takeaway: A user profile is semantic memory about a person. RAG is semantic memory retrieved from documents.

Episodic Memory — remembering past events

*Episodic memory stores summaries of previous interactions: **what** happened, **when** it happened and **who** it involved.*

How Episodic Memory Works

1. **Capture the event :** *Summarise an important interaction or completed session.*
2. **Store it with metadata :** *Save the user ID, timestamp, topic and outcome.*
3. **Retrieve relevant events** : *Search using relevance and recency, filtered by the correct user.*
4. **Use the memory :** *Add only the most relevant past events to the new prompt.*

Implementation

```
# LLM-written summary & vector search
episode = summarise.invoke({"messages":
history.messages})
memory_vs.add_texts([episode], metadatas=[{"user": uid,
"ts": time.time()}])

past = memory_vs.similarity_search(question, k=2,
filter={"user": uid})
```

Episodic memory is RAG over selected past experiences. Remember meaningful events—not complete transcripts.
Storing successful past interactions and replaying them as few-shot examples is episodic memory used for quality, not just personalization.

Putting it together — a memory architecture

*Agent memory combines a read path with a controlled write-back loop around the LLM.. Every arrow is a **Runnable** you already own.*

Before Answering - Read Path (Every Turn)

User Msg –► **Short-Term Buffer** –► Model —► Reply

1. **Episodic:** Relevant past episodes (similarity + recency)
2. **Semantic:** User facts / profile (structured vector store)
3. **Procedural:** System prompt + standing rules

After Answering = Write Path (After Turns / BG)

The loop captures and persists fresh context actively:

1. **Append** to chat history
2. **Extract** key structured facts
3. **Summarise** semantic episodes
4. **Reflect** on rules and behaviors

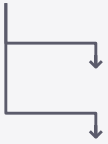
Composing retrievers, structured outputs, and summarisation chains. *Your existing runnable modules chain together to build complete, production-grade memory architectures.*

Conversational RAG — Memory + Retrieval

Memory *understands the conversation;* **retrieval** *supplies trusted, up-to-date facts.*

Previous Msg : "Tell me about the Team plan."

Follow-up: "Is it cheaper yearly?"



1 **Memory:** "it" → **Team plan**; rewrite

2 **Retrieval:** Fetch **current yearly pricing**

3 LLM: Grounded, Context-Aware Answer

Who Does What? (Context + Facts)

- **Memory** — resolves "it" and rewrites: "Is the Team plan cheaper yearly?"
- **Retrieval** — finds trusted, current pricing.
- **LLM** — combines both into a grounded answer.

Another follow-up: "What about for 20 users?"
Memory identifies the plan; retrieval finds the price.

Memory tells the system what the user means; retrieval tells it what is currently true.

Tools vs RAG — two ways to ground

RAG provides reliable knowledge, while tools provide live data, calculations and controlled actions.

Comparison	RAG	Tools
What does it do?	Retrieves relevant content and adds it to the prompt	Calls a function, database or external API
Best for	Policies, manuals, reports and knowledge bases	Live data, calculations, system updates and actions
Example	“What is our refund policy?”	Check the status of order 5482.”
Result	Evidence-based answer with supporting context	Structured result or confirmed action

Use RAG when the answer is in documents. Use tools when the task requires current data, computation or action. Combine both for reliable agentic workflows.

Next Lecture & How to Prepare

Lecture 07 — Agents & Multi-Agent Systems: Automating the tool loop, coordinating multiple agents, and adding middleware, guardrails, & observability

- **Work through the Lecture 06 lab sheet end to end**
- **Review the background reading on RAG, embeddings and agent memory**
- **Experiment with Azure AI Foundry or Google AI Studio before the next session**

SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 07

Agents, multi-agent systems, middleware, MCP, guardrails & observability

Year 3 • Semester 1 • 4 Credits

Lecturer: Lasal Hettiarachchi | lasal.h.visiting@slit.lk | Course web: courseweb.sliit.lk

Today's Roadmap – 3 Hours

Block A 0–55 mins	**From tool-calling to agents** Understanding agent vs. chain architectures and their limitations
Block B 65–115 mins	**Multi-agent systems & coordination patterns** Orchestration, collaboration models, and multi-agent workflows
Block C 125–170 mins	**Middleware, guardrails, observability & evaluation** Managing safety, logging, system tracking, and LLM evaluation frameworks
Wrap 170–180 mins	**Production checklist, capstone & course close + Q&A** Best practices for deployment, final project guidance, and open discussion

Short breaks in between (55-65m & 115-125m) • Questions welcome at any point

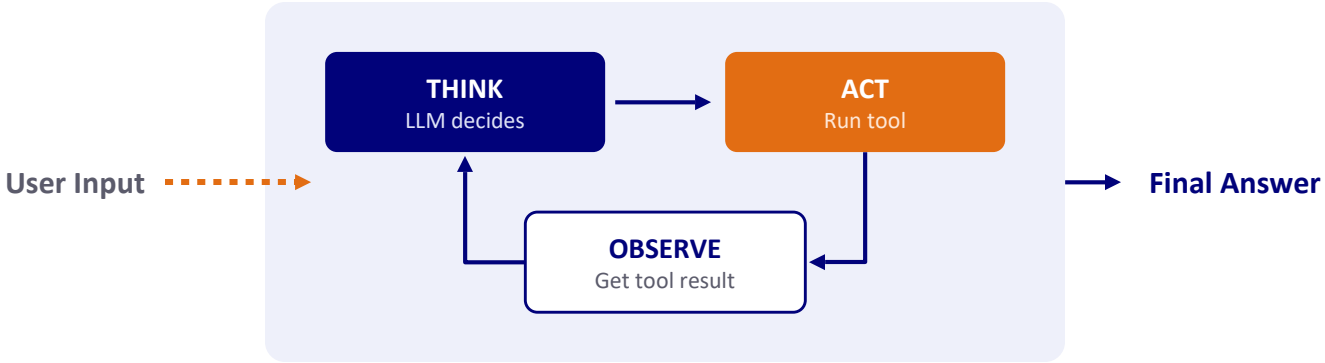
PART 1 OF 4

From tool-calling to agents



What is an agent?

An **agent** is an LLM placed **in a loop** with tools, deciding *which* tool to use, *when*, and with *what arguments* — until it can answer.



Week 2 you ran this loop manually. An agent runs it **for you**, repeatedly, stopping when the model emits a final answer (no more tool requests).

The reasoning pattern (ReAct, conceptually)

Most agents follow a *Reason + Act* rhythm:

Thought: I need the Team plan price.

Action: `search_handbook("Team plan price")`

Observation: *"Team: $25/user/month..."*

Thought: Now compute 8 users * 12 months * 0.8.

Action: `calculate("8 * 25 * 12 * 0.8")`

Observation: *1920*

Thought: I can answer now.

Answer: `"$1,920/year."`

The model interleaves reasoning with actions, using each observation to decide the next step.

That interleaving is what makes it an ***agent***, not a chain.

Tools revisited — the stakes just rose

An agent is only as good as its tools — and this week **no human is in the loop**:

The docstring is the interface

The docstring acts as an interface that helps the agent understand when and how to use the tool.

Week 2's good-tool principles (clear names, typed args, single purpose) were craft advice while *you* closed the loop. With the model choosing **unsupervised**, they become **correctness requirements**.

Retriever as a tool

A retriever can also be provided as a tool.

A retriever can be a tool → the agent decides *if*, *when*, and *how many times* to search. That's the **agentic RAG** teased at the end of Week 2's production block — today you can finally build it.

Same tools as Week 5. What changed is who's choosing — and that changes everything.

Why agents need limits

The Risk: Uncapped Loops

An agent can easily loop: call a tool call another and another ... forever.

Uncapped agents are the **#1 way to burn money** and **hang a live demo**.

How to Bound Them

- **Hard step cap**: max allowable iterations
- **Time budget**: strictly enforce a wall-clock limit
- **Error recovery**: malformed requests shouldn't crash
- **Prompt rules**: "stop and say so after N tries"

> ***"Autonomy without limits isn't intelligence — it's an infinite loop with a credit card."***

Failure modes of agents

Symptom	Cause	Mitigation
Infinite / long loops	No stop condition; ambiguous goal	**Step cap, timeout, clearer prompt**
Wrong tool chosen	Vague tool descriptions	**Better names/docstrings; fewer tools**
Hallucinated tool args	Loose arg schema	**Typed, constrained arguments**
Ignores tool result	Weak prompt; too much context	**Tighter prompt; summarise history**
Runaway cost	Too many tools/steps	**Specialise (multi-agent); cap steps**

> Notice the cure for several rows is **fewer, clearer tools** — which motivates multi-agent design (Block B).

PART 2 OF 4

Multi-agent systems & coordination



Why multi-agent?

The transition from monolithic cognitive systems to modular, collaborative intelligence.

The Monolithic Problem

One single agent loaded with **20+ tools** suffers severe limitations during execution:

- Gets **confused** selecting the right tool
- Becomes **slow** with bloated context lengths
- Extremely **hard to debug** execution pathways

The Multi-Agent Fix

Mirror software engineering practices: **specialise and coordinate** across distinct roles:

- **Researcher**: Equipped with search tools
- **Coder & Writer**: Focused prompt & toolsets
- **Coordinator**: Orchestrates and routes tasks

Mental model: **Each agent receives only the tools and instructions needed for its role. This is similar to microservices, where each service has one clear responsibility.**
Several focused agents can work more effectively than one overloaded agent.

Benefits & costs of going multi-agent

A strategic evaluation of architectural tradeoffs in collaborative cognitive systems.

Architectural Benefits

- **Focused prompts & toolsets:** Leads to highly specialized agents and better, more accurate decisions.
- **Independently testable:** Isolated agent behaviors make execution paths easy to debug and verify.
- **Reusable specialists:** Modular agent roles can be composed and shared across diverse tasks.
- **Separation of concerns:** Decouples complex workflows into manageable, single-responsibility units.

System Costs & Friction

- **More total LLM calls:** Increased delegation leads to higher token consumption and cost.
- **Coordination complexity:** Managing state, routing, and reliable agent interaction adds systemic overhead.
- **Increased latency:** Sequential handoffs and recursive reasoning cycles slow down overall response times.
- **Observability challenge:** Multi-step cognitive pathways are harder to trace without advanced telemetry.

> *Don't reach for multi-agent reflexively. Use it when a single agent is* **demonstrably overloaded** *or when you have* **genuinely distinct specialties**.

Four coordination patterns

Architectural mechanisms for orchestrating multi-agent workflows.

Router

Classify, then dispatch to **one** specialist.

WHEN:
Distinct categories

Pipeline

Output of one feeds the next (research → write → edit).

WHEN:
Fixed stages

Parallel

Run specialists at once, then merge results.

WHEN:
Independent subtasks

Supervisor

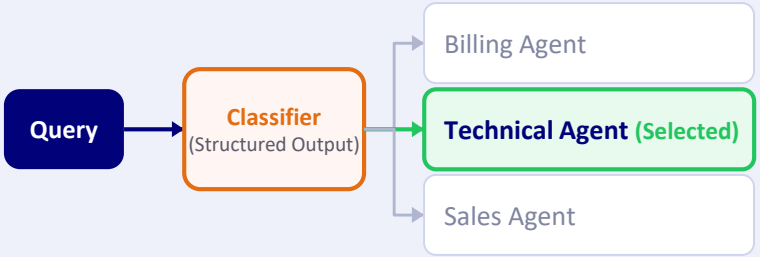
A coordinator calls specialists **as tools**, dynamically.

WHEN:
Unknown, multi-step plan

*The first three patterns normally **follow a predefined workflow**, while the supervisor makes decisions during execution. The supervisor is the only one that's itself an agent.* **Multi-agent ≈ LCEL composition of agents.**

Pattern 1 — Router (dispatch)

A structured-output classifier picks exactly one specialist to handle the incoming request.



Pattern Details

Use when:

Requests cleanly partition into distinct categories.

Strengths:

- **Cheap:** Minimal LLM tokens used.
- **Predictable:** Direct routing logic.
- **Easy to Test:** Evaluate components in isolation.

> ***Classify the request first, and then dispatch it to the correct specialist.***

Pattern 2 — Pipeline (sequential)

Output of one feeds the next — fixed, known stages with no dynamic decisions.

Pipeline Diagram



When to Use

The workflow has clear, ordered, and structured phases with predictable transitions between each specialized task.

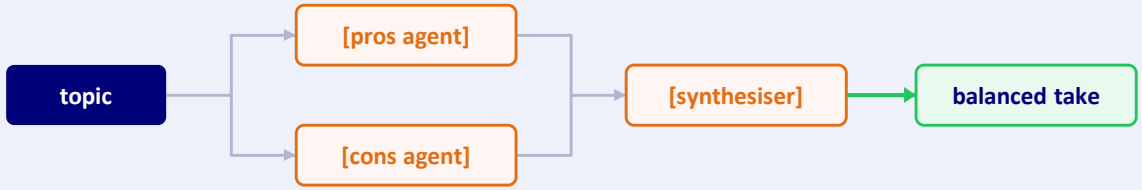
Key Strengths

- **Simple & Predictable:** Sequential execution flow.
- **Plain LCEL:** Next stage strictly consumes the previous output.
- **Independently Improvable:** Optimize each stage in isolation.

> ***If you can define all the stages in advance, a pipeline is usually better than a supervisor agent.***

Pattern 3 — Parallel (fan-out / merge)

Independent subtasks run concurrently, then a step merges them.



Mapping Architecture

Directly maps to Week-1 **parallel + lambda/merge** structures.

When to Use

Apply this pattern when the required subtasks are completely independent of each other and can be executed simultaneously without needing intermediate results from one another.

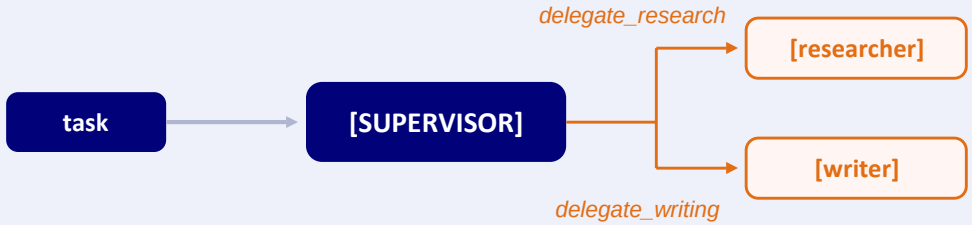
Key Strengths

- **Faster Execution:** Concurrency significantly reduces total latency.
- **Diverse Perspectives:** Allows specialized agents to analyze the same topic from contrasting angles before synthesis.

Run independent tasks together, and then combine their results.

Pattern 4 — Supervisor (orchestrator)

An orchestrator agent dynamically plans and delegates tasks to specialized agent tools.



Agents-as-Tools

A specialist agent becomes just another standard tool that the supervisor dynamically calls.

When to Use

Apply this pattern when the execution plan is unknown up front and requires dynamic, step-by-step orchestration, planning, delegation, and observation over multiple iterations.

Key Strengths

- **Dynamic Flexibility:** Plans and adjusts paths on the fly.
- **Agentic Tools:** Exposes complex sub-agents through a clean tool interface.

The supervisor plans and coordinates while specialists perform focused tasks.

Four multi-agent architectures

Architectural mechanisms for orchestrating multi-agent workflows.

Subagents

A With subagents, one main agent remains in control and calls specialist agents as tools.

All subagent results return to the main agent before the workflow continues.

Handoffs

With handoffs, control can move from one agent to another based on the current state.

For example, a support agent may hand the conversation to a billing agent.

Skills

With skills, one agent stays in control but loads specialized instructions or knowledge only when needed.

Router

With a router, the request is classified and directed to one or more suitable agents.

Their outputs can then be combined into one final response.

Reference: https://docs.langchain.com/oss/python/langchain/multi-agent#handoffs

Choosing a pattern — decision guide

Select the simplest agentic coordination structure that satisfies your architecture requirements:

If your architectural question is...	Then utilize this pattern →
Distinct categories, one handler each?	**Router**
Fixed, ordered stages?	**Pipeline**
Independent subtasks to merge?	**Parallel**
Plan is dynamic / discovered at run time?	**Supervisor**

Strategic Advice: Start at the **simple end** (router/pipeline). Move toward **supervisor** only as the plan becomes genuinely dynamic — you trade predictability and cost for flexibility.

Handoffs — what does the specialist actually see?

A **handoff** is the information passed from one agent to another.

Handoff style	Passes	Trade-off
Full transcript	**everything so far**	max context; max tokens; **distracts the specialist**
Task brief	**goal + constraints + key facts**	cheap & focused; **risks omitting context**
Structured brief	**typed schema** (`task, inputs, output`)	machine-checkable; **needs upfront design**

Treat every handoff like an **API contract**, clearly define what information is passed and what result is expected. **Pass enough context to complete the task, but avoid unnecessary information.**

Shared state — save outputs, pass references

Keep large reports, datasets, and code outside prompts. Share a **file path or record ID** instead.

1. Save and share

- **Save large outputs:** Specialists store reports, datasets, or code in a file, database row, or document store.
- **Return a reference:** Share a path or record ID—such as **research/acme.md**—so another agent opens it only when needed.

2. Why it helps

- **Fewer tokens:** Shorter prompts use fewer tokens and keep other agents focused.
- **Recover and audit:** Saved work survives crashes and makes each handoff easier to inspect and audit.

Key idea: share a **document link** instead of copying the whole document into every message.

Partial failure — follow these four steps

When one agent fails, the supervisor should **follow four clear recovery steps**.

01 / Retry

Try the failed task again with limited attempts and increasing wait times.

02 / Fall Back

Use a backup model, cached result, or a simpler but honest answer.

03 / Re-Plan

Tell the supervisor what failed so it can change the plan or assign another agent.

04 / Fail Loudly

Clearly tell the user when the task failed or the answer is incomplete.

Recovery flow: Retry → Fall Back → Re-Plan → Fail Loudly.

Control cost and latency as the workflow grows

Every result adds to the supervisor's context, so later steps become slower and more expensive.

Model Tiering

Use cheaper models for routing and simple tasks. Reserve stronger models for complex decisions.

Parallel Fan-Out

Run independent agents together. Total latency becomes the longest branch, not the sum.

Summaries Hops

Compress specialist outputs before adding them to the supervisor's context.

Caching

Reuse stable prompts and deterministic tool results instead of computing them again.

Budgets

Set limits for tokens, steps, time, and cost—and stop or reroute when exceeded.

Control context growth early so coordination does not become too slow or expensive.

Least privilege — limit each agent's access

Multi-agent design should separate capabilities as well as responsibilities.



01 / Compartmentalize

Give each agent only the tools and credentials it needs:

- **Researcher:** read-only search
- **Writer**: no unnecessary tools
- No agent: unrestricted shell

02 / Limit the Blast Radius

A compromised agent cannot use tools it never received.

Example: without an email tool, a research agent cannot send messages.

03 / Add Approval Gates

Require approval before sending email, deleting files, making payments, or deploying.

Specialists propose actions; the supervisor or a human authorizes them.

An attack in one compartment should not spread—similar to IAM for each microservice.

Observing & evaluating a multi-agent system

How to gain visibility, measure performance, and establish systematic quality gates:

01 / Traces become trees

Hierarchical Run IDs

A multi-agent execution trace looks like a tree. The supervisor is the parent run, and each specialist creates a child run.

Connected run IDs help us inspect the complete execution path and identify where a failure occurred.

02 / Metrics per agent

Granular Accounting

Track cost, latency, and failure rate **by specialist**. Pinpoint precisely which agent is the slow, expensive, or flaky bottleneck in the system.

03 / Multi-Level Evals

Dual Testing Strategy

- **Component evaluation:** test individual parts, such as router precision, F1 score, or specialist accuracy.
- **End-to-end evaluation:** test whether the complete system successfully finishes real scenarios.

The Multi-Agent Payoff for Testing: Component-level evaluations are what let you **swap one agent's model** and verify that absolutely nothing else broke across the rest of the architecture.

Discussion checkpoint #2 (5 min)

With a neighbour, pick an agent coordination pattern and justify your choice for each scenario:

Scenario 01

Ticket Router: A helpdesk that sends each ticket to billing, tech, or sales.

Pattern tip: Consider a router/supervisor or a classification chain that statically dispatches requests based on intent.

Scenario 02

Blog Factory: Research a topic, draft a post, and then copy-edit it.

Pattern tip: Look at a sequential pipeline or a workflow with dedicated agents passing structured handoffs.

Scenario 03

Dynamic Entry: "Plan and execute a market-entry analysis" — steps unknown.

Pattern tip: Think about a dynamic plan-and-solve agent or supervisor loop that decomposes the goal on the fly.

Discussion Goal: Map these scenarios to specific architectures (e.g., chains, routers, supervisor/worker, or fully autonomous DAG execution) and justify the trade-offs in complexity.

PART 3 OF 4

Middleware, guardrails, observability & evaluation



Cross-cutting concerns

As agents grow, you need behavior that **isn't** the agent's core "thinking". We want them **separated**.

The Problem: Scattered Logic

- Cross-cutting concerns are supporting functions, not the agent's main reasoning.
- Examples include logging, retries, approvals, authentication, redaction, and tracing.
- Adding these functions directly into the agent logic creates complex and messy code.
- It also makes the agent harder to understand, test, reuse, and maintain.

The Solution: Two Clean Mechanisms

1. Middleware: wraps the agent's execution and can modify inputs, outputs, or behavior. (middleware can retry a failed tool call or remove sensitive information.).

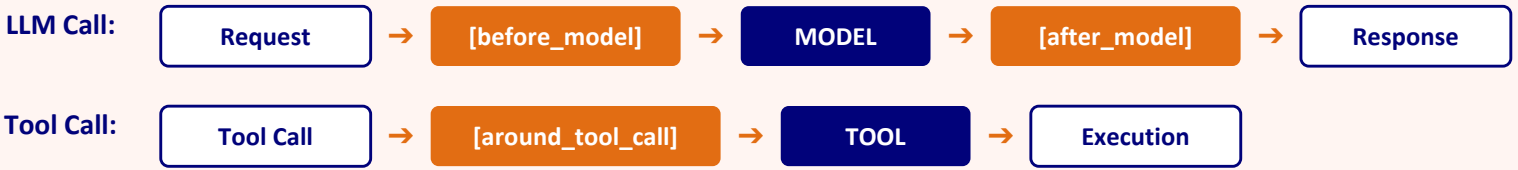
2. Callbacks: observe execution events for monitoring, tracing, and logging. Callbacks normally observe the process without changing it.

Middleware wraps and modifies execution steps directly, while **Callbacks** provide read-only observability to monitor events as they happen.

Middleware — concerns *around* the loop

Middleware works around the agent loop, similar to middleware in web applications. It adds supporting behaviour without rewriting the agent's main reasoning logic.

Visualizing the Loop Wrapper



Execution Hooks

before model call (each LLM step)
→ it can add context, trim old history, or validate the request.

after model call (each LLM step)
→ it can log, validate, or remove sensitive information from the response.

around tool call (each tool run)
→ It can handle authorization, retries, caching, timeouts, and sandboxing.

run boundaries (start / end)
→ It can collect performance metrics and auditing information.

Key Takeaway

Middleware keeps operational tasks separate from the agent's decision-making process.

Middleware in practice (concept)

Middleware can be either built-in or custom-made.

Built-in Middleware

provides common features that can be used immediately.

- **History Summarization:** automatically shortens old conversation history to reduce token usage.
- **Human-in-the-Loop:** pauses risky actions and requests approval from a person.
- **Guardrails & Retries:** detect errors, enforce safety rules, and retry failed operations.

Custom Middleware

is created for the specific requirements of an application.

- **before_model:** checks or modifies the prompt before it reaches the model.
- **wrap_tool_call:** checks tool parameters and prevents unauthorized actions..

Middleware keeps safety, cost control, and logging separate from the main business logic.

Guardrails — layered safety

Guardrails are safety controls placed around an AI agent.Guardrails apply at **three layers**:

01 / INPUT

Input Layer

Reject prompt-injection, off-topic, or unsafe requests **before** the model runs.

02 / TOOL

Tool Layer

Allow-list tools, validate arguments, sandbox execution, and require **human approval** for destructive actions.

03 / OUTPUT

Output Layer

validate the response format, remove sensitive data, and prevent **secret leakage.**

Golden Rule: Never let an agent run arbitrary shell, SQL, or `eval` from raw model text. Restrict what the agent is allowed to do instead of simply trusting it.

Observability — you can't fix what you can't see



- Observability means understanding what happens inside the agent while it is running.
- The final answer alone does not show how the agent reached its decision.
- Important hidden details include the selected tools, arguments, model calls, errors, and repeated loops

Two complementary lenses:

Callbacks are triggered when specific events occur.

- ***Examples*** include model start/end, tool start/end, errors, and token streaming.
- Callbacks are useful for logging, cost tracking, monitoring, and displaying live progress.

Tracing records the agent's complete execution path from start to finish.

- Tools such as LangSmith can visually display every step taken by the agent.

Difference: A callback responds to an individual event, while a trace connects all events into one complete journey.
If we can clearly observe the agent's actions, we can debug problems much faster.

PART 4 OF 4

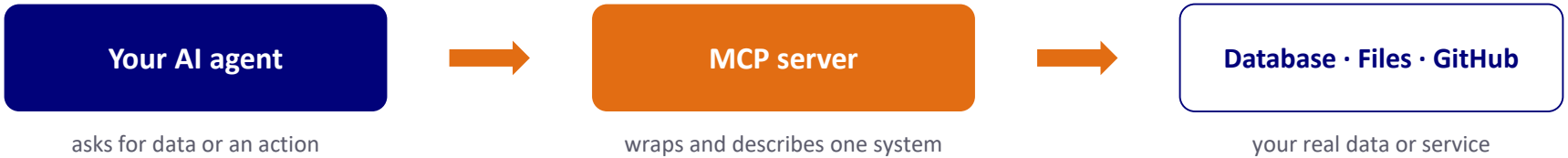
Model Context Protocol (MCP) Servers



What Is an MCP Server?

The one idea to hold on to before we look at any architecture.

An MCP server is a **small program that wraps one system** and exposes it to **any AI application** through **one shared protocol**.



Everyday analogy: a power socket. Any appliance with the right plug just works — and it never needs to know how the power station runs.

The M × N Integration Problem

MCP reduces the custom work needed to connect AI applications with tools and data.

Without MCP (M × N)

- **Separate connectors:** Each AI app needs its own link to every tool or data source.
- **Example: 3 × 4 = 12** custom integrations for 3 AI apps and 4 tool/data services.
- **Duplicated code:** Similar adapters are rewritten.
- **Inconsistent security:** Each integration handles access differently.
- **High maintenance:** One API change can break several integrations.

With MCP (M + N)

- **Standard interface:** AI apps and servers use the same protocol.
- **Example: 3 + 4 = 7** reusable components: 3 app integrations + 4 server integrations.
- **Reuse & discovery:** Apps discover tools dynamically and reuse compatible servers.
- **Consistent controls:** Hosts can apply permissions, authentication and human approval.
- **Less maintenance:** Reusable integrations reduce duplicated work.

Think USB-C: One common interface for many systems. MCP makes integration simpler and tools easier to reuse and maintain.

MCP Architecture

*MCP architecture contains three main components: **Host, Client**, and **Server.** A decoupled, stateful client-server topology communicating via JSON-RPC 2.0.*

01 / MCP HOST

The User Application

Such as Claude Desktop, Cursor, VS Code, or a custom Python/TypeScript agent runner.

Owns the LLM: Manages reasoning loops, model API keys, conversation history, and user UI.

Security Authority: Intercepts tool calls, enforces permission allow-lists, and requests user approval before execution.

02 / MCP CLIENT

Protocol Connector

is located inside the host and acts as the protocol connector; maintains 1:1 or 1:Many connections to MCP servers.

Lifecycle Manager: Spawns server subprocesses or establishes network SSE streams; conducts handshakes.

Capability Registry: Aggregates schemas from all connected servers and presents them cleanly to the LLM.

03 / MCP SERVER

Domain Specialist

Lightweight external program exposing specific tools, resources, and prompt templates.

Model-Agnostic: Zero dependency on LLM providers; purely executes JSON-RPC requests.

Isolated Boundaries: Encapsulates database drivers, native binaries, and private API credentials securely.

Protocol Dataflow: Host (LLM + UI) ⇄ MCP Client (1:Many Routing) ⇄ [JSON-RPC 2.0 / stdio or SSE] ⇄ MCP Servers (DB, Files, APIs) . The LLM does not directly access external systems; the MCP architecture controls and separates each responsibility.

Three Core Primitives — Resources, Prompts & Tools

MCP standardizes data exchange and execution into three strictly typed primitives.

01 / RESOURCES

GET / Read-Only Context

provide information that the AI can read and use as context. ***Examples*** include files, database schemas, logs, and documentation.

URI-based addressing: e.g., postgres://schema/public, file:///app/config.json.

Dynamic Subscriptions: servers can notify clients when resource contents update.

Risk Level: Low (read-only data)

02 / PROMPTS

Interactive Templates

are reusable instruction templates or predefined workflows. They can accept **parameters**, such as a query to analyse or an error to debug.

User Triggers: Prompts are normally selected or triggered by the user.

Risk Level: Low (guided user interaction)

03 / TOOLS

POST / Model Actions

are functions that allow the AI to perform actions. ***Examples*** include sending an email, updating a database, or creating a file.

JSON Schema: Tool parameters are defined using a schema so the inputs can be validated.

High Impact: requires host guardrails and human approval gates for writes.

Risk Level: High (autonomous action)

Summary: Resources provide data, prompts provide guidance, and tools perform actions.

SE3090 – Software Engineering Frameworks

Lecture 07 • Closing

Questions & Discussion

Thank you

Next: Lecture 08 — Git, Collaborative Development, CI/CD, Security & Code Quality

SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 09

Introduction to Flutter and Mobile Application Development



Year 3 • Semester 1 • 4 Credits • Duration: 3 Hours

Learning Outcomes: LO1, LO2, LO4 | Mobile App Development Frameworks

Where This Lecture Fits

L01	L02	L03	L04	L05	L06	L07	L08	L09	L10	L11

11 lectures — today we zoom into mobile development

This module section

Mobile App Development Frameworks

15 contact hours • spans Lectures 09 – 10

- Mobile application architecture across platforms
- Building cross-platform apps — introduction to Flutter
- Accessing device features, handling UI, managing data (Lecture 10)

In the next 3 hours you will learn

- Native, cross-platform, hybrid & PWA approaches
- What Flutter is and how it draws every pixel
- The Dart language — null safety & async basics
- Widgets: the building blocks of every screen
- State, events & navigation between screens
- Layered mobile architecture for maintainable apps
- Building “TaskFlow” — a complete cross-platform app

Learning Outcomes for This Lecture

LO1

Evaluate frameworks

Evaluate the types of mobile frameworks used in modern software — native, cross-platform, hybrid and PWA.

LO2

Apply & build

Apply Flutter, Dart and widget-based design to build a working cross-platform mobile application.

LO4

Select & justify

Select and justify an appropriate mobile framework and architecture for given project requirements.

By the end of today you will be able to…

- Explain how cross-platform frameworks build one codebase for Android and iOS, and when to choose them.
- Read and write basic Dart, compose widgets into screens, manage state with setState(), and navigate between screens.
- Describe a layered mobile architecture and structure a small Flutter project the way a professional team would.

Why Mobile Development — and Why Flutter?

Why mobile matters

Mobile-first world

Most digital traffic and many products are now phone-first, not desktop-first.

Always-on engagement

Push notifications, location and the camera make apps central to modern systems.

Business demand

Banking, health, retail and logistics all ship mobile apps — strong job market.

Why Flutter for this module

- **One codebase → Android, iOS, web & desktop**
- Backed by Google, free and open-source
- **Fast UI with a rich widget library**
- Hot reload makes the build-see loop almost instant
- **Great fit for full-stack student projects**
- Pairs naturally with the REST APIs you built earlier in SE3090
- **In-demand skill for internships and industry**

Today’s Roadmap — 3 Hours

Hour 1

Foundations & frameworks

Mobile approaches (native / cross-platform / hybrid / PWA) • framework landscape • cross-platform development • selection criteria • Checkpoint 1

Hour 2

Flutter, Dart & widgets

What Flutter is & its architecture • Dart basics, null safety, async • project structure • widgets, layout, lists, input • reusable widgets • Activity

Hour 3

State, navigation, architecture & build

setState & events • forms & validation • navigation • layered architecture • building TaskFlow end-to-end • mistakes • Quiz

Short breaks after each hour — ask questions at any time.

PART 1 OF 6

Mobile Development Foundations



- What mobile application development really involves
- Four ways to build: native, cross-platform, hybrid, PWA
- The framework landscape and the Android / iOS ecosystems
- Benefits, limitations and how teams choose a framework

What Is Mobile Application Development?

Definition. Designing, building, testing and maintaining software that runs on mobile devices — smartphones and tablets — making use of their screen, touch input, sensors, camera, location and connectivity.

Role in modern software systems

The front door

Often the main way users reach a system — the client for your backend APIs.

Device-aware

Uses GPS, camera, notifications and offline storage that web apps cannot easily reach.

Connected

Talks to REST services, databases and cloud — part of a larger full-stack system.

Experience-critical

Performance, responsiveness and offline behaviour shape how users judge the product.

Four Ways to Build a Mobile App



Native

Kotlin/Java • Swift

- Built per-platform with the vendor SDK
- Best performance & full device access
- Two codebases → double the work



Cross-platform

Flutter • React Native

- One codebase → Android + iOS
- Near-native performance
- Today's focus — best balance



Hybrid

Ionic • Cordova

- Web tech (HTML/CSS/JS) in a shell
- Fast to reuse web skills
- Heavier, less native feel



Progressive Web App

Installable website

- Runs in the browser, installable
- No app store needed
- Limited device access

Comparing the Four Approaches

Criteria	Native	Cross-platform	Hybrid	PWA
Codebases	One per platform	Single	Single	Single
Performance	Highest	Near-native	Moderate	Browser-bound
Device access	Full	Very good	Limited	Limited
Dev speed / cost	Slowest • high	Fast • low	Fast • low	Fastest • lowest
App store	Yes	Yes	Yes	No (web)
Best for	Performance / HW-heavy	Most apps, MVPs	Content + web reuse	Reach, light apps

Rule of thumb: start cross-platform; go native only when a hard requirement (performance or deep hardware) forces it.

Frameworks & the Android / iOS Ecosystems

Android ecosystem

~70% of phones worldwide

- Language: Kotlin (or Java)
- Tools: Android Studio, Gradle
- Store: Google Play — open distribution

iOS ecosystem

Premium, tightly controlled

- Language: Swift (or Objective-C)
- Tools: Xcode — macOS only
- Store: App Store — strict review

Common mobile development frameworks

Flutter

Dart • Google

React Native

JS • Meta

.NET MAUI

C# • Microsoft

Kotlin MP

Kotlin • JetBrains

Ionic

Web • hybrid

Benefits, Limitations & Selection Factors

Benefits of using a framework

- Reusable UI components & proven patterns
- Faster delivery — less boilerplate
- Large community, packages & documentation
- Cross-platform reach from one codebase

Limitations to watch

- Abstraction can hide platform details
- Large apps & bigger binary size
- New platform features may arrive late
- Learning curve for the framework itself

Factors that drive framework selection

Project requirements	**Performance needs**	**Team skills**	**Timeline & budget**	**Device features**	**Long-term support**

PART 2 OF 6

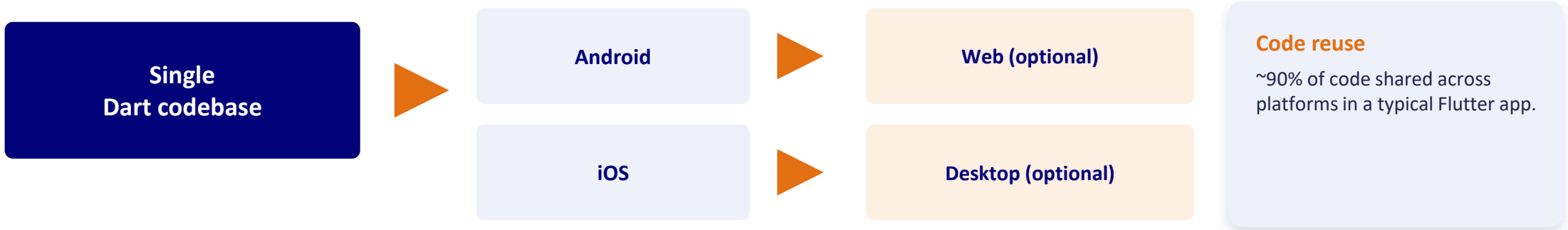
Cross-Platform Development



- What 'single codebase' really means in practice
- The benefits — and the honest trade-offs
- The landscape: Flutter, React Native, MAUI, KMP, Ionic
- Criteria and a simple way to choose

What Is Cross-Platform Development?

Idea. Write your application logic and UI once in a single codebase, then compile or run it on multiple platforms — Android and iOS (and often web and desktop) — instead of building a separate app for each.



Code reuse

~90% of code shared across platforms in a typical Flutter app.

- One language, one team, one set of tests — not two.
- Platform-specific tweaks are still possible where needed.
- Faster releases: fix a bug once and it ships everywhere.

Benefits & Trade-offs of Cross-Platform



Lower cost

One team and codebase instead of two.



Faster to market

Ship to both stores from a single build.



Consistent UX

Same look and behaviour on every device.



Easier maintenance

Fix and test once; everyone benefits.

Trade-offs to plan for — performance & maintenance considerations

- Performance: excellent for typical apps; very heavy graphics or compute-bound games may still favour native.
- Binary size: cross-platform apps ship a runtime, so installs are a little larger.
- Platform parity: a brand-new OS feature can take time to reach the framework — plan around release timing.
- Native escape hatch: when you truly need it, you can call platform code — but that adds per-platform work.

The Cross-Platform Landscape

Framework	Language	Backed by	Strengths	Typical use
Flutter	Dart	Google	Fast UI, one widget set, great tooling	Most new cross-platform apps
React Native	JavaScript	Meta	Reuse React skills, huge ecosystem	Web teams going mobile
.NET MAUI	C#	Microsoft	Native controls, .NET integration	Enterprise / Microsoft shops
Kotlin Multiplatform	Kotlin	JetBrains	Share logic, native UI per platform	Android-first teams
Ionic	HTML/CSS/JS	Ionic	Web tech, quick hybrid apps	Content apps, web reuse

This module: Flutter — a single, consistent widget toolkit and excellent developer experience.

Choosing a Mobile Framework

Requirements

Features, performance & device access the app must have.

Team skills

What languages your developers already know well.

Time & budget

Deadline pressure and available resources.

Support & longevity

Community size, backing and update cadence.

A simple decision flow

Need extreme perf or deep HW?

→ Native

Strong web/React team & deadline?

→ React Native

Want one UI, top tooling?

→ Flutter

Mostly content / web reuse?

→ PWA / Ionic

Checkpoint 1 — Match the Scenario



In pairs (4 minutes): which approach or framework fits each scenario, and why? Match to native, cross-platform (name one), hybrid or PWA.

1. A startup needs an MVP on both stores in 6 weeks with a small team.
2. A 3D-heavy mobile game that must squeeze every frame from the GPU.
3. A university wants a lightweight, installable notice-board reachable from any browser.
4. A company with a strong C#/.NET team wants an internal staff app.
5. An existing React web team must ship a mobile version of their product quickly.

Pair up — then we compare answers as a class.

PART 3 OF 6

Flutter & Dart Foundations



- What Flutter is and how it renders every pixel
- Hot reload, architecture, advantages and limits
- The Dart language: variables, control flow, classes
- Null safety and asynchronous programming basics

What Is Flutter?

Flutter is Google's open-source UI toolkit for building natively-compiled apps for mobile, web and desktop from a single Dart codebase.



Open-source • Google

Stable, widely adopted in industry

Draws its own UI

Flutter paints every pixel with its engine instead of wrapping native controls — identical look on every platform.

Everything is a widget

Screens are built by composing small widgets into a tree — the core idea you'll use all day.

Powered by Dart

A clean, typed language built for UIs, compiled ahead-of-time to fast native code.

Hot reload

See code changes in under a second without losing app state — a huge productivity boost.

Flutter Architecture

Three layers — your Dart code sits on top; the engine does the drawing.

Framework (Dart)

Material & Cupertino widgets • rendering, animation, gestures • your app code

Widgets

Rendering

Foundation

Engine (C/C++)

Skia/Impeller graphics • Dart runtime • text layout • low-level paint

Graphics

Dart VM

Text

Embedder (per-platform)

Hosts Flutter on Android, iOS, web, desktop • input, surface, lifecycle

Android

iOS

Web / Desktop

Your app *lives in the Framework layer. You rarely touch the engine or embedder — Flutter handles them for you.*

Drawing the UI & the Hot-Reload Workflow

How Flutter turns your code into pixels

1 Widget tree

You describe the UI as nested widgets (config).

2 Element tree

Flutter builds & diffs the live element tree.

3 Render tree

Layout + size each box; decide what to paint.

4 Engine paints

Skia / Impeller draws the frame to the screen.

The workflow superpower

Hot Reload

< 1 second

- Injects changed code into the running app
- Keeps current state (you stay on the same screen)
- Use constantly while tweaking UI

Hot Restart

a few seconds

- Restarts the app from scratch — state is lost
- Needed after changing main(), globals or state classes
- Still far faster than a native rebuild

Rule: *reach for hot reload first; hot restart only when reload says it can't apply the change.*

Flutter: Advantages, Limitations & What You Can Build

Advantages

- Single codebase → Android + iOS (and web/desktop)
- Fast development with hot reload
- Consistent UI — Flutter draws every pixel
- Great performance (compiled to native)
- Rich widget library: Material & Cupertino
- Strong community, backed by Google

Limitations

- Larger app size than pure native
- New platform widgets can arrive late
- Dart is less common than JS/Kotlin
- Very heavy 3D / games still favour native
- Some deep-platform features need plugins

What you can build

- Business & productivity apps
- E-commerce and delivery apps
- Social and content apps
- Dashboards and internal tools
- MVPs and startup products
- TaskFlow — our case study today

Dart Basics 1 — Variables, Types & Operators

Dart is statically typed but friendly: annotate types, or let var infer them.

main.dart

```dart
void main() {
  String name = 'Flutter';    // text
  int year = 2026;            // whole number
  double rating = 4.8;        // decimal
  bool isStable = true;       // true / false
  var language = 'Dart';      // type inferred
  const pi = 3.14;            // compile-time const

  // string interpolation with $
  print('$name in $year');
  print('Rating: $rating, stable: $isStable');

  // operators
  int total = year + 4;       // 2030
  bool good = rating > 4.0;   // true
}
```

Core types

- String — text in quotes
- int / double — numbers
- bool — true or false
- var — infer from value
- const / final — unchanging

Remember

- Use $var or ${expr} inside strings
- final = set once at runtime
- const = fixed at compile time
- Prefer explicit types in public APIs

Dart Basics 2 — Control Flow & Functions

control_flow.dart

```dart
// conditions
int score = 72;
if (score >= 50) {
  print('Pass');
} else {
  print('Retry');
}

// classic for loop
for (int i = 1; i <= 3; i++) {
  print('Task $i');
}

// for-in over a list
final tasks = ['Study', 'Code', 'Test'];
for (final t in tasks) {
  print(t);
}
```

functions.dart

```dart
// simple typed function
int add(int a, int b) {
  return a + b;
}

// arrow (one-line) function
int square(int n) => n * n;

// named + default parameters
String greet(String name,
    {String title = 'Student'}) {
  return 'Hello $title $name';
}

void main() {
  print(add(2, 3));         // 5
  print(greet('Sara'));     // Hello Student Sara
}
```

Dart Basics 3 — Classes & Objects

Dart is object-oriented — and your Flutter widgets are just classes.

task.dart

```dart
class Task {
  final String title;   // set once
  bool done;            // can change

  // constructor with named param + default
  Task(this.title, {this.done = false});

  // method
  void toggle() => done = !done;
}

void main() {
  final t = Task('Read chapter 9');
  print(t.done);   // false
  t.toggle();
  print(t.done);   // true
}
```

Anatomy of a class

- Fields — the data it holds
- Constructor — how you create one
- Methods — what it can do
- this.title — shortcut to assign a field

Why this matters for Flutter

- Every widget is a class
- A screen = objects composed together
- Your data models (like Task) are classes
- You'll subclass StatelessWidget next

Modern Dart — Null Safety & Async

Null Safety

Types are non-null by default — fewer crashes.

null_safety.dart

```dart
String name = 'Sara';    // never null
String? nickname;        // may be null

// safe access + default
int len = nickname?.length ?? 0;
print(len);    // 0

nickname = 'Sash';
// '!' asserts it is not null now
print(nickname!.length);  // 4
```

Async (Future / await)

Do slow work without freezing the UI.

async.dart

```dart
Future<String> loadTasks() async {
  await Future.delayed(
      Duration(seconds: 2));
  return 'Tasks loaded';
}

void main() async {
  print('Loading...');
  final msg = await loadTasks();
  print(msg);    // after 2s
}
```

? may be null **??** default if null **!** trust me, not null **async/await** wait without blocking

Checkpoint 2 — Read the Dart Code

Trace each snippet on paper. What does it print — and why? (3 minutes, then we compare.)

A

```dart
int add(int a, int b) => a + b;
void main() {
  print(add(4, 5));
}
```

B

```dart
String? city;
int n = city?.length ?? -1;
print(n);
```

C

```dart
final t = Task('Email', done: false);
t.toggle();
print(t.done);
// class Task { ... toggle() => done = !done; }
```

D

```dart
var nums = [1, 2, 3];
var sum = 0;
for (final x in nums) { sum += x; }
print(sum);
```

PART 4 OF 6

Building User Interfaces with Widgets



- Setting up the SDK and running your first app
- Project structure and the “everything is a widget” model
- Stateless vs stateful widgets and the app skeleton
- Core widgets, layouts, lists and reusable components

Setting Up & Running Flutter

One-time setup

1 Install the Flutter SDK

Download from flutter.dev; add it to your PATH.

2 Install an editor

VS Code or Android Studio + the Flutter / Dart plugins.

3 Set up a device

Android emulator, iOS simulator (Mac) or a real phone.

4 Verify the toolchain

Run flutter doctor and fix every ✗ it reports.

Everyday commands

terminal

```
flutter doctor      # check setup
flutter create app  # new project
cd app
flutter run         # build & launch
r  = hot reload   R = hot restart
q  = quit the app
```

If something breaks

- flutter doctor → read the ✗ lines first
- No device? start an emulator/simulator
- Weird errors? flutter clean, then run again

Flutter Project Structure

What flutter create gives you — and how to grow it sensibly.

Key files & folders

`lib/` All your Dart code lives here.

`lib/main.dart` Entry point — runApp() starts the app.

`pubspec.yaml` Dependencies, assets, fonts — the project manifest.

`android/` `ios/` Native project wrappers (rarely edited).

`test/` Automated tests for your app.

A scalable folder layout

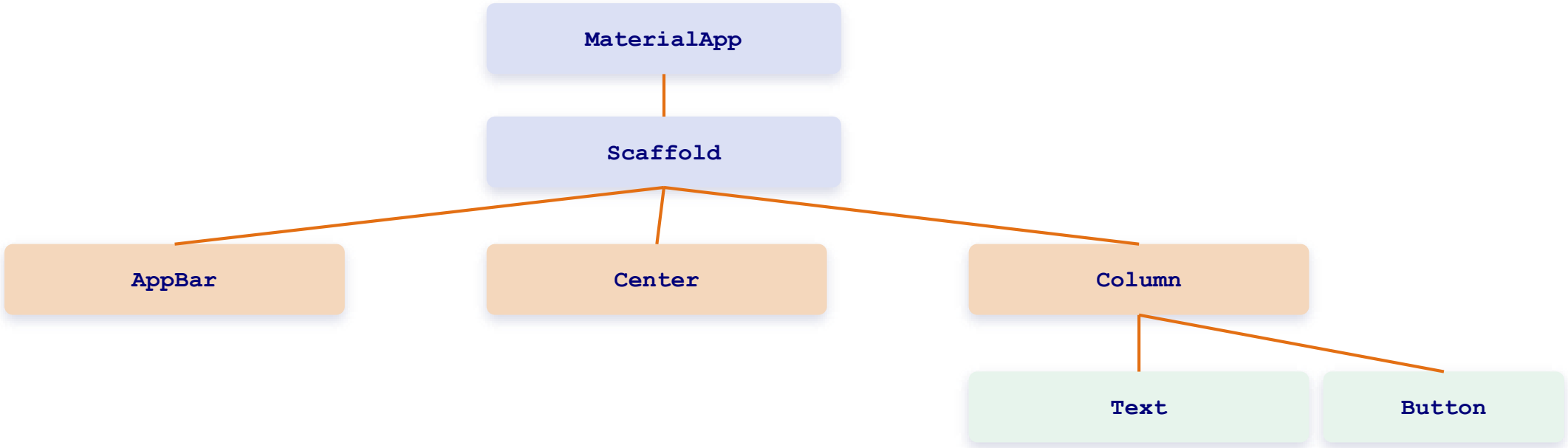
lib/

```
lib/
  main.dart           app entry
  models/             data classes
    task.dart
  screens/            full pages
    home_screen.dart
    add_task_screen.dart
  widgets/            reusable UI
    task_tile.dart
  services/           data / API
    task_repository.dart
```

Everything Is a Widget

The core idea. In Flutter, every part of the UI — text, a button, padding, even the whole screen and the app itself — is a widget. You build screens by composing small widgets into a tree.

A simple widget tree



Parent widgets hold child widgets — nesting builds the screen.

Stateless vs Stateful Widgets

Two kinds of widgets — the difference is whether they can change over time.



StatelessWidget

Never changes after it is built

- Draws once from the data it is given
- No internal mutable state
- Rebuilds only if its inputs change
- Lightweight and simple

Use for: labels, icons, a static task card, layout.

Example: TaskCard showing a title



StatefulWidget

Can rebuild itself when data changes

- Holds mutable state in a State object
- Call setState() to update the UI
- Rebuilds in response to interaction
- Slightly more code than stateless

Use for: checkboxes, forms, counters, toggles.

Example: a Task that can be ticked done

The App Skeleton — MaterialApp, Scaffold, AppBar

main.dart

```dart
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'TaskFlow',
      home: Scaffold(
        appBar: AppBar(title: Text('TaskFlow')),
        body: Center(child: Text('Hello, Flutter!')),
      ),
    );
  }
}
```

`runApp()`

Boots the app with your root widget.

`MaterialApp`

Wraps the app; themes, routes, title.

`Scaffold`

Standard screen: app bar + body.

`AppBar`

The top bar with a title.

`body`

The main content area of the screen.

`build()`

Returns the widget tree to display.

Core UI Widgets You'll Use Constantly



Text

Display a string, styled with TextStyle.



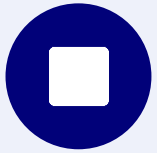
Image

Show assets or network images.



Buttons

ElevatedButton, TextButton, IconButton.



Container

A box: size, padding, colour, decoration.



Card

Rounded, elevated surface for grouping.



Padding

Adds space around a child widget.



Icon

Built-in Material icons via Icons.*



ListTile

Ready-made row: leading, title, trailing.

Layout Widgets — Row, Column & Stack

Arrange children horizontally, vertically, or layered on top of each other.

layout.dart

```dart
Column(
  crossAxisAlignment:
      CrossAxisAlignment.start,
  children: [
    Text('TaskFlow',
      style: TextStyle(fontSize: 22)),
    SizedBox(height: 8),
    Row(
      children: [
        Icon(Icons.check_circle),
        SizedBox(width: 6),
        Text('3 of 5 done'),
      ],
    ),
  ],
)
```

Column ↓

Stacks children top-to-bottom (the main axis is vertical).

Row →

Places children left-to-right (the main axis is horizontal).

Stack ⊕

Layers children on top of each other (e.g. a badge on an avatar).

Showing Many Items — ListView & GridView

Turn a list of data into a scrollable list or grid of widgets.

list_view.dart

```dart
final tasks = ['Study', 'Code', 'Test'];

ListView.builder(
  itemCount: tasks.length,
  itemBuilder: (context, index) {
    return ListTile(
      leading: Icon(Icons.task_alt),
      title: Text(tasks[index]),
    );
  },
)
```

grid_view.dart

```dart
GridView.count(
  crossAxisCount: 2,
  children: [
    Card(child: Center(
      child: Text('Today'))),
    Card(child: Center(
      child: Text('Upcoming'))),
    Card(child: Center(
      child: Text('Done'))),
  ],
)
```

ListView.builder *is efficient — it builds only the rows currently on screen, so long lists stay fast.*

Input & Flexible Sizing — TextField, Expanded, Flexible

input_layout.dart

```dart
// 1) text input
TextField(
  decoration: InputDecoration(
    labelText: 'New task',
    border: OutlineInputBorder(),
  ),
  onSubmitted: (value) {
    print('Add: $value');
  },
),

// 2) share space in a Row
Row(children: [
  Expanded(child: TextField()),
  Icon(Icons.add),
])
```

TextField

- InputDecoration adds label + border
- onChanged / onSubmitted handle input
- Pair with a TextEditingController to read text

Expanded

- Fills the remaining space in a Row/Column
- Fixes the dreaded overflow error
- Several Expanded → share space by flex
- Great for a wide input beside a button

Flexible

- Like Expanded but may take less space
- Use when a child can shrink but needn't fill
- Expanded = Flexible(fit: tight)

Building Reusable Custom Widgets

Why. Extract repeated UI into your own widget class — write it once, reuse it everywhere, and keep build methods short and readable. This is composition: small widgets combined into bigger ones.

task_tile.dart

```
class TaskTile extends StatelessWidget {
  final Task task;
  final VoidCallback onToggle;

  const TaskTile({super.key,
    required this.task, required this.onToggle});

  @override
  Widget build(BuildContext context) {
    return ListTile(
      leading: Icon(task.done
        ? Icons.check_circle
        : Icons.circle_outlined),
      title: Text(task.title),
      onTap: onToggle,
    );
  }
}
```

How to use it

```
TaskTile(
  task: myTask,
  onToggle: () {},
)
```

Benefits

- No copy-paste — one source of truth
- Pass data in via constructor fields
- Pass actions in via callbacks
- Shorter, clearer parent widgets

Activity — Build the Widget Tree

Your task (8 minutes, in pairs). Sketch the widget tree for a single task row in TaskFlow: a card containing a checkbox icon on the left, the task title in the middle, and a delete icon on the right. Name every widget you would use.

Hints

- Start from a Card as the outer container
- A Row places the three parts side by side
- Use Icon for the checkbox and delete
- Wrap the title in Expanded so it fills space
- Add Padding / SizedBox for breathing room
- Which parts could change → stateful?

Deliverable

- A labelled tree diagram (boxes + lines)
- Parent → child relationships shown clearly
- Each node names a real Flutter widget
- One sentence: stateless or stateful, and why

We'll sketch one solution together on the board afterwards.

PART 5 OF 6

State, Interaction & Navigation



- Managing change with setState and handling events
- Forms, validation and the limits of setState
- Moving between screens with the Navigator
- Named routes and passing data between screens

State & Interaction — setState

State = data that can change while the app runs. Update it inside setState(), and Flutter rebuilds the widget to show the new value.

counter.dart

```dart
class Counter extends StatefulWidget {
  @override
  State<Counter> createState() => _CounterState();
}

class _CounterState extends State<Counter> {
  int count = 0;            // the state

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: () => setState(() => count++),
      child: Text('Done: $count'),
    );
  }
}
```

The update loop

1 User acts
taps the button (an event)

2 setState()
you change count inside it

3 Rebuild
build() runs again automatically

4 UI updates
new value appears on screen

Forms, Validation & the Limits of setState

form.dart

```dart
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: TextFormField(
    decoration: InputDecoration(
      labelText: 'Task title'),
    validator: (value) =>
      (value == null || value.isEmpty)
        ? 'Title is required'
        : null,
  ),
)

// on submit:
if (_formKey.currentState!.validate()) {
  // all fields valid → save
}
```

Form validation

- Wrap fields in a Form with a GlobalKey
- Each field gets a validator function
- Return an error string, or null if OK
- validate() checks them all at once

When setState isn’t enough

- Fine for state inside one widget
- Painful when many screens share data
- Then reach for state management:
- **Provider • Riverpod • BLoC**
- (Covered further in Lecture 10)

Navigation — Moving Between Screens

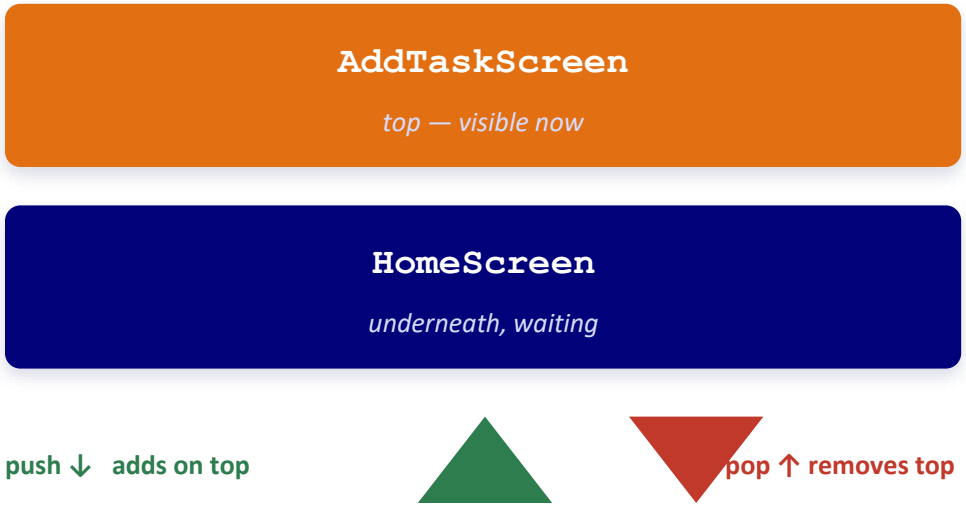
The navigator is a stack. push a new screen on top to go forward; pop the top screen off to go back — just like a stack of cards.

navigation.dart

```dart
// go to a new screen
ElevatedButton(
  onPressed: () {
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => AddTaskScreen(),
      ),
    );
  },
  child: Text('Add task'),
)

// go back (e.g. an AppBar back button)
Navigator.pop(context);
```

The navigation stack



Flutter draws the back arrow automatically when you push a new route.

Named Routes & Passing Data

Name your screens in one place, then navigate by name — and pass data both ways.

routes.dart

```dart
MaterialApp(
  initialRoute: '/',
  routes: {
    '/':    (context) => HomeScreen(),
    '/add': (context) => AddTaskScreen(),
  },
)

// navigate by name
Navigator.pushNamed(context, '/add');
```

pass_data.dart

```dart
// HOME: open add screen, wait
final newTask = await
  Navigator.pushNamed(context, '/add');
// use the returned value
print('Got: $newTask');

// ADD SCREEN: send a result back
Navigator.pop(context, 'Buy groceries');
```

Two directions: pushNamed sends you forward (and can carry arguments); pop(context, value) sends a result back to the screen that opened it.

Checkpoint 3 — Stateless or Stateful?



For each widget, decide: StatelessWidget or StatefulWidget? Be ready to justify in one line. (3 minutes.)

1	An app logo shown in the header	*hint: never changes*
2	A checkbox that toggles a task done / not done	*hint: changes on tap*
3	A label that displays a task's title text	*hint: just displays*
4	A login form that validates and shows errors	*hint: holds input*
5	A counter showing how many tasks are completed	*hint: value changes*
6	An icon button that simply navigates to another screen	*hint: no own data*

PART 6 OF 6

Mobile Architecture & Case Study



- Why architecture matters: separation of concerns
- Layered and feature-based architectures for mobile
- Case study: designing the TaskFlow app
- Building TaskFlow — model, UI, state and navigation

Why Architecture Matters

Separation of concerns. Keep UI, business logic and data in separate, well-named places. As an app grows, this is what keeps it understandable, testable and changeable.



Everything in one file

- UI, logic and data tangled together
- Hard to find or change anything
- Almost impossible to test in isolation
- Breaks easily — one change ripples everywhere
- Two developers constantly collide



Clear architecture

- Each layer has one clear responsibility
- Easy to locate and modify code
- Layers can be tested independently
- Changes stay local — fewer surprises
- Teams work in parallel without clashing

Layered Mobile Architecture

A common, beginner-friendly structure — each layer talks only to the one below it.

UI Layer (Widgets / Screens)

What the user sees and taps — screens, widgets, layout.

HomeScreen, TaskTile

Service / Logic Layer

App rules and coordination — the 'what should happen'.

TaskService: add, complete, filter

Repository / Data Layer

Gets and saves data — hides where it comes from.

TaskRepository: local DB / API

Model Layer (Data classes)

The shape of your data — plain Dart objects.

class Task { title; done; }

Data flows down to fetch, and back up to display. *The UI never talks to the database directly.*

Feature-Based Structure & Knowing When to Scale

Organise by feature, not just by type

lib/features/

```
lib/
  features/
    tasks/
      task.dart         model
      task_service.dart logic
      home_screen.dart  UI
      task_tile.dart    UI
    auth/
      login_screen.dart
  shared/
    widgets/  utils/
```

Each feature is self-contained — easy to find, easy to grow.

Match structure to project size

Student / small app

- A few folders: models, screens, widgets
- setState for state — keep it simple
- Goal: clear and working, not perfect

Production / large app

- Feature folders + clear layer separation
- Dedicated state management (Provider/BLoC)
- Testing, CI/CD, error handling, logging
- Multiple developers, long lifespan

Case Study — Designing TaskFlow

TaskFlow — a simple task manager. You can see your tasks, add a new one, and tick tasks as done. Small enough to build today, but it exercises lists, forms, state and navigation together.

What it does

View tasks
A scrolling list of all tasks.

Add a task
A form to enter a new task.

Complete
Tap to mark a task done.

Screen flow



push → add → pop with the new task

Building TaskFlow (1) — Model & Task List

Step 1: the data model and a home screen that lists tasks.

task.dart + models

```dart
class Task {
  final String title;
  bool done;
  Task(this.title, {this.done = false});
}

// seed data
final tasks = <Task>[
  Task('Read chapter 9'),
  Task('Build TaskFlow'),
  Task('Submit lab', done: true),
];
```

home_screen.dart

```dart
Scaffold(
  appBar: AppBar(title: Text('TaskFlow')),
  body: ListView.builder(
    itemCount: tasks.length,
    itemBuilder: (context, i) {
      final t = tasks[i];
      return ListTile(
        leading: Icon(t.done
          ? Icons.check_circle
          : Icons.circle_outlined),
        title: Text(t.title),
      );
    },
  ),
)
```

Building TaskFlow (2) — Add Task & State

Step 2: a form to add tasks, with validation, updating the list via setState.

add_task_screen.dart

```dart
final _controller = TextEditingController();
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: TextFormField(
    controller: _controller,
    decoration: InputDecoration(
      labelText: 'Task title'),
    validator: (v) => (v == null || v.isEmpty)
      ? 'Please enter a title' : null,
  ),
)
```

on save (stateful)

```dart
void _save() {
  if (_formKey.currentState!.validate()) {
    setState(() {
      tasks.add(Task(_controller.text));
    });
    Navigator.pop(context);
  }
}

// completing a task, back on Home:
onTap: () => setState(
  () => t.done = !t.done),
```

Building TaskFlow (3) — Navigation & Running It

Step 3: wire the screens together, then run the same code on Android and iOS.

wiring it up

```
// Home: a button to open the add screen
floatingActionButton: FloatingActionButton(
  child: Icon(Icons.add),
  onPressed: () {
    Navigator.push(context,
      MaterialPageRoute(builder: (_) =>
        AddTaskScreen()));
  },
),

// Add screen: save button calls _save()
ElevatedButton(onPressed: _save,
  child: Text('Save task')),
```

TaskFlow now lists, adds and completes tasks across both platforms.

One codebase, both platforms



Android

flutter run on an emulator or device — Material look by default.



iOS

Same Dart code on the simulator (Mac) — no rewrite needed.

That's the win. One codebase → two native apps.

Activity — Design Your App's Screens & Folders

Your task (10 minutes, in pairs). Pick a simple app idea (e.g. a notes app, expense tracker, or habit tracker). Design its screens, its data model, and a sensible folder structure — applying today's architecture ideas.

Deliver three things

- **1. Screen list**
- Name each screen and how you move between them
- **2. Data model**
- One or two Dart classes (fields only)
- **3. Folder structure**
- models / screens / widgets / services

Checklist

- Which screens are stateful vs stateless?
- What does each Navigator route lead to?
- Where does data live (which layer)?
- Is the UI kept separate from the data?
- Could a teammate find things easily?
- Is it as simple as it can be?

Sketch it on paper — a few volunteers will share at the start of next session.

Common Beginner Mistakes in Flutter

Recognise these early — they trip up almost everyone at first.

Changing state outside setState()

Fix: Wrap every state change in setState() so the UI rebuilds.

Overflow errors in Row / Column

Fix: Wrap flexible children in Expanded or Flexible.

Using Stateful when Stateless will do

Fix: Default to Stateless; go Stateful only when data changes.

Everything piled into main.dart

Fix: Split into models, screens, widgets and services.

Forgetting to handle null

Fix: Use ?, ?? and ! deliberately — lean on null safety.

Building all list rows at once

Fix: Use ListView.builder for long or dynamic lists.

ASSESSMENT

Quick Quiz — Questions 1–5

Choose the best answer. We'll review them together — answers are on the next part of the session.

1. Which approach uses ONE codebase to target both Android and iOS?

(A) Native (B) Cross-platform (C) Two native apps (D) Assembly

2. Which programming language does Flutter use?

(A) JavaScript (B) Kotlin (C) Dart (D) Swift

3. In Flutter, "everything is a …"

(A) function (B) widget (C) file (D) service

4. Which call triggers a UI rebuild in a StatefulWidget?

(A) build() (B) rebuild() (C) setState() (D) refresh()

5. Which widget arranges its children horizontally?

(A) Column (B) Row (C) Stack (D) ListView

ASSESSMENT

Quick Quiz — Questions 6–10

Last five — same rules. Jot your answers; we'll go through the full key together.

6. A StatelessWidget is the best choice for …

(A) a checkbox (B) a validated form (C) a static label (D) a live counter

7. Which call navigates FORWARD to a new screen?

(A) Navigator.pop (B) Navigator.push (C) setState (D) runApp

8. Where does a Flutter app start executing?

(A) pubspec.yaml (B) android/ (C) lib/main.dart (D) test/

9. In layered architecture, the UI layer should …

(A) talk to the DB directly (B) go through a service/repository (C) hold all logic (D) be skipped

10. While applying code changes, hot reload keeps …

(A) nothing (B) the current app state (C) only the AppBar (D) the file names

Summary — What We Covered Today

Part 1 • Mobile foundations

Native, cross-platform, hybrid and PWA — and how teams choose.

Part 2 • Cross-platform & Flutter

One codebase, Flutter's architecture, advantages and limits.

Part 3 • The Dart language

Variables, control flow, classes, null safety and async.

Part 4 • Building UIs with widgets

Everything is a widget; layouts, lists and custom widgets.

Part 5 • State & navigation

setState, forms, and moving between screens.

Part 6 • Architecture & TaskFlow

Layered design, and building the case-study app end to end.

From "why mobile?" to a working cross-platform app — you now have the full beginner toolkit.

Learning Outcomes, Lab & What's Next

How today maps to the module outcomes

LO	Outcome — covered today
LO1	Evaluate mobile frameworks: compared native / cross-platform / hybrid / PWA.
LO2	Apply a framework: built TaskFlow with Flutter UI, state and navigation.
LO4	Select & justify: chose approaches against requirements, team and timeline.

LO3 (integration, CI/CD, quality, deployment) is developed in later lectures and the project.

In the lab

- Install Flutter and run flutter doctor
- Build and run TaskFlow yourself
- Extend it: add delete, or a task count

Next — Lecture 10

- Richer Flutter UI and device features
- Working with data and local storage
- State management beyond setState
- Prep: revise widgets, state & navigation

References & Further Reading

Use these to go deeper and to support your lab work and project.

Core resources

- Rose, R. (2022) Flutter and Dart Cookbook. Sebastopol, CA: O'Reilly Media.
- Flutter official documentation — flutter.dev/docs (widgets, layout, navigation, cookbook).
- Dart language tour — dart.dev/guides (syntax, null safety, async).

Tools & packages

- pub.dev — the Dart & Flutter package registry
- Flutter widget catalog — docs.flutter.dev/ui/widgets
- DartPad — dartpad.dev (run Dart in the browser)
- Flutter DevTools — debugging & performance

Module & supplementary

- SE3090 module outline — LMS (outcomes & schedule)
- Lecture slides & lab sheets — course LMS
- Banks, A. & Porcello, E. (2020) Learning React. O'Reilly.
- Google Codelabs — 'Write your first Flutter app'

Always prefer the official flutter.dev and dart.dev docs — they are current, complete and free.

SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 10

Flutter UI Handling, Device Features & Data Management



Year 3 • Semester 1 • 4 Credits • Duration: 3 Hours

Learning Outcomes: LO1, LO2, LO4 | Mobile App Development Frameworks

Where This Lecture Fits — Learning Outcomes

From a working app to a production app. Lecture 09 covered the building blocks. Today we add the layers a real mobile product needs: adaptive UI, robust navigation, managed state, validated input, device integration, storage, networking, tests and release.



LO1

Evaluate

Compare UI, state, storage and routing options and pick what fits the problem.



LO2

Apply & build

Build advanced Flutter features into one coherent application.



LO3

Integrate

Connect the app to REST APIs, handle data, test and prepare for release.



LO4

Select & justify

Justify packages, patterns and trade-offs for production readiness.

What You Already Know — A Quick Recap

From Lecture 09 we assume you are comfortable with:

Dart basics
Variables, null safety, functions, classes, async/await

Project structure
lib/, main.dart, pubspec.yaml, organising by folder

Widgets
Stateless vs Stateful, the build method, the widget tree

Layout
Row, Column, Stack, Container, Padding, ListView

Simple navigation
Navigator.push / pop between two screens

setState()
Local state updates that rebuild a StatefulWidget

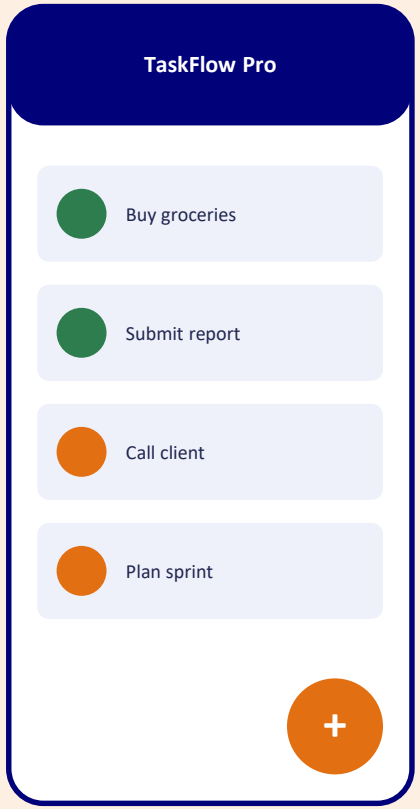
Today we do NOT re-teach these. We build directly on top of them. If any of the above feels shaky, revisit the Lecture 09 slides and the lab before the next practical — today moves quickly into advanced territory.

Why It Matters — Our Case Study: TaskFlow Pro

TaskFlow Pro

We take last week's simple task list and grow it into a realistic, production-shaped app — one feature at a time, mapped to today's topics.

- Responsive home screen (phone & tablet)
- Multi-screen navigation with tabs & a drawer
- Shared state with Provider
- A validated 'add task' form
- Attach a photo & tag a location
- Local storage + secure token storage
- Sync with a REST API + basic auth
- Unit & widget tests, then release prep



Today’s Roadmap — 3 Hours

Hour 1

Advanced UI & Navigation

Responsive/adaptive layouts • MediaQuery & LayoutBuilder • themes & UI states • accessibility & performance • navigation stack, named & declarative routing, guards • Activity + Checkpoint

Hour 2

State, Forms, Device & Data

State types & Provider/Riverpod • state-management comparison • forms & validation • plugins & permissions • camera, location, connectivity • JSON, SharedPreferences, secure storage • Activities

Hour 3

APIs, Testing & Deployment

REST integration, service layer, auth & pagination • loading/error handling • testing pyramid (unit/widget/integration) • release modes, signing & store prep • monitoring • Quiz

Two short breaks • three hands-on activities • a checkpoint and a 10-question quiz — ask questions anytime.

PART 1 OF 6

Advanced Flutter UI Handling

- Responsive & adaptive layouts for every screen size
- MediaQuery, LayoutBuilder, SafeArea and the layout toolkit
- Themes, light/dark mode and consistent styling
- Loading / empty / success / error UI states, accessibility & performance



Responsive vs Adaptive Design

Responsive

Same design fluidly resizes

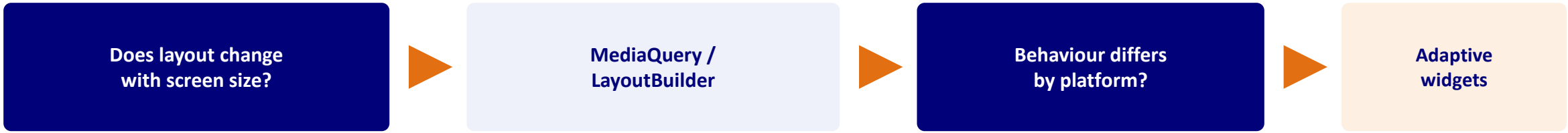
- One layout that stretches, wraps and reflows
- Driven by available width / height
- Phone → tablet → desktop from one widget tree

Adaptive

Different design per platform/device

- Distinct widgets/behaviour per platform
- e.g. Material on Android, Cupertino on iOS
- Use .adaptive constructors & platform checks

A simple decision flow



Most apps need both: responsive for size, a touch of adaptive for platform feel.

MediaQuery & LayoutBuilder

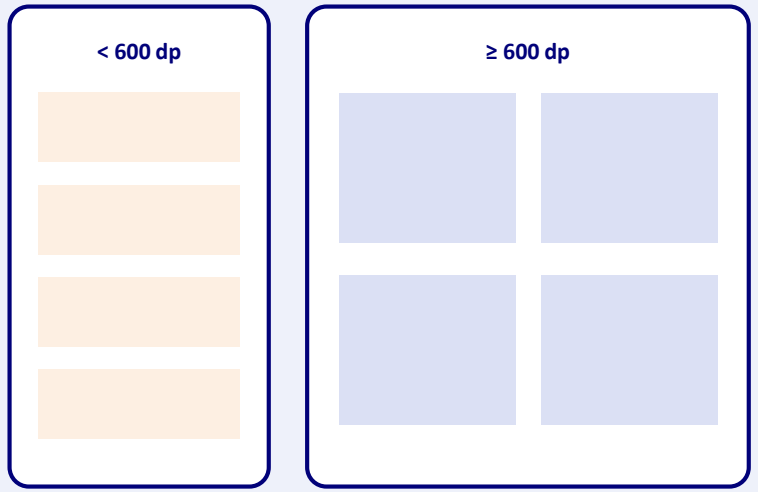
Two tools to read available space and react to it. Prefer LayoutBuilder for a widget's own box; MediaQuery for whole-screen facts.

responsive_list.dart

```dart
@override
Widget build(BuildContext context) {
  // whole-screen info: size, orientation, text scale
  final width = MediaQuery.of(context).size.width;

  return LayoutBuilder(
    builder: (context, constraints) {
      // constraints = this widget's own space
      if (constraints.maxWidth < 600) {
        return TaskListView();        // phone: 1 column
      }
      return TaskGridView(columns: 2); // tablet: 2 columns
    },
  );
}
```

What it produces



600 dp is the common phone/tablet breakpoint.

The Responsive Layout Toolkit

SafeArea

Insets content past notches, status bars and rounded corners.

Expanded / Flexible

Share remaining space inside a Row or Column proportionally.

Wrap

Flows children to the next line when they run out of width — great for chips/tags.

Stack

Layers widgets on top of each other; position with Positioned.

ListView / GridView

Scrollable lists and grids; .builder for long or infinite data.

CustomScrollView + slivers

Collapsing app bars and mixed scrolling sections in one scroll view.

Reach for these before writing manual pixel maths — they keep layouts flexible across devices.

ThemeData, Light & Dark Mode

Define colours, typography and shapes once; every widget inherits them via Theme.of(context). Ship light and dark from the same seed.

theme.dart

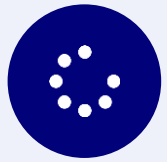
```
MaterialApp(
  themeMode: ThemeMode.system,    // follow device
  theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(
      seedColor: Color(0xFF010179)),
    useMaterial3: true,
  ),
  darkTheme: ThemeData(
    colorScheme: ColorScheme.fromSeed(
      seedColor: Color(0xFF010179),
      brightness: Brightness.dark),
  ),
  home: HomeScreen(),
);
// use it:  Theme.of(context).colorScheme.primary
```

Why theme centrally

- **One source of truth for colour & type**
- Change the brand in one place
- **Free dark mode from the same seed**
- Consistent spacing & components
- **No hard-coded Colors.blue everywhere**
- Respects the user's system setting

Designing for Every UI State

Rule. Every screen that loads data has at least four states. Design all of them — a blank screen during loading or a crash on error is a real, common bug.



Loading

Spinner or skeleton while data is fetched.



Success

The data, rendered — the happy path.



Empty

No items yet — guide the user to act.



Error

Friendly message + a retry button.



FutureBuilder / state notifiers make these transitions explicit — we wire them up in the API section.

Accessibility & UI Performance

Accessibility

- Add Semantics labels for screen readers
- Respect text-scale: avoid fixed heights for text
- Tap targets ≥ 48x48 dp
- Sufficient colour contrast (don't rely on colour alone)
- Test with TalkBack / VoiceOver

Performance

- Use const constructors wherever possible
- ListView.builder / GridView.builder for long lists
- Keep setState scope small — rebuild only what changes
- Avoid deeply nested widget trees
- Use RepaintBoundary for expensive, isolated paints

Biggest beginner win: replace a Column-of-everything inside a scroll view with ListView.builder, and mark static widgets const. Both cut needless rebuilds and jank.

Activity 1 — Make This Screen Responsive

Your task (8 minutes, pairs). You are given a TaskFlow detail screen built with fixed widths (width: 350) and a non-scrolling Column. It overflows on small phones and looks lost on tablets. Redesign it to be responsive.

Apply what you've learned

- Remove fixed widths — use Expanded / Flexible
- Wrap the body in SafeArea + a scroll view
- Use LayoutBuilder to switch 1 vs 2 columns at 600 dp
- Replace literal colours with Theme.of(context)
- Make a row of tags use Wrap instead of Row

Deliverable & expected outcome

- A widget that fits a small phone with no overflow
- A sensible two-column layout on a tablet
- No hard-coded sizes or colours remaining
- One sentence: which tool fixed which problem

We'll compare two solutions on the projector. Time: ~8 minutes.

PART 2 OF 6

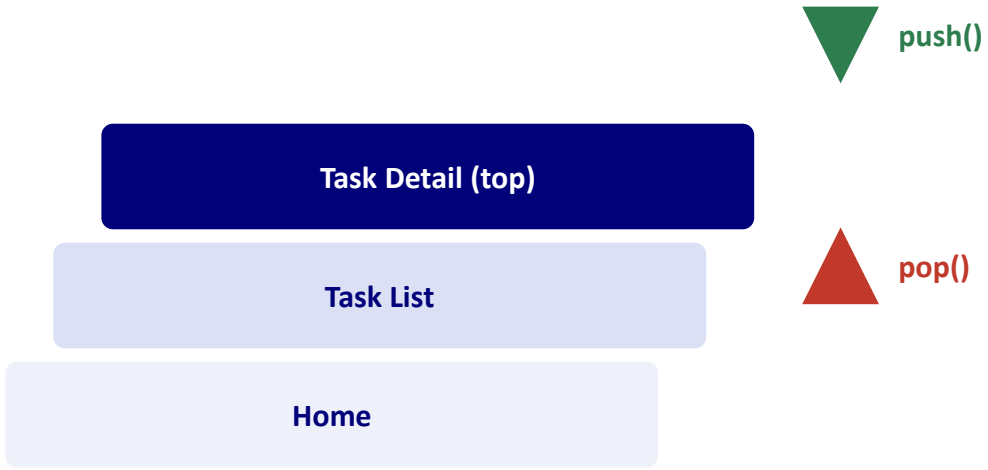
Navigation & Routing



- The navigation stack: push, pop and passing data
- Named routes vs declarative routing with GoRouter
- Bottom navigation, drawers and tabs
- Route guards, auth-aware navigation and deep linking

The Navigation Stack

Mental model. Screens are a stack of cards. push() places a new screen on top; pop() removes the top one and reveals what's beneath — exactly like the back button.



navigation.dart

```dart
// go to a new screen (push on top)
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => TaskDetailScreen(task)),
);

// go back (pop the top screen)
Navigator.pop(context);
```

Imperative navigation is fine for simple flows; we'll move to declarative routing for larger apps.

Named Routes & Passing Data

Register routes by name once, then navigate by string. Pass arguments in, and await a result coming back.

routes.dart

```dart
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (_) => HomeScreen(),
    '/add': (_) => AddTaskScreen(),
  },
);

// navigate by name + pass an argument
Navigator.pushNamed(
  context, '/add', arguments: 'Work');
```

return_value.dart

```dart
// read the argument on the new screen
final tag = ModalRoute.of(context)!
    .settings.arguments as String;

// return a result when popping
Navigator.pop(context, newTask);

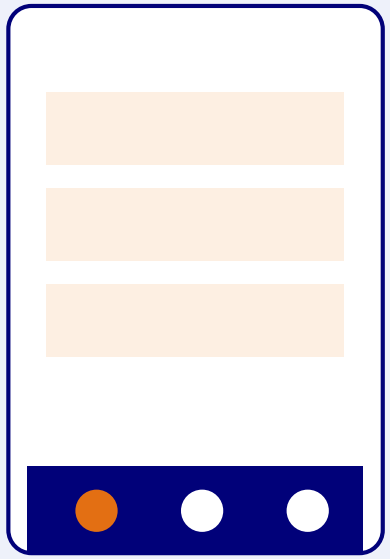
// await it back on the caller
final task = await Navigator.pushNamed(
    context, '/add') as Task?;
if (task != null) controller.add(task);
```

Two directions: pushNamed sends data forward via arguments; pop(context, value) returns a result to the screen that opened it.

Bottom Nav, Drawers & Tabs

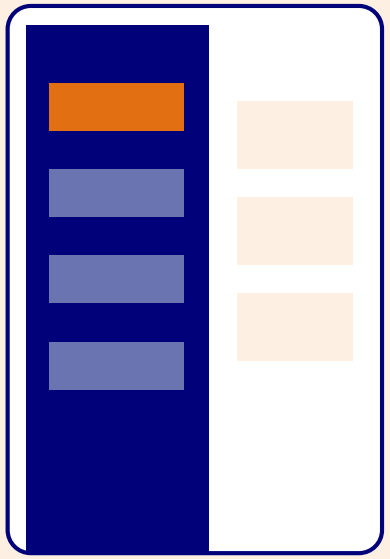
BottomNavigationBar

3–5 top-level destinations



Drawer

Many secondary destinations



TabBar

Switch peer views in a section



Combine them: a bottom bar for top-level sections, tabs inside a section, a drawer for the long tail (settings, help, logout).

Route Guards & Auth-Aware Navigation

Some screens must be protected. A guard checks a condition (logged in?) before a route renders and redirects if it fails.



After login, redirect back to the originally requested screen.

guard.dart (GoRouter redirect)

```
GoRouter(
  redirect: (context, state) {
    final loggedIn = context.read<AuthService>().isLoggedIn;
    final goingToLogin = state.matchedLocation == '/login';
    if (!loggedIn && !goingToLogin) return '/login';    // block + send to login
    if (loggedIn && goingToLogin)  return '/';          // already in → go home
    return null;                                        // no redirect needed
  },
  routes: [ /* ... */ ],
);
```

Declarative Routing with GoRouter

Declarative routing maps URLs to screens. It scales better than named routes and supports deep links, web URLs and typed parameters.

router.dart

```dart
final router = GoRouter(
  routes: [
    GoRoute(path: '/', builder: (_, __) => Home()),
    GoRoute(
      path: '/task/:id',                  // path param
      builder: (context, state) {
        final id = state.pathParameters['id']!;
        return TaskDetail(id: id);
      },
    ),
  ],
);

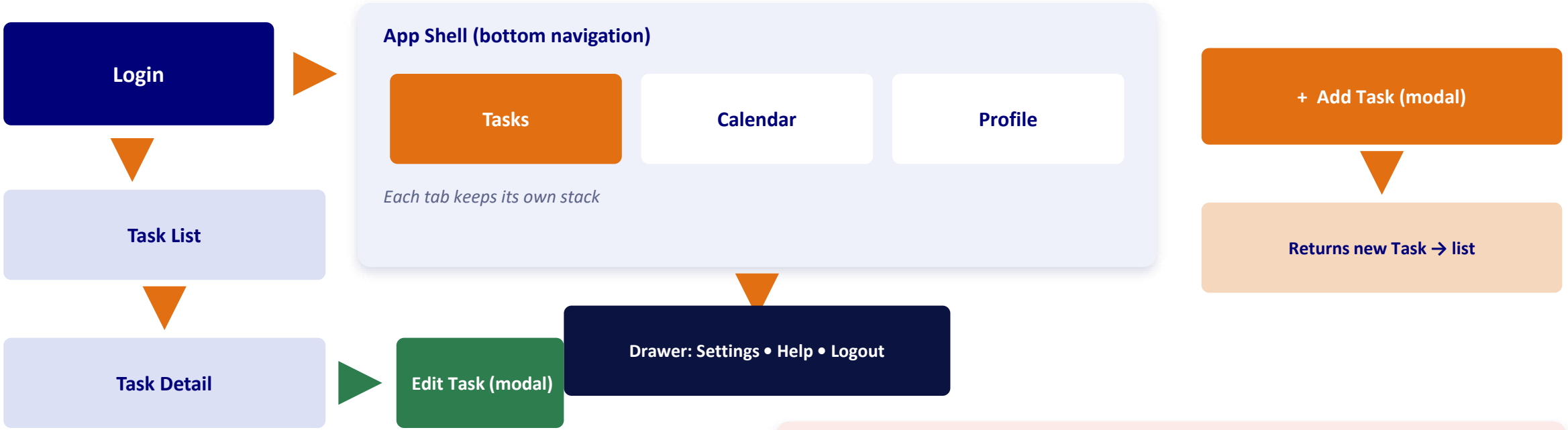
// navigate declaratively
context.go('/task/42');
```

Why GoRouter

- **URL-based & typed routes**
- Path params: /task/:id
- **Built-in redirect for guards**
- Deep links open the right screen
- **Same routes for mobile & web**
- Cleaner than deep push/pop chains

Deep linking: tapping taskflow://task/42 (or a web URL) opens Task 42 directly — the router resolves the path.

TaskFlow Pro — Navigation Flow



Pattern: bottom bar for top-level areas, modals for create/edit, a drawer for the long tail, GoRouter + a redirect guard tying it together.

Guarded: every screen except Login requires a valid token. An expired token redirects to Login, then returns the user to where they were. The + button opens a modal that returns the created task to the list.

PART 3 OF 6

State Management & Forms

- What state is — ephemeral (local) vs application (shared) state
- Why excessive setState() hurts, and how to lift state up
- Provider + ChangeNotifier in practice, with a Riverpod look
- Building robust, validated forms for task and expense entry



What Is Application State?

Definition. State is any data that can change while the app runs and that the UI depends on. When state changes, the affected UI must rebuild to reflect it.

Ephemeral (local) state

Lives in one widget • setState

- A checkbox toggle or switch value
- The currently selected tab index
- Animation progress, a text field's focus
- Safe to lose when the widget is disposed

Application (shared) state

Shared across screens • a state manager

- The logged-in user and auth token
- The task list shown on several screens
- App settings (theme, language)
- Must persist for the whole session

The deciding question: “Does any OTHER widget need this data?” No → ephemeral (setState). Yes → application state (Provider / Riverpod).

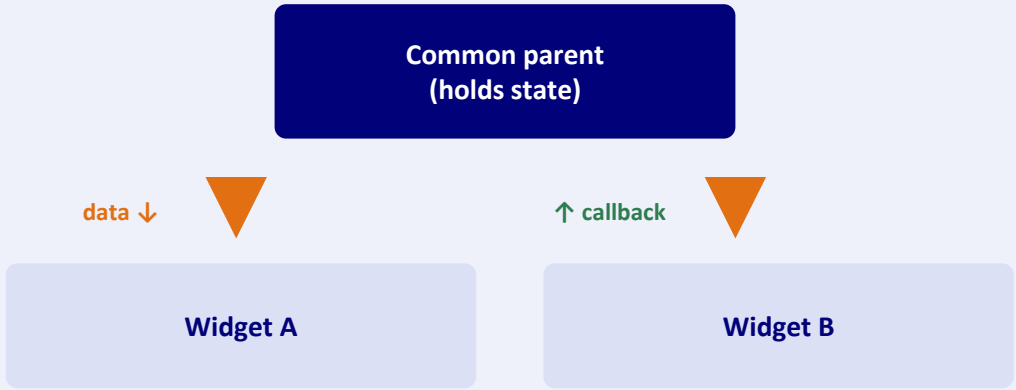
When setState() Isn't Enough

Problems with excessive setState()

- **Rebuilds the whole widget subtree, not just what changed**
- State is trapped inside one widget — siblings can't see it
- **“Prop drilling”: passing data and callbacks down many levels**
- Business logic gets tangled into the UI
- **Hard to test logic without building widgets**
- Two widgets showing the same data can fall out of sync

A first solution is **lifting state up** to a common parent.

Lifting state up



Both children now read and update the same state through the parent — they stay in sync.

Better still: lift it all the way to a state manager so any screen — not just nearby widgets — can share it.

Provider + ChangeNotifier Architecture

Provider is the recommended baseline: a **ChangeNotifier** holds the state, and widgets that depend on it rebuild only when it calls notifyListeners().

The layers

ChangeNotifierProvider
creates & exposes the model

▼

TaskModel extends ChangeNotifier
state + notifyListeners()

▼

Consumer
(task list)

Consumer
(badge count)

Listeners subscribe; each rebuilds when notified.

The update cycle

1. User taps “Add Task”

▼

2. model.add(task)

▼

3. notifyListeners()

▼

4. Consumers rebuild → UI updates

Provider in Practice

Three steps: make a ChangeNotifier model, provide it above the tree, then read it in the UI with Consumer.

task_model.dart

```dart
// 1. the model holds state + notifies
class TaskModel extends ChangeNotifier {
  final List<Task> _tasks = [];
  List<Task> get tasks => List.unmodifiable(_tasks);

  void add(Task t) {
    _tasks.add(t);
    notifyListeners();   // rebuild listeners
  }
}

// 2. provide it once, above the widgets
ChangeNotifierProvider(
  create: (_) => TaskModel(), child: const MyApp());

// 3. read it in the UI
Consumer<TaskModel>(
  builder: (context, model, _) =>
    Text('${model.tasks.length} tasks'));
```

What to notice

There are **three main steps**.

Step 1: Create a model that extends ***ChangeNotifier***.
The model stores the shared state, such as the task list.
Keep the list private using **_tasks**.

Update the state through methods such as add().

After changing the state, call **notifyListeners**().

Step 2: Use ***ChangeNotifierProvider*** to provide the model to the app.

Step 3: Use **Consumer<TaskModel>** to listen to the model and update the UI.

- **context.watch() listens for changes.**
- **context.read() accesses the model without listening.**

Riverpod — A Modern Alternative

Same mental model as Provider, with compile-time safety and easier testing. Providers are declared as top-level objects.

providers.dart

```dart
// 1. declare a provider (global, testable)
final taskProvider =
  ChangeNotifierProvider((ref) => TaskModel());

// 2. wrap the app once
ProviderScope(child: const MyApp());

// 3. read it in a ConsumerWidget
class TaskCount extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final model = ref.watch(taskProvider);
    return Text('${model.tasks.length} tasks');
  }
}
```

Why teams pick it

- **Compile-safe: no “provider not found” at runtime**
- Read state without a BuildContext
- **Trivial to override providers in tests**
- Works outside the widget tree (services, logic)
- **Scales cleanly as the app grows**

For SE3090 we use Provider as the baseline; Riverpod is the natural next step — the concepts transfer directly.

State Management at a Glance

Approach	Style	Learning curve	Boilerplate	Best for
Provider	InheritedWidget-based	Low	Low–Medium	Most apps — our baseline
Riverpod	Provider, compile-safe	Low–Medium	Low	Testable, scalable apps
BLoC / Cubit	Events & streams	Medium–High	Higher	Large, complex, event-driven
GetX	All-in-one toolkit	Low	Very low	Rapid prototyping (use with care)

There is no single best. All four solve the same problem — separate state from UI. Pick by app complexity, team familiarity and testing needs. We standardise on Provider.

Choosing & Using State Well

Decision factors

- **App size & complexity**
- How much state is truly shared
- **Team familiarity with the approach**
- Testability requirements
- **Async complexity (streams, queues)**
- Tolerance for boilerplate

Best practices

- **Keep UI and business logic separate**
- Expose read-only views; mutate only via methods
- **Rebuild the smallest widget possible (Consumer / Selector)**
- Avoid one giant “god” model — split by concern
- **Prefer immutable state — build a new object, don't mutate**

Immutable state. Instead of editing an object in place, create a copy with the change (copyWith). Changes become explicit and traceable, and rebuilds stay predictable.

Activity 2 — Pick the Right State Approach

Your task (6 minutes, individually). For each scenario decide setState, Provider, or a heavier solution — and justify your choice in one short line.

Scenarios

1. A single checkbox's on/off inside one widget
2. The logged-in user, needed on five screens
3. A three-tab screen remembering its selected tab
4. A cart shared app-wide that recomputes totals
5. A large app with complex async flows and a big team

Expected answers

- **1 → setState (ephemeral)**
- **2 → Provider (shared)**
- **3 → setState (local UI)**
- **4 → Provider / Riverpod**
- **5 → BLoC or Riverpod**

Full reasoning in the speaker notes. Time: ~6 minutes + 3 to discuss.

Form Architecture: Form, Keys & Controllers

A Form groups fields under a GlobalKey<FormState>. TextFormField carries its own validator; controllers read and set the text.

task_form.dart

```dart
final _formKey = GlobalKey<FormState>();
final _title = TextEditingController();

Form(
  key: _formKey,
  child: Column(children: [
    TextFormField(
      controller: _title,
      decoration: const InputDecoration(
        labelText: 'Task title'),
      textInputAction: TextInputAction.next,
      validator: (v) =>
        (v == null || v.isEmpty) ? 'Required' : null,
    ),
    ElevatedButton(
      onPressed: () {
        if (_formKey.currentState!.validate()) {
          // all validators passed → save the task
        }
      },
      child: const Text('Save')),
  ]));
```

The four pieces

- **Form + GlobalKey<FormState>**

  Handle to validate(), save() and reset() every field at once

- **TextFormField**

  Is used for user input and supports a built-in validator.

- **TextEditingController**

  lets us read, set, or clear the text entered by the user.

- **textInputAction / FocusNode**

  help move the user smoothly from one field to the next.

When the user clicks **Save**, call validate().

If all validators pass, then we can continue and save the task.

Validation: Required, Email & Cross-Field

Keep validators as small, reusable, pure functions — out of the widget. Each returns null when valid, or an error message string.

validators.dart

```dart
// each returns null = valid, else an error
String? required(String? v) =>
  (v == null || v.trim().isEmpty) ? 'Required' : null;

String? email(String? v) =>
  (v != null && v.contains('@')) ? null : 'Invalid email';

String? minLen(String? v, int n) =>
  (v != null && v.length >= n) ? null : 'Min $n chars';

// cross-field: confirm must equal password
String? confirm(String? v, String pwd) =>
  v == pwd ? null : 'Passwords do not match';

// use it in the field
TextFormField(validator: (v) => email(v));
```

Common rules

- Required / not-empty
- Email & phone format
- Password strength & length
- Numeric range, date in range
- Cross-field (confirm = password)

Client-side ≠ secure. Validation here is for fast UX. Always re-validate on the server — a client can be bypassed.

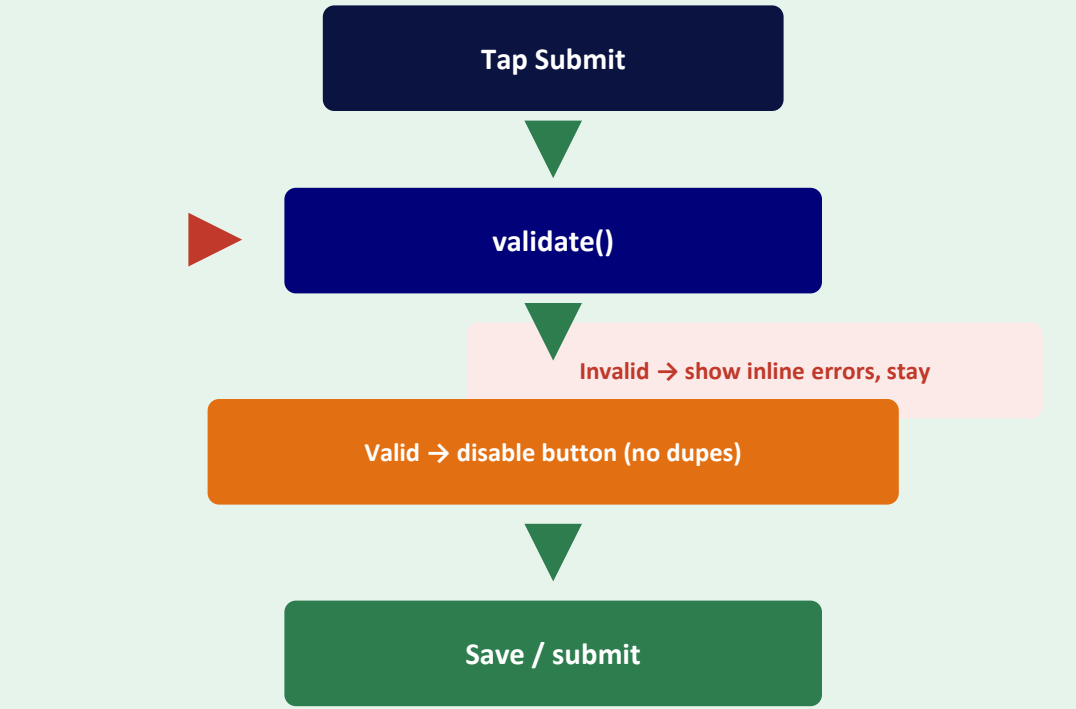
Inputs, Pickers & the Submission Flow

Beyond text fields

- Dropdown
- Switch
- Checkbox
- Radio
- Date picker
- Time picker

Dynamic fields: add or remove rows at runtime (e.g. multiple expense items).

Submission workflow



PART 4 OF 6

Device Features & Data

- The plugin ecosystem and how to evaluate a package
- Permissions: the lifecycle, Android vs iOS, and handling denial
- Camera, image picker, location and connectivity in practice
- Data models, JSON, SharedPreferences and secure storage



The Flutter Plugin Ecosystem

pub.dev hosts thousands of packages. Plugins bridge Dart to native Android/iOS code so you can use real device hardware.

What plugins unlock

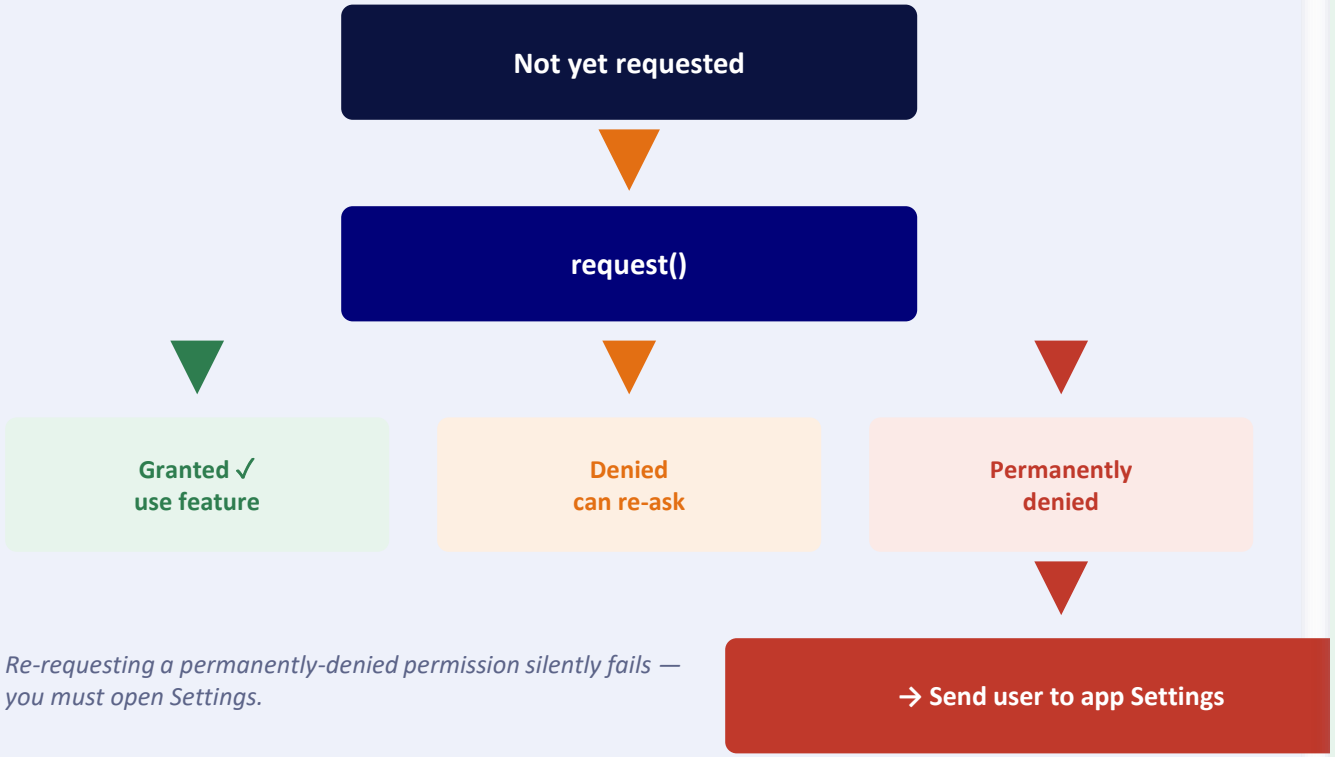
- **Camera**
- **Location / GPS**
- **Local storage**
- **Connectivity**
- **Notifications**
- **Sensors**

Before adding a package, check

- **Likes, popularity & pub points**
- Recently maintained — last publish date
- **Supports both Android and iOS (and web if needed)**
- Sound null-safety, current Flutter SDK
- **Clear docs and example code**
- Sensible permissions — not over-reaching
- **Healthy issue tracker, not abandoned**

Permission Lifecycle & Platform Differences

The permission lifecycle



Re-requesting a permanently-denied permission silently fails — you must open Settings.

Android vs iOS

- **Declare up front**
  - Android: AndroidManifest.xml
  - iOS: Info.plist + a usage-description string
- **Prompt behaviour daiffers**
  - Android can re-prompt after a denial
  - iOS prompts once; after that → Settings
- **Always handle denial gracefully**
  - Explain why, then degrade — never crash

Camera & Image Picker

The **image_picker** plugin captures a photo or picks one from the gallery. It returns an **XFile**, or **null if the user cancels.**

receipt_picker.dart

```dart
final picker = ImagePicker();

Future<void> attachReceipt() async {
  final XFile? photo = await picker.pickImage(
    source: ImageSource.camera,    // or .gallery
    maxWidth: 1200,                // downscale to save space
  );

  if (photo == null) return;       // user cancelled
  setState(() => _receiptPath = photo.path);
}
```

Get it right

- **Request camera permission first**
- Always handle null (cancel)
- **Downscale large images (maxWidth)**
- Show a preview after capture
- **file_picker covers documents too**



Preview the captured receipt before saving it to the task.

Location, GPS & Connectivity

Check the service is on and permission is granted before reading location. Watch connectivity to drive offline behaviour.

location.dart

```dart
Future<Position> getLocation() async {
  final on = await Geolocator
      .isLocationServiceEnabled();
  if (!on) throw Exception('GPS off');

  var perm = await Geolocator.checkPermission();
  if (perm == LocationPermission.denied) {
    perm = await Geolocator.requestPermission();
  }
  return Geolocator.getCurrentPosition();
}
```

connectivity.dart

```dart
// one-off check
final r = await Connectivity()
    .checkConnectivity();
final online = r != ConnectivityResult.none;

// react to changes
Connectivity().onConnectivityChanged
  .listen((r) {
    setState(() =>
      _online = r != ConnectivityResult.none);
  });
```

Same pattern everywhere: maps (google_maps_flutter), sensors (sensors_plus), notifications (flutter_local_notifications) — check permission, then call, then handle failure.

Privacy, Responsibility & Error Handling



Responsible data collection

- **Request only what the feature needs**
- Explain why before you prompt (rationale)
- **Prefer coarse over precise data where possible**
- Don't keep data longer than necessary
- **Let users decline and still use the app**



Device feature error handling

- **Permission denied → explain + offer Settings**
- Hardware missing (no camera) → fallback path
- **Service off (GPS) → prompt to enable**
- Wrap every plugin call in try / catch
- **Never crash on a denied permission**

Privacy is a requirement, not a nicety. Stores demand a privacy policy and justified permissions; regulations expect data minimisation and user consent.

Data Models & JSON Serialization

APIs and storage speak **JSON**; your app speaks **dart objects**. fromJson / toJson convert between a Map and a typed Dart model.

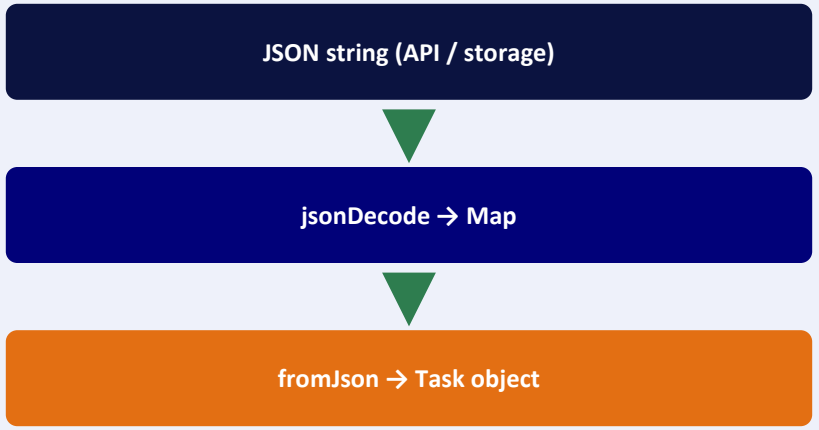
task.dart

```dart
class Task {
  final String id;
  final String title;
  final bool done;
  Task({required this.id, required this.title,
        this.done = false});

  factory Task.fromJson(Map<String, dynamic> j) =>
    Task(
      id: j['id'] as String,
      title: j['title'] as String,
      done: j['done'] as bool? ?? false,
    );

  Map<String, dynamic> toJson() =>
    {'id': id, 'title': title, 'done': done};
}
```

JSON ↔ object flow



toJson + jsonEncode reverse it on the way out.

At scale: generate fromJson/toJson with json_serializable + build_runner instead of hand-writing them — fewer typos on big models.

Local Storage: Shared Preferences & Secure Storage

SharedPreferences stores simple settings in plain text. **Secure storage** encrypts sensitive values in the OS keystore.

prefs.dart

```dart
final prefs = await
    SharedPreferences.getInstance();

// save a simple setting
await prefs.setBool('darkMode', true);

// read it back, with a default
final dark = prefs.getBool('darkMode')
    ?? false;
```

secure_storage.dart

```dart
const storage = FlutterSecureStorage();

// store an auth token (encrypted)
await storage.write(key: 'jwt',
    value: token);

// read it / clear on logout
final jwt = await storage.read(key: 'jwt');
await storage.delete(key: 'jwt');
```

Security rule. Never store tokens, passwords or keys in ***SharedPreferences*** — it's plain text. Secrets go in **secure storage** (Keychain on iOS, Keystore on Android).

Choosing the Right Storage

Option	Data kind	Encrypted	Best for
SharedPreferences	Simple key-value	No	Settings, flags, small prefs
Secure storage	Key-value secrets	Yes (OS keystore)	Tokens, passwords, keys
SQLite (sqflite)	Relational tables	No by default	Complex, related local data + queries
Hive / Isar	NoSQL objects	Optional	Fast local objects, offline cache
Remote DB (API)	Server-side	In transit (HTTPS)	Shared, multi-device source of truth

Offline & sync: local-first apps cache remote data on the device so the UI works offline, then synchronise when connectivity returns. Rule of thumb — secrets → secure storage • structured local → SQLite / Hive • settings → prefs • shared truth → API.

PART 5 OF 6

REST API Integration

- How REST and the HTTP request/response cycle work
- An API service layer: base URLs, model mapping, GET and POST
- Loading, empty and error states with FutureBuilder
- Auth headers, JWT tokens, timeouts and resilient requests



REST APIs & the HTTP Workflow

REST in one line. The app sends an HTTP request to a URL (endpoint); the server returns a response — usually JSON. Each request is independent (stateless).

HTTP methods ≈ CRUD

GET	Read data
POST	Create new
PUT	Replace
PATCH	Update part
DELETE	Remove

Status codes

2xx Success — 200 OK • 201 Created

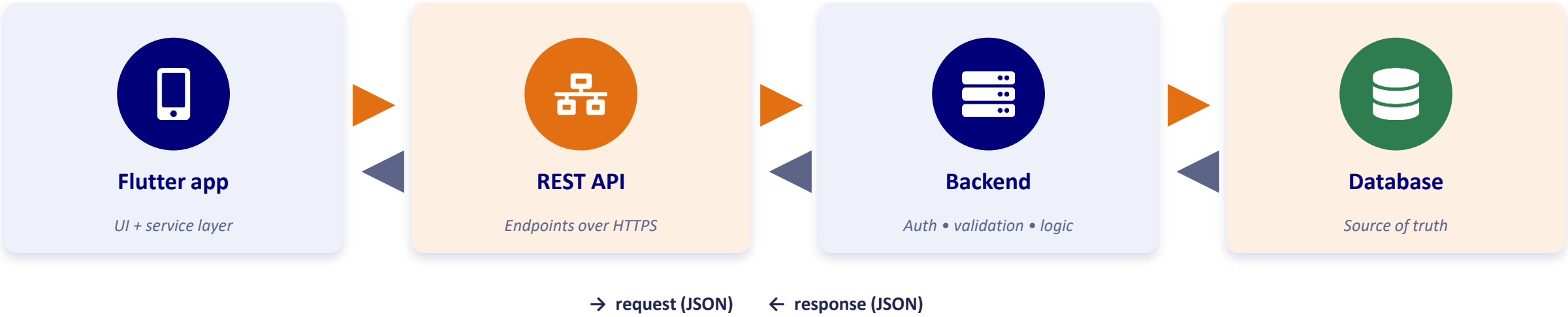
4xx Client error — 400 • 401 • 403 • 404

5xx Server error — 500 • 503

Always branch on the status code — a 200 is not guaranteed.

Flutter ↔ API ↔ Backend ↔ Database

Your app talks only to the API. The backend handles auth, validation and business logic, then reads or writes the database.



Why the app never touches the database directly: the backend enforces authentication, validates input, applies business rules and keeps credentials server-side. Putting DB access in the app would leak secrets and bypass every safeguard. The mobile app trusts nothing and verifies everything through the API.

An API Service Layer

Wrap all networking in **one service class**. The UI calls methods; the **service owns the base URL, Request headers, JSON parsing** and **error handling**.

api_service.dart

```dart
class ApiService {
  // base URL switches per environment
  static const _base = String.fromEnvironment(
    'API_URL',
    defaultValue: 'https://api.taskflow.dev');

  final http.Client _client;
  ApiService(this._client);   // inject for testing

  Uri _url(String path) => Uri.parse('$_base$path');
}

// run with:  flutter run
//   --dart-define=API_URL=https://staging.api...
```

Why a service layer

- Keeping networking in **one place** makes the code ***cleaner and easier to maintain.***
- **String.fromEnvironment('API_URL')** allows the app to use different URLs for development, staging, and production.
- This means we can switch environments without changing the source code.
- The **http.Client** is passed into the service through the constructor.
- This is **dependency injection**, and it makes the service easier to test with a mock client.
- Widgets should not build URLs or parse JSON themselves.

GET: Fetch, Map & Render

Check the status, decode the JSON list, map to typed objects — then let FutureBuilder drive the loading / empty / error / success UI.

fetch_tasks.dart

```dart
Future<List<Task>> fetchTasks() async {
  final res = await _client.get(_url('/tasks'));
  if (res.statusCode != 200) {
    throw Exception('Failed: ${res.statusCode}');
  }
  final list = jsonDecode(res.body) as List;
  return list
    .map((j) => Task.fromJson(j))
    .toList();
}
```

task_list_view.dart

```dart
FutureBuilder<List<Task>>(
  future: api.fetchTasks(),
  builder: (context, snap) {
    if (snap.connectionState !=
        ConnectionState.done) {
      return const CircularProgressIndicator();
    }
    if (snap.hasError) return ErrorView(snap.error);
    final tasks = snap.data ?? [];
    if (tasks.isEmpty) return const EmptyView();
    return TaskListWidget(tasks);
  },
)
```

This is the four-state UI from Part 1, now wired to real data: loading → error → empty → success.

Create, Update & Delete

POST a JSON body to create; **PUT/PATCH** to change; **DELETE** to remove. Check the status, and guard against duplicate submissions.

mutations.dart

```dart
Future<Task> addTask(Task t) async {
  final res = await _client.post(
    _url('/tasks'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode(t.toJson()),
  );
  if (res.statusCode != 201) {
    throw Exception('Create failed');
  }
  return Task.fromJson(jsonDecode(res.body));
}

// update part / replace / remove
_client.patch(_url('/tasks/${t.id}'), body: body);
_client.delete(_url('/tasks/${t.id}'));
```

Don't create duplicates

- **Disable the submit button while the request is in flight**
- Track an isSubmitting / loading flag
- **A double-tap must not POST twice**
- Re-enable on success or error
- **PUT replaces the whole resource; PATCH updates part**
- Always verify the status before trusting the result

Authentication, Tokens & Failures

Read the token from secure storage, send it as a Bearer header, and handle 401, timeouts and lost connections explicitly.

authed_request.dart

```dart
Future<http.Response> authedGet(String path) async {
  final token = await storage.read(key: 'jwt');
  try {
    final res = await _client.get(
      _url(path),
      headers: {'Authorization': 'Bearer $token'},
    ).timeout(const Duration(seconds: 10));

    if (res.statusCode == 401) throw AuthException();
    return res;
  } on TimeoutException {
    throw Exception('Server timed out');
  } on SocketException {
    throw Exception('No internet connection');
  }
}
```

The JWT flow

- **Login → server returns a JWT**
- Store it in secure storage (not prefs!)
- **Attach it as a Bearer header on each call**
- 401 → token expired → refresh or re-login
- **Set a timeout on every request**
- Catch SocketException / TimeoutException
- **Never log tokens**

Robust API Integration

Scaling the data

- **Pagination — page & limit, or cursors**
- Don't fetch 10,000 tasks at once
- **Search & filter via query parameters**
- Debounce search input before calling
- **Infinite scroll loads the next page**

Resilience

- **Timeout on every call**
- Retry transient errors with backoff — not 4xx
- **Offline → serve cache, queue writes**
- Prevent duplicate in-flight requests
- **Centralise error handling in the service**

Golden rule. Keep API logic OUT of widgets. Widgets call the service; the service owns URLs, headers, parsing and errors — making it testable and reusable.

Activity 3 — Find the Bugs in This API Service

Your task (7 minutes, pairs). This getTasks() method has several real-world bugs. List every problem you can find and how you'd fix it.

buggy_service.dart

```dart
Future<List<Task>> getTasks() async {
  final res = await client.get(url('/tasks'));

  // straight to parsing...
  final data = jsonDecode(res.body);
  return data
    .map((j) => Task.fromJson(j))
    .toList();
}
```

Look for…

- Is the status code checked?
- Network / timeout handling?
- Is the auth token attached?
- Are the JSON types safe?
- What about an empty or error body?
- Where should this logic live?

Model answer in the speaker notes. Time: ~7 minutes + 4 to review together.

PART 6 OF 6

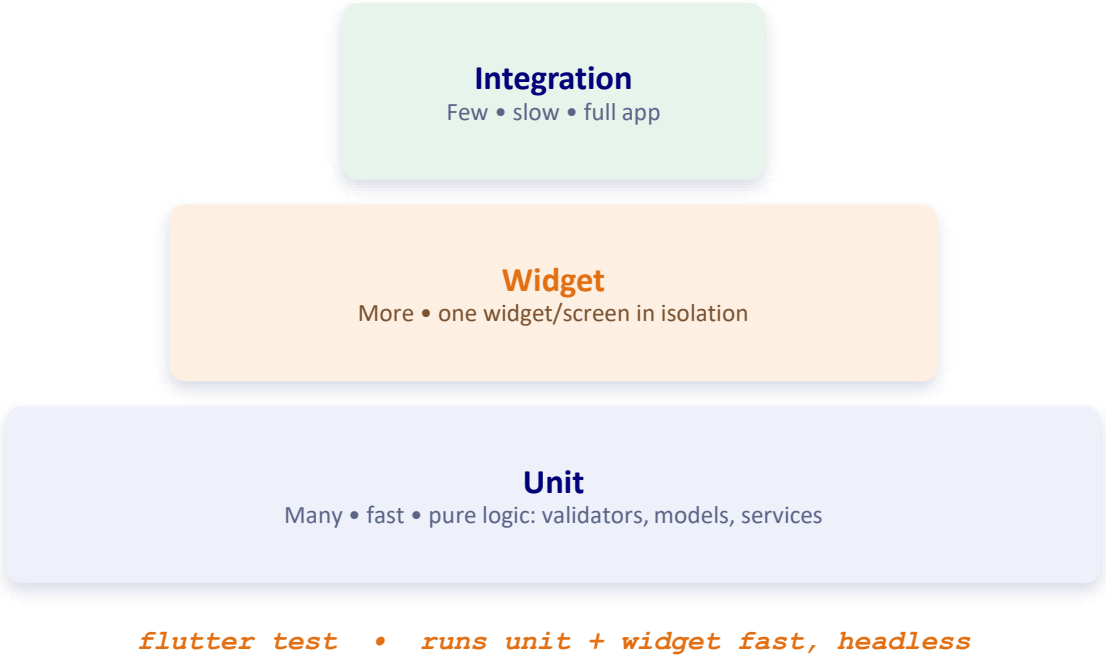
Testing & Deployment

- The Flutter testing pyramid: unit, widget and integration tests
- Writing readable unit and widget tests; mocking the API
- Debug / profile / release modes, versioning and signing
- Store submission, secrets, crash reporting and post-release monitoring



The Flutter Testing Pyramid

More tests at the bottom (fast, cheap, isolated); fewer at the top (slow, broad, high-confidence). Balance — don't invert it.



Why test a mobile app?

- Many devices, OS versions and screen sizes — you can't tap through them all by hand
- Catch regressions before users do; ship with confidence
- Tests document expected behaviour for the whole team

Testable architecture = dependency injection

Hard to test: a widget that news up its own ApiService() — the test hits the real network.

Testable: inject the service (constructor / Provider). Tests pass a fake; production passes the real one.

```
TaskListView(api: injectedService) → swap real ↔ fake
```

Unit & Widget Tests in Practice

Unit test = pure logic, no UI. **Widget test** = pump a widget, find elements, interact, assert. Same patterns test validation, navigation and storage.

validator_test.dart

```dart
void main() {
  group('email validator', () {
    test('rejects a bad address', () {
      expect(validateEmail('nope'),
             isNotNull); // returns error text
    });
    test('accepts a valid address', () {
      expect(validateEmail('a@b.com'),
             isNull);    // null = valid
    });
  });
}
```

add_task_test.dart

```dart
testWidgets('shows error on empty title',
    (tester) async {
  await tester.pumpWidget(
    const MaterialApp(home: AddTaskScreen()));

  // tap Save without typing a title
  await tester.tap(find.text('Save'));
  await tester.pump();

  expect(find.text('Title is required'),
         findsOneWidget);
});
```

Notice: the widget test verifies the SAME validator from Part 3 — unit tests prove the rule, widget tests prove it's wired to the UI.

Mocking the API, Integration & Coverage

Tests must not hit the real network. Inject a fake service to return canned data — instantly, offline, and on demand for error paths.

fake_api_test.dart

```dart
class FakeApi implements ApiService {
  @override
  Future<List<Task>> fetchTasks() async =>
    [Task(id: 1, title: 'Demo')];
}

testWidgets('renders tasks from API',
    (tester) async {
  await tester.pumpWidget(MaterialApp(
    home: TaskListView(api: FakeApi())));
  await tester.pumpAndSettle();
  expect(find.text('Demo'), findsOneWidget);
});
```

Integration tests

- integration_test package drives the REAL app on a device/emulator
- Verify full journeys: log in → add task → see it in the list
- Slower and fewer — cover only critical end-to-end flows

Coverage & where to run

- `flutter test --coverage`
- Coverage shows untested lines — a guide, not a goal; 100% proves nothing
- Emulators = fast & free for most tests; physical devices for camera, GPS, sensors & performance

Build Modes, Versioning, Signing & Secrets

Ship the RELEASE build — compiled, optimised, no debug tooling. Then version it, sign it, and keep secrets out of the binary.

Debug

Hot reload, asserts, DevTools. Slow & large. For development only.

Profile

Release-like performance + profiling on. For measuring frames/jank.

Release

AOT-compiled, optimised, stripped. The ONLY build you ship.

Versioning

version: 1.4.2+27

- 1.4.2 = version NAME shown to users
- +27 = build NUMBER, must increase every upload
- Set once in pubspec.yaml; stores reject a re-used build number

Android build

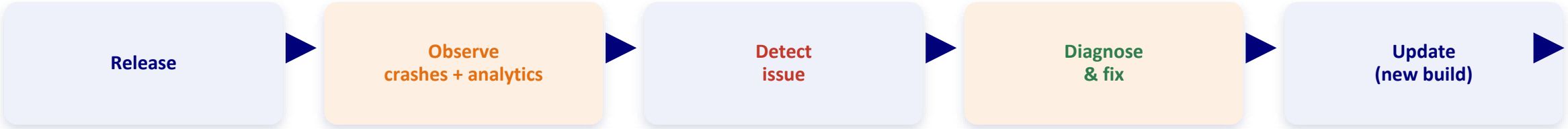
- flutter build appbundle → .aab for Google Play (optimised per device)
- flutter build apk → .apk for direct install / sideload
- Sign with a keystore (release key). Lose it = can't update the app — back it up

iOS & secrets

- iOS: archive in Xcode; needs an Apple Developer cert + provisioning profile
- **Secrets via --dart-define, NOT in source**
- `flutter build apk --dart-define=API_URL=…`
- Never commit keys/tokens; use env config per build

Submission, Monitoring & the Release Lifecycle

Shipping isn't the finish line — it's the start of a loop: release, observe, learn, improve.



↺ continuous loop — each release informs the next

Before you submit

- App icon + splash screen set; store listing & screenshots ready
- **Declare every permission and explain WHY (camera, location…)**
- Publish a privacy policy — required if you collect any user data
- Run release-build tests on real devices; check performance
- Both stores review submissions — allow days, not minutes

Observe in production

- Crash reporting (Crashlytics / Sentry) — stack traces from real users you'll never reproduce locally
- Analytics: track key flows (sign-ups, task completion) to guide decisions
- **Respect privacy: anonymise, make analytics opt-in, collect the minimum**
- Watch ratings, ANRs and crash-free rate; patch and re-release

Quick Quiz — Questions 1–5

Try each one before revealing — answers and explanations are in the speaker notes.

1 RESPONSIVE UI

Which widget reports the screen's width, height and orientation?

a) Container
b) MediaQuery
c) Scaffold

2 LAYOUT

Inside a Row, which widget makes a child expand to fill leftover space?

a) Padding
b) Center
c) Expanded

3 ROUTING

Which package gives declarative, URL-based routing with deep-link support?

a) provider
b) go_router
c) http

4 STATE MANAGEMENT

You extend ChangeNotifier and call ____ to tell listeners state changed.

(one method name)

5 STATE MANAGEMENT

"Ephemeral" state is best described as…

a) app-wide data
b) state local to one widget
c) data on a server

Quick Quiz — Questions 6–10

Covering forms, device features, storage, REST APIs, testing and deployment.

6 FORMS

A TextFormField validator returns ____ when the value is valid.

a) true
b) null
c) an empty string

7 DEVICE FEATURES

On modern Android/iOS, before opening the camera you must first…

a) nothing
b) request a runtime permission
c) restart the app

8 LOCAL STORAGE

Where should a JWT authentication token be stored?

a) SharedPreferences
b) a global variable
c) secure storage

9 REST API

In a GET handler, what must you check before jsonDecode(res.body)?

a) the body length
b) the status code
c) the device clock

10 TESTING

Which one design choice makes a widget easy to unit/widget test?

a) more setState
b) dependency injection
c) bigger build()

Lecture 10 in One Slide

From a working app to a production-ready one — here is everything TaskFlow Pro gained today.

Adaptive UI

MediaQuery, LayoutBuilder, themes, 4 UI states

Navigation

Stack, named & declarative routing, guards

State

Provider + ChangeNotifier; lift state, fewer rebuilds

Forms

Validation, controllers, pickers, submission

Device + storage

Permissions, camera/GPS, secure storage, JSON

API + ship

REST service, auth, tests, release & monitor

Learning outcomes exercised

LO1	Evaluated UI, routing, state & storage options
LO2	Built advanced features into one app
LO3	Integrated REST APIs, data, tests & release
LO4	Justified packages & trade-offs for production

In this week's lab. Extend TaskFlow Pro: responsive home, Provider state, a validated task form, a securely-stored token, and a REST task list with loading/error states.

Before Next Time & References

Prepare for the lab & next session

- Have the Flutter SDK plus an emulator or a real device ready to run
- Re-open last week's TaskFlow project — you'll evolve it into TaskFlow Pro
- Skim the pub.dev pages for provider, go_router, http, shared_preferences and flutter_secure_storage
- Revisit today's three activities — responsive screen, state choice, and the buggy API service
- Bring one design decision from your own app where you're unsure which option to pick
- **Come to the lab having attempted the responsive home screen — we build from there**

References & further reading

Flutter documentation
docs.flutter.dev — UI, navigation, testing & deployment guides

Dart language
dart.dev — language tour, null safety, async

pub.dev
Package docs: provider, go_router, http, dio, image_picker, geolocator, connectivity_plus

Secure storage & prefs
flutter_secure_storage and shared_preferences package READMEs

Testing guide
docs.flutter.dev/testing — unit, widget & integration

Material 3
m3.material.io — theming & adaptive design

Flutter Apprentice
Kodeco — practical project-based Flutter book

Thank you — see you in the lab.

SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks



Lecture 11

DevOps and Cloud Hosting Platforms

Focus: deploying the full-stack app

Year 3 • Semester 1 • 4 Credits

Lecturer: Eishan Weerasinghe • eishan.w@sliit.lk

Where This Lecture Fits in SE3090

Earlier	Earlier	Prev	TODAY	Goal
React frontend & .NET REST API	PostgreSQL, auth & security	Git, CI/CD & code quality	DevOps & cloud deployment	A live, monitored full-stack app

- **The capstone of the module: take everything built so far and ship it to the cloud**
- Builds directly on the CI/CD pipeline from the previous lecture — deployment is the final stage
- You will deploy the SAME stack you've been building: React + ASP.NET Core + PostgreSQL
- Directly assessed: the Main Assignment expects a deployed, working, full-stack application

Lecture Learning Outcomes

LO2

Apply suitable frameworks & tools

Deploy a full-stack application (frontend, backend API and database) to appropriate cloud hosting platforms with correct configuration.

LO3

Use best practices for delivery & operations

Apply DevOps workflows, environment configuration, secret management, monitoring, logging and maintenance to a deployed system.

LO4

Select & justify approaches

Choose and justify suitable hosting platforms and deployment strategies based on cost, scalability, security and reliability.

Why DevOps & Cloud Hosting Matter

Software has to ship

Code that isn't deployed delivers zero value. Users only ever see the running app, never your repo.

Faster, safer delivery

DevOps + automation let teams release frequently and reliably — from once a year to many times a day.

Cloud removes the heavy lifting

No buying servers or running a data centre — provision infrastructure in minutes, pay for what you use.

Scale & reliability on demand

Handle 10 users or 10 million; the platform adds capacity, backups and uptime you couldn't build alone.

Lecture Roadmap – 3 Hours

1	**DevOps culture & practices** Mindset · collaboration · automation · shared responsibility	~30 min
2	**Build, test, deploy & monitor workflows** The DevOps pipeline · deployment strategies · feedback loops	~30 min
3	**Cloud hosting platforms** IaaS/PaaS/SaaS/serverless · the landscape · choosing one	~35 min
4	**Deploying frontend & backend** Static/SPA hosting · API hosting · CORS · health checks · errors	~40 min
5	**Databases, configuration & monitoring** Managed DBs · env vars & secrets · logging · alerts · maintenance	~40 min

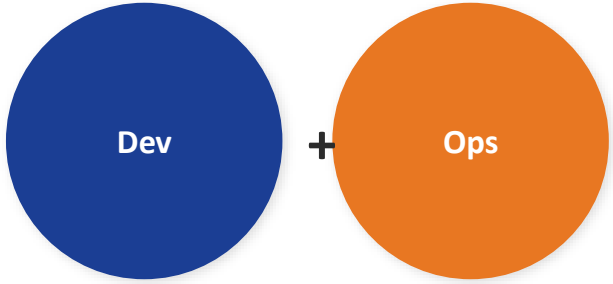
PART 1

DevOps Culture & Practices

- What DevOps is — and isn't
- Breaking down the Dev–Ops wall
- Mindset: automation, shared responsibility, continuous improvement
- DevOps vs traditional delivery, and its benefits

What Is DevOps?

- **DevOps = Development + Operations working as ONE team**
  - A culture, set of practices & tools that shorten the path from code to production
- **Goal: deliver software faster, more reliably and more frequently**
- **Built on automation, collaboration and fast feedback**
- **NOT just a tool or a single person's job title**
  - NOT only CI/CD — that's one important piece of it



DevOps

Build, ship and run software together — continuously.

Breaking Down the Dev–Ops Wall

❌ Traditional silos

- Dev writes code, 'throws it over the wall' to Ops
- Ops deploys & operates, often with little context
- Conflicting goals: features vs stability
- Slow, manual, risky releases
- Blame when things break: 'works on my machine' vs 'your code broke prod'
- Problems found late, fixed slowly

✓ DevOps collaboration

- One team owns build → ship → run
- Shared responsibility & shared tools
- Aligned goal: deliver value reliably
- Automated, frequent, low-risk releases
- Blameless culture: fix the system, not people
- Fast feedback: problems caught & fixed early

The DevOps Mindset — CALMS

Culture	Collaboration, shared ownership, blameless learning
Automation	Automate builds, tests, deployment & infrastructure
Lean	Small batches, eliminate waste, continuous improvement
Measurement	Monitor everything; decisions driven by data & feedback
Sharing	Open knowledge, tools & responsibility across the team

Automation, Improvement & Shared Responsibility

Automate everything repetitive

Builds, tests, deployments and infrastructure. Consistent, fast and error-free — humans focus on solving problems, not clicking steps.

Continuous improvement

Every release and incident is a chance to learn. Small, frequent changes + retrospectives make the system and team better over time.

Shared responsibility

'You build it, you run it.' The whole team owns quality, deployment and uptime — not a separate team to blame.

DevOps vs Traditional Delivery

Aspect	Traditional	DevOps
Teams	Separate Dev & Ops silos	Collaborative, shared ownership
Releases	Rare, large, scheduled	Frequent, small, on-demand
Deployment	Manual, error-prone	Automated pipelines (CI/CD)
Feedback	Slow; problems found late	Fast; monitored continuously
Risk	High per release	Low — small changes, easy rollback
On failure	Blame the other team	Blameless; fix the system

The shift: from rare, risky, manual, blame-heavy releases → frequent, safe, automated, collaborative delivery.

Benefits of DevOps

Faster delivery

Ship features and fixes to users in minutes/hours, not months — competitive advantage.

Higher reliability

Automation + monitoring mean fewer failures and faster recovery when they happen.

Better scalability

Automated, repeatable infrastructure scales smoothly as demand grows.

Stronger collaboration

Shared goals and tools break silos; happier, more productive teams.

Continuous feedback

Monitoring and fast loops drive constant, data-driven improvement.

Security built in

'DevSecOps' bakes security checks into the pipeline, not bolted on at the end.

Checkpoint 01 – Quick Questions

Answer with your neighbour — 4 minutes

Q1 In one sentence: what is DevOps, and why is it NOT just 'a tool' or 'one person's job'?

Q2 Before DevOps, what conflicting goals did Dev and Ops teams have?

Q3 Why is deploying many small changes often SAFER than one big release?

Q4 What does a 'blameless culture' mean, and why does it improve reliability?

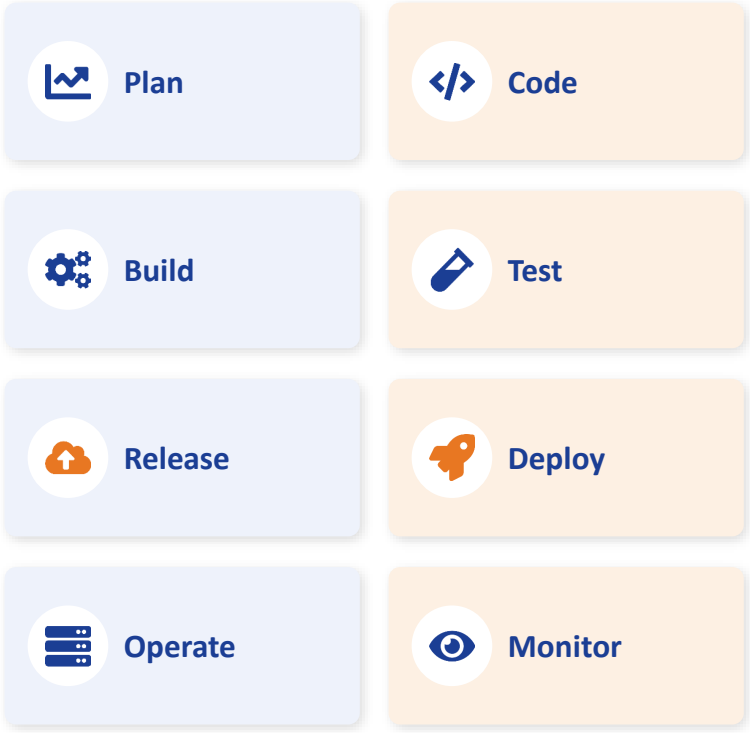
PART 2

Build, Test, Deployment & Monitoring Workflows

- The software delivery lifecycle & the DevOps loop
- Build, test, deploy and monitor workflows
- Deployment strategies: rolling, blue-green, canary
- Feedback loops & the CI/CD connection

The Software Delivery Lifecycle

- **The journey from an idea to running, monitored software**
  - DevOps makes this a continuous CYCLE, not a one-way line
- **Each stage feeds the next; monitoring feeds back to planning**
- **Automation connects the stages into a smooth pipeline**
- **The faster and safer the loop, the more value you deliver**



Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → (back to Plan)

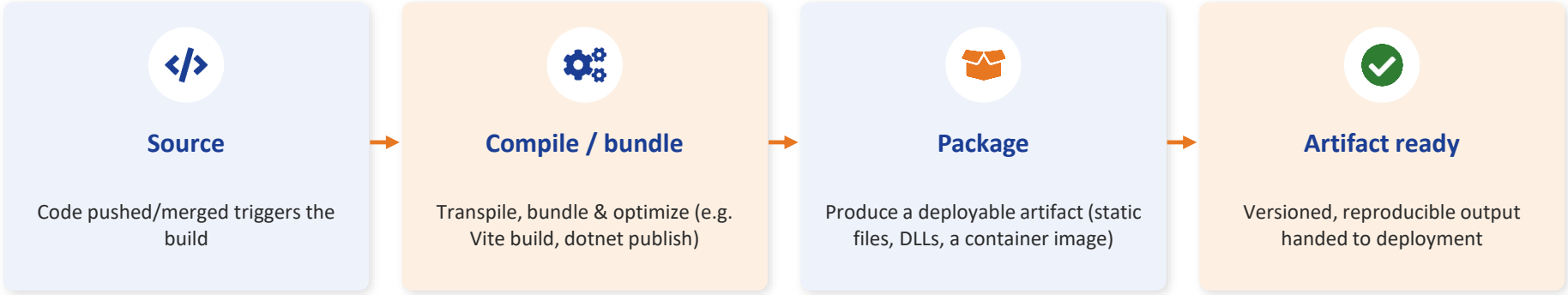
The DevOps Pipeline — A Continuous Loop



Why a loop?

- The two halves connect: CI (build & test code) flows into CD (release, deploy & run it); monitoring feeds back to planning
- It never stops — continuous integration, delivery and improvement around the loop

Build Workflow



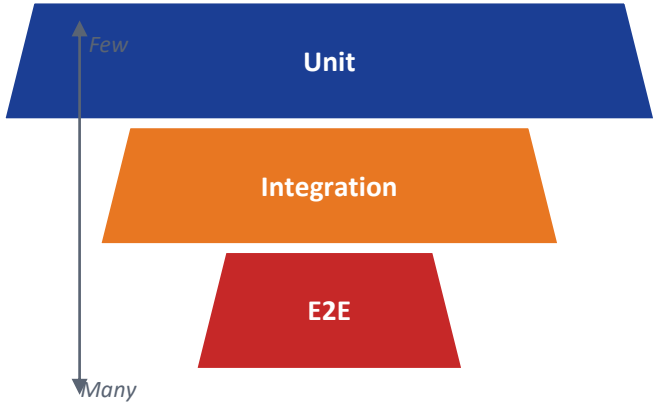
Key ideas

- **Build from a CLEAN checkout every time — reproducible, no 'works on my machine'**
- Frontend: npm run build → optimized static files in a dist/ or build/ folder
- Backend: compile to runnable output (dotnet publish) or a container image
- **The same artifact is promoted through environments — build once, deploy many times**

Automated Testing Workflow

- **Tests run automatically in the pipeline on every change**
- **A failing test STOPS the pipeline — broken code never deploys**
- **The testing pyramid:**
  - Unit tests – many, fast tests for individual functions/components.
  - Integration tests – check whether different components work together.
  - End-to-End (E2E) tests – test complete user workflows.

Gates quality before release; catches regressions instantly



Lots of fast unit tests at the base; a few slow end-to-end tests at the top.

Deployment Workflow & Strategies

Deployment means moving the tested application into an environment where users can access it.

Normally, software moves through **Development → Staging → Production**.

There are three common deployment strategies :-



Rolling Deployment

Gradually replace old instances with the new version. Usually no downtime.



Blue-Green Deployment

Maintain two environments. Blue = current version, Green = new version. Switch traffic to Green when ready.



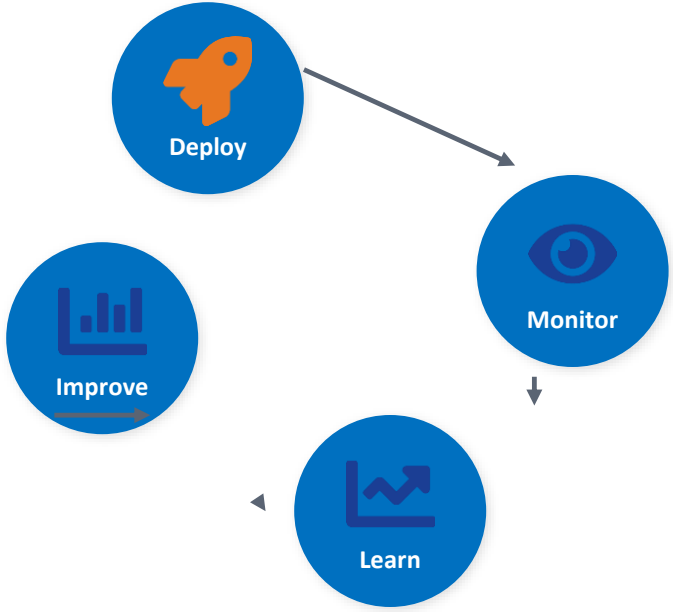
Canary Deployment

Release the new version to a small percentage of users first and monitor it before releasing to everyone.

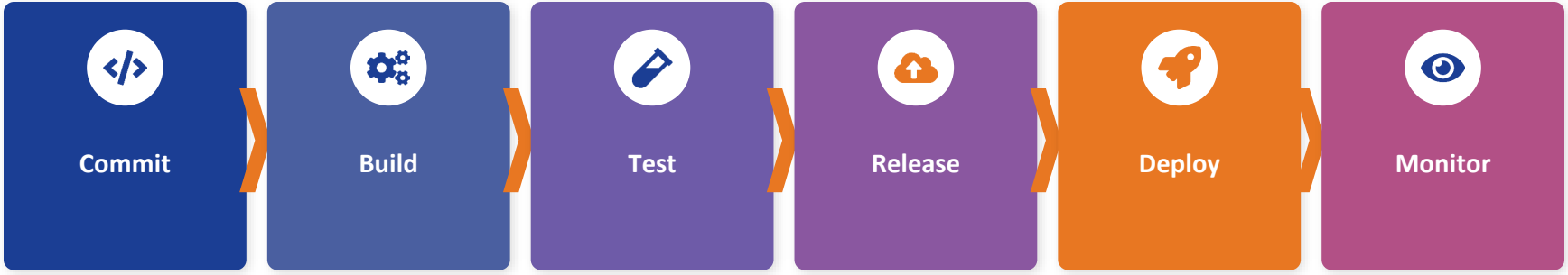
Always keep ROLLBACK easy. If a deploy goes wrong, you must be able to return to the last good version fast. Blue-green and canary make rollback near-instant; that safety net is what makes frequent deployment possible.

Monitoring Workflow & Feedback Loops

- **Deployment is the START of operations, not the end**
  - Monitor the live system: is it up? fast? error-free?
- **Collect logs, metrics, errors and uptime continuously**
- **Alerts notify the team the moment something goes wrong**
- **Feedback loop: monitoring data → informs the next changes**
  - Closes the DevOps loop back to planning & improvement



How CI/CD Connects It All



CI — Continuous Integration

- Commit → Build → Test, **automatically**
- Catches integration bugs early
- Produces a tested artifact

CD — Continuous Delivery/Deployment

- Release → Deploy the tested artifact
- Delivery = one click; Deployment = automatic
- Then operate & monitor in production

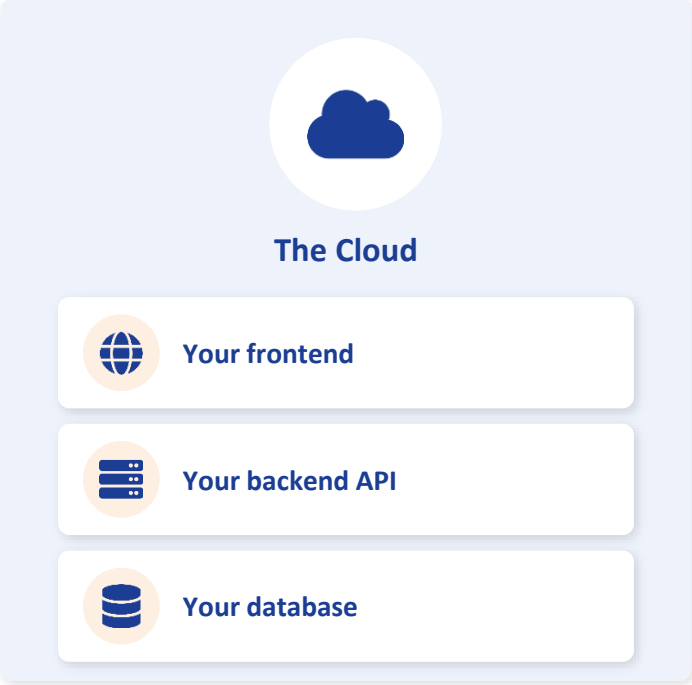
PART 3

Cloud Hosting Platforms

- What cloud hosting is & why teams use it
- IaaS, PaaS, SaaS & serverless
- The platform landscape: AWS, Azure, GCP, Vercel, Netlify, Render…
- Choosing a platform: cost, scale, security, reliability

Introduction to Cloud Hosting

- **Cloud hosting = running your app on someone else's managed servers, over the internet**
  - Rent computing power instead of owning physical machines
- **Provision resources in minutes, on demand**
- **Pay-as-you-go: pay only for what you use**
- **Provider handles hardware, networking, power, cooling & much of the maintenance**
  - Accessible from anywhere; scales up or down as needed



Why Modern Systems Use Cloud Platforms

Lower upfront cost

No buying servers; pay-as-you-go turns big capital costs into small running costs. Free tiers for small projects.

Elastic scalability

Scale up for a traffic spike, down when quiet — automatically. Handle growth without re-architecting.

Reliability & uptime

Redundancy, backups and SLAs you couldn't build yourself; data centres worldwide.

Speed & ease of deployment

Go from code to live in minutes; managed services remove most ops work.

Global reach

Serve users worldwide with low latency via regions & CDNs.

Built-in security

Providers invest heavily in security, compliance & DDoS protection.

Cloud Service Models



IaaS

Infrastructure as a Service

we rent infrastructure such as virtual machines, but we manage the **OS, runtime, and application.** Max control & work.

AWS EC2, Azure VMs



PaaS

Platform as a Service

we mainly provide our **application code**, while the platform manages the servers, OS, and runtime.

Render, Railway, Heroku, App Service



Serverless

Functions / managed

Run code per-request; auto-scales to zero. No servers to think about at all.

AWS Lambda, Cloud Functions, Vercel



SaaS

Software as a Service

Ready-made software you just use — no deployment. The end-user model.

Gmail, Figma, Notion

More control & responsibility ← IaaS

SaaS → less control, less work

Who Manages What? (Responsibility)

	On-Prem	IaaS	PaaS	SaaS
Application	You	You	You	Provider
Data	You	You	You	Provider
Runtime	You	You	Provider	Provider
OS	You	You	Provider	Provider
Virtualization	You	Provider	Provider	Provider
Servers	You	Provider	Provider	Provider
Networking	You	Provider	Provider	Provider

The Cloud Platform Landscape

The hyperscalers — full-service, everything-as-a-service (steeper learning curve)

- **AWS**
- **Microsoft Azure**
- **Google Cloud (GCP)**

Developer-friendly platforms — simple, fast, great free tiers (ideal for your project)

Vercel Frontend / Next.js	**Netlify** Frontend / static	**Render** Full-stack / APIs	**Railway** Full-stack / APIs
Heroku Full-stack (classic)	**Firebase** Google BaaS	**Supabase** Postgres + backend	**Neon** Serverless Postgres

Cloud Platform Comparison

Platform	Best for	Model	Free tier?
AWS / Azure / GCP	Large-scale, full control, enterprise	IaaS + PaaS + serverless	Limited / credits
Vercel	Frontend & Next.js apps	Serverless / PaaS	Yes — generous
Netlify	Static sites & SPAs	Serverless / PaaS	Yes — generous
Render / Railway	Full-stack apps & APIs	PaaS	Yes
Heroku	Quick full-stack (classic)	PaaS	No (paid)
Supabase / Neon	PostgreSQL + backend services	Managed DB / BaaS	Yes

No single 'best' platform — it depends on the project. Match the tool to your requirements & justify it (LO4).

Selecting a Hosting Platform (LO4)

Cost

Free tier limits, pricing model, cost at scale. Predictable for a student project?

Scalability

Can it grow with demand? Auto-scaling? Will you hit limits soon?

Security

HTTPS by default, access controls, compliance, DDoS protection.

Reliability

Uptime/SLA, redundancy, backups, track record.

Ease of deployment

Git integration, build automation, learning curve, good docs.

Ecosystem fit

Supports your stack (.NET, Node, Postgres)? Integrations you need?

Checkpoint 02 – Discussion

Discuss in pairs — 5 minutes, then we compare answers

D1 Explain IaaS vs PaaS vs serverless to a friend — using the 'how much do YOU manage?' idea.

D2 On PaaS, name two things the provider handles and two things you're still responsible for (incl. security).

D3 Why might a student team pick Vercel + Render + Supabase over raw AWS for their project?

D4 Your client needs strict data-residency compliance and 99.99% uptime. Does that change your platform choice? How?

PART 4

Deploying the Frontend & Backend

- Frontend: build output, static/SPA hosting, domains, HTTPS & CDN
- Backend: API hosting, server vs container, CORS & health checks
- Build commands & output folders
- Common deployment errors & how to fix them

Frontend Build & Static / SPA Hosting

- **A React app isn't served as-is — it's BUILT into static files first**
  - npm run build → optimized, minified HTML, CSS & JS bundles
- **Output is a folder of plain static assets (dist/ or build/)**
- **Static hosting + a global CDN serve those files fast, worldwide**
- **An SPA is ONE index.html that swaps views with JavaScript**
  - So the host must fall back to index.html for every route

build output

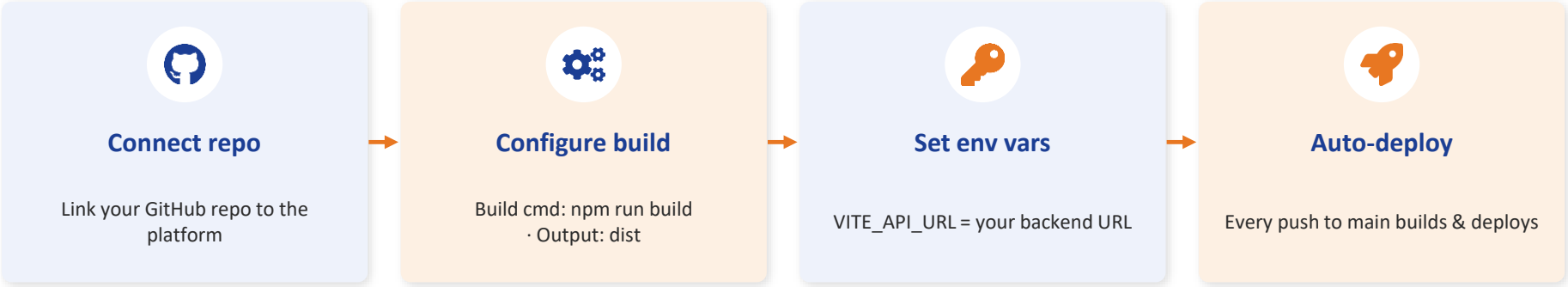
```
# build the frontend
npm run build

dist/              # the deployable output
  ├─ index.html
  ├─ assets/app-3f9.js    # hashed
  └─ assets/app-3f9.css
```

SPA routing gotcha

Refreshing /products asks the server for a file that doesn't exist → 404. Fix: configure a rewrite so ALL paths serve index.html and React Router handles them.

Frontend Deployment Workflow (Vercel / Netlify)



Typical platform settings

Setting	Value
Framework preset	Vite / Create React App
Build command	npm run build
Output directory	dist (or build)
Install command	npm ci

Why these platforms?

- Zero-config for popular frameworks
- Automatic HTTPS + global CDN
- Git-based: push to deploy
- Free tier ideal for students
- Preview deploys per PR

Domains, HTTPS, CDN & Common Frontend Errors

Going live properly

- **Custom Domain**: connect your own domain to the hosting platform using DNS records such as CNAME or A records.
- **HTTPS**: modern hosting platforms can provide TLS certificates automatically, allowing secure HTTPS access.
- **CDN**: static files are cached on edge servers closer to users, improving loading speed.
- **Cache-busting:** hashed filenames help ensure users receive the latest version of files after deployment.

Common frontend deploy errors

- **Blank page — wrong output dir or base path**
- **404 on refresh — missing SPA index.html fallback**
- **API calls fail — wrong/missing API URL env var or CORS**
- **Assets 404 — incorrect base / sub-path config**
- **Old version showing — stale cache; redeploy / hard refresh**

Backend API Hosting Concepts

- **A backend is a PROCESS that must keep running to serve requests**
  - Unlike static frontends, it needs CPU, memory & a runtime alive 24/7
- **Two common hosting models:**



Server-based (PaaS)

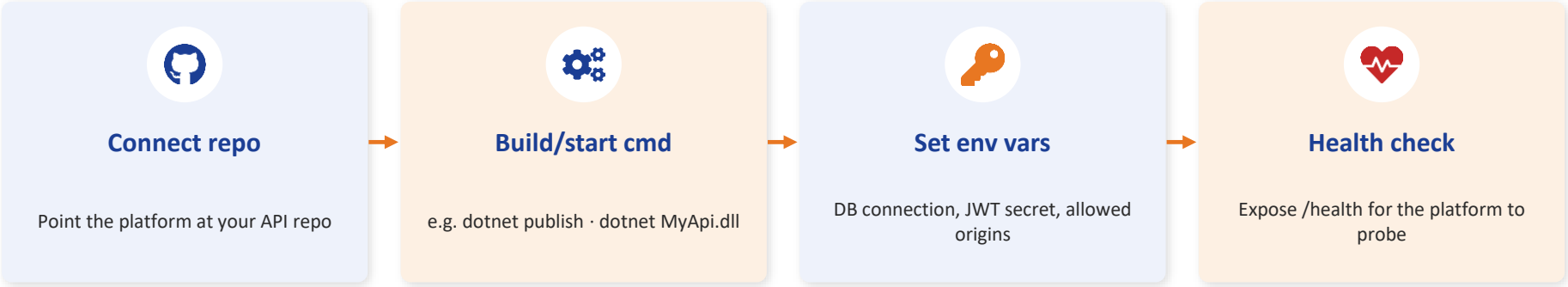
- We push our code, and the **platform builds and runs the application.**
- The platform handles areas such as the **OS, runtime, scaling, and infrastructure management.**
- *Examples include* Render, Railway, Azure App Service, and Heroku.
- This is generally a simpler approach because developers can focus mainly on the application.



Container-based (Docker)

- We package the **application together with its dependencies** into a container image.
- This helps the application run consistently across different environments — reducing the “works on my machine” problem.
- Examples of platforms/services that can run containers include Google Cloud Run, AWS ECS, and Kubernetes.
- Containers provide more portability and control, but they also introduce additional concepts to learn.

Backend API Deployment Workflow



Where to host a REST API

Render PaaS, simple	**Railway** PaaS, fast setup	**Azure App Service** Enterprise PaaS
AWS Beanstalk Managed AWS	**Cloud Run** Containers	**Heroku** Classic PaaS

Base URLs, CORS & Health Checks

```
// FRONTEND: call the deployed API, not localhost
const API = import.meta.env.VITE_API_URL;
fetch(`${API}/api/products`)

// BACKEND (ASP.NET Core): allow the frontend origin
app.UseCors(p => p
  .WithOrigins("https://lankamart.vercel.app")
  .AllowAnyHeader().AllowAnyMethod());

// health check endpoint
app.MapGet("/health", () => Results.Ok("OK"));
```

The 3 things that break

- **Base URL: frontend must point at the DEPLOYED API, never localhost**
- **CORS: the API must explicitly allow the frontend's origin**
- **Health: expose /health so the platform knows you're alive**

CORS errors are fixed on the BACKEND (allow the origin), not the frontend — a browser security rule, not a bug in your fetch call.

Common Backend Deployment Issues

App crashes on start	Missing env var (DB URL, secret) → set them BEFORE deploy; read the logs
Wrong port binding	Bind to the platform's $PORT, not a hardcoded 5000
Can't reach database	Wrong connection string, or DB firewall blocks the host
CORS blocked	Add the frontend origin to the API's CORS allow-list
Secrets exposed / missing	Use platform env vars; never commit .env or hardcode keys
Cold starts (free tier)	Idle apps sleep; first request is slow — expected on free plans

When in doubt: READ THE LOGS — the platform's log output names the exact failure almost every time.

Checkpoint 03 – Quick Questions

Answer with your neighbour — 4 minutes

Q1 Why does a React app need a BUILD step before deployment, and what is the output?

Q2 A deployed SPA returns 404 when you refresh on /products. What's the cause and fix?

Q3 The deployed frontend can't call the deployed API (CORS error). Which side do you fix?

Q4 Your API crashes on startup in the cloud. What's the most likely cause and your first step?

PART 5

Databases, Configuration, Monitoring & Maintenance

- Cloud database hosting & managed PostgreSQL
- Environment variables & production configuration
- Monitoring, logging & error tracking
- Uptime, alerts, maintenance & continuous improvement

Cloud Database Hosting & Managed PostgreSQL

- **Don't run your own database server in production — use a MANAGED one**
  - The provider handles backups, patching, scaling & availability
- **You get a connection string; you focus on your data**
- **Managed PostgreSQL options:**
  - Supabase, Neon — modern, generous free tiers (great for students)
  - Render PostgreSQL, Railway PostgreSQL — sit beside your app
  - AWS RDS, Azure Database for PostgreSQL — enterprise-grade

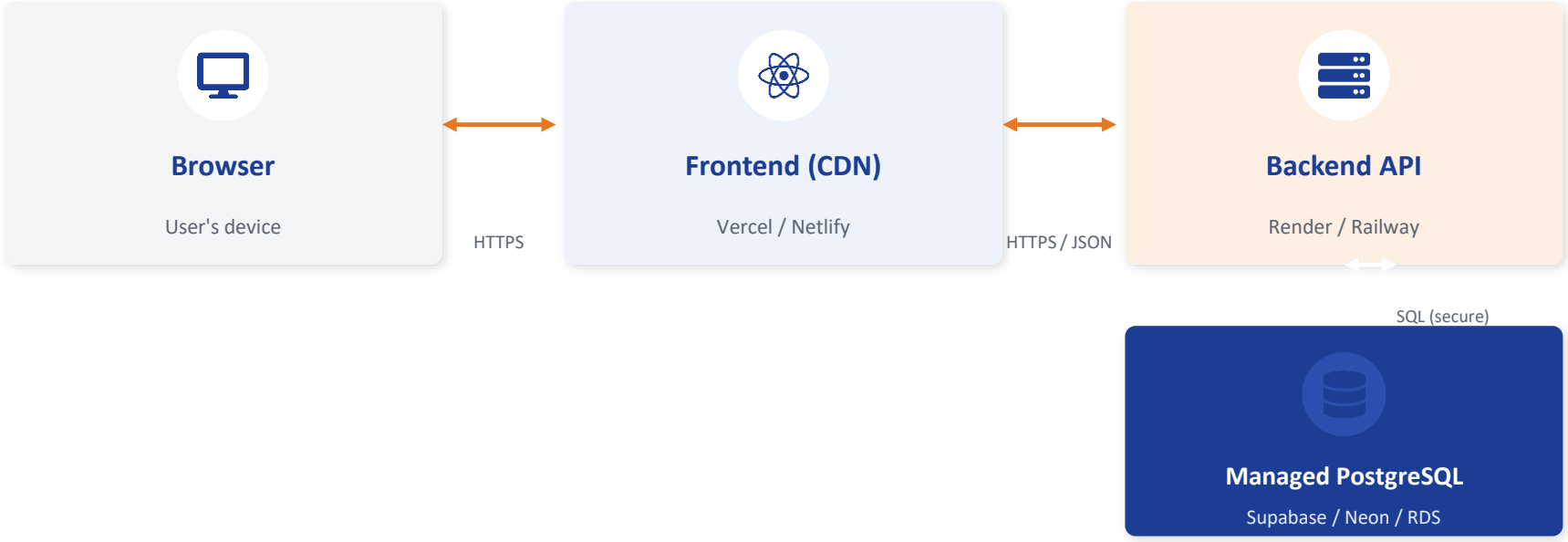
Managed handles for you

- Automated backups & restore
- Security patches & updates
- Scaling & failover
- Monitoring & metrics

You still own

- Schema design & migrations
- Keeping the connection string secret
- Sensible access rules
- Your data's correctness

Full-Stack Hosting Architecture



Each tier is hosted separately, connected over HTTPS — and only the backend talks to the database.

Connection Strings, Backups & DB Security

```
connection string

# a PostgreSQL connection string (a SECRET!)
postgresql://user:password@host:5432/dbname

# stored as an env var, never in code
DATABASE_URL=postgresql://...  # platform setting
```

- **Treat the connection string as a top secret credential**
  - Inject via env vars; rotate if ever exposed
- **Backups: managed providers back up automatically — know how to RESTORE**
  - Test restores; a backup you can't restore isn't a backup

Hosted database security checklist

- ✓ Strong, unique credentials — never the defaults
- ✓ Require SSL/TLS for connections
- ✓ Restrict access: only the backend's host/IP, not the public internet
- ✓ Least privilege: the app's DB user gets only what it needs
- ✓ Encrypt data at rest (managed providers do this)
- ✓ Regular backups + a tested restore plan

Environment Variables & Production Config

- **Env vars = configuration that lives OUTSIDE your code**
  - Same code, different values per environment (dev / staging / prod)
- **Hold secrets & settings: DB URL, JWT secret, API keys, base URLs**
- **Never hardcode these — keep them out of the repo entirely**
- **Locally: a git-ignored .env file. In production: platform settings**

```
.env  (local only, git-ignored)
DATABASE_URL=postgresql://...
JWT_SECRET=super-long-random-value
VITE_API_URL=https://api.lankamart.app

# read in code (never the literal!)
process.env.JWT_SECRET   // backend
import.meta.env.VITE_API_URL // frontend
```

Frontend vs backend env vars — a critical difference

- **Backend env vars are PRIVATE (on the server) → safe for real secrets like JWT_SECRET & DB passwords**
- **Frontend (VITE_*) vars are BUNDLED INTO PUBLIC JS → visible to anyone; only put non-secret values there**

Monitoring: Why It Matters After Deployment

- **Deployment isn't the finish line — it's the start of operations**
  - Without monitoring you learn about outages from angry users
- **Goal: detect & diagnose problems BEFORE (or faster than) users report them**



Uptime

Is the app reachable & responding? Track availability (e.g. 99.9%).



Performance

How fast? Response times, latency, throughput, slow endpoints.



Errors

What's failing? Error rates, exceptions, failed requests.



Logs

What happened & why? The detailed record for diagnosis.

Logging & Error Tracking

Good logging

- **Logging** records what happens while the application is running.
- Use log levels: **DEBUG, INFO, WARN, ERROR**.
- Log meaningful information with context such as **timestamp, request ID and user ID.**
- Never log **passwords, tokens, or other secrets.**
- **Structured logs**, such as JSON logs, are easier to search and filter.

Error tracking

- **Error-tracking tools** such as Sentry automatically capture application exceptions.
- They provide **stack traces, error context, affected requests/users, and alerts**.
- Duplicate errors can be grouped together, making problems easier to investigate.

Uptime Monitoring, Alerts & Incident Response

Status dashboard

API	Operational	120 ms
Frontend	Operational	45 ms
Database	Degraded	high load
Auth	Operational	98 ms

- **Uptime monitors ping your app from outside, 24/7**
- **Down? You get alerted (email / SMS / Slack) in seconds**
- **Set alerts on thresholds: errors, latency, downtime**
  - Avoid alert fatigue — alert on what's actionable

Incident response (when something breaks)

Detect (alert fires) → Acknowledge → Diagnose (logs/metrics) → Mitigate / rollback → Fix & verify → Post-mortem

Maintenance & Continuous Improvement

Update dependencies

Keep libraries current for fixes & features; review before merging (Dependabot).

Apply security patches

Patch known vulnerabilities promptly — in your code, deps & runtime.

Tend the database

Monitor growth, add indexes, archive old data, verify backups.

Improve continuously

Use monitoring data to find slow spots & errors; iterate and optimise.

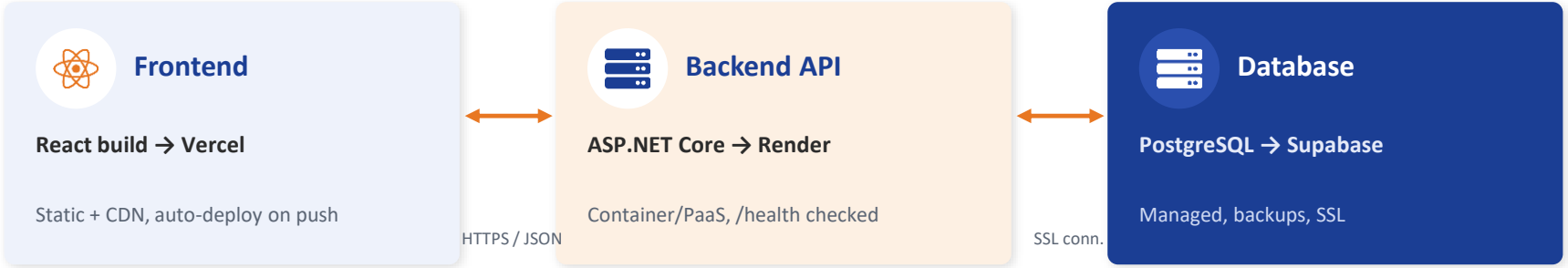
PART 6

Case Study, Activities & Wrap-Up

- End-to-end: deploying a full-stack app
- Group task: choose a hosting stack
- Deployment troubleshooting challenge
- Quiz, summary & what's next

Case Study: Deploying Full-Stack LankaMart

One commit → three hosted services, wired together by configuration



The configuration that wires them together

```
# Vercel (frontend) env
VITE_API_URL=https://lankamart-api.onrender.com
# Render (backend) env
DATABASE_URL=postgresql://...supabase.co:5432/db
JWT_SECRET=••••••   ALLOWED_ORIGINS=https://lankamart.vercel.app
```

Case Study: The Deployment Decisions

How the team deploys safely (practice → why — LO4)

Practice	What the team does	Why
Platform choice	PaaS (Vercel / Render / Supabase), not raw VMs	Small team, fast setup, managed ops — low overhead
Auto-deploy	Push to main → CI/CD builds, tests & deploys	Fast, repeatable releases; no manual FTP
Config	All URLs, keys & secrets in env variables	Same code in dev & prod; no secrets in the repo
CORS & health	Backend allow-lists the frontend; /health endpoint	Browser calls succeed; platform restarts on failure
Database	Managed PostgreSQL with backups + SSL	Durable, secure data without DBA effort
Monitoring	Uptime check + error tracking + alerts	Know about problems before users complain

Group Activity: Choose a Hosting Stack (20 min)

Groups of 3–4 · whiteboard or paper · present in 60 seconds

Scenario: "EventLanka" ticketing app

React frontend + ASP.NET Core REST API + PostgreSQL. Small team, low budget, spiky traffic around big event launches.

- **Task 1 — Pick a host for the frontend, backend & database; name each**
- **Task 2 — IaaS, PaaS or serverless for the API? Justify it**
- **Task 3 — List the environment variables each service needs**
- **Task 4 — Which selection criteria drove your choices (cost, scale, ease…)?**

Marking lens (= LO4)

- Sensible managed platforms for a small team?
- Choice JUSTIFIED against criteria, not just named?
- Service model (PaaS/serverless) fits spiky traffic?
- Env vars cover API URL, DB URL, JWT secret, CORS?
- Trade-offs acknowledged (cost vs control vs scale)?

Activity: Diagnose the Deployment

Match each symptom to its likely cause & fix — 6 min in pairs

Symptom	Likely cause	Fix
Frontend loads; all API calls fail with CORS error	Backend doesn't allow the frontend origin	Add the frontend URL to ALLOWED_ORIGINS / CORS policy
Frontend works locally, blank/404 on refresh in prod	SPA routing not configured on the host	Add SPA fallback rewrite to index.html
Backend crashes on startup in the cloud	Missing env var (e.g. DATABASE_URL)	Set the env vars in the platform; read the logs
App can't reach the database	Wrong connection string / SSL / IP not allowed	Fix DATABASE_URL; enable SSL; allow-list / use pooled URL
Calls go to localhost in production	API base URL hardcoded, not from env	Use VITE_API_URL; rebuild & redeploy
First request after idle is very slow	Free-tier cold start (instance slept)	Expected on free tier; use paid/always-on or warmups

Golden rule of deployment debugging: READ THE LOGS first — they almost always name the real problem.

Quick Quiz — 6 Questions

1	DevOps is BEST described as…	A) a tool B) a culture of dev+ops collaboration C) a cloud
2	You manage code only; the platform runs servers & scaling. This is…	A) IaaS B) PaaS C) on-prem
3	Where should a JWT secret live in production?	A) in the code B) an env variable / secret store C) the README
4	Every API call from the deployed site fails with CORS. Fix on…	A) frontend host B) backend ALLOWED_ORIGINS C) the database
5	Which gives a managed PostgreSQL with backups?	A) Vercel B) Supabase / Neon / RDS C) ESLint
6	First request after idle is slow on a free tier because of…	A) a virus B) cold start C) too many env vars

Lecture Summary — Key Takeaways

DevOps is culture + automation

Dev & ops share responsibility; automate build, test, deploy & monitor; improve continuously.

Full-stack = 3 hosted services + config

Frontend, backend & database, wired by env vars (API URL, DB URL, secrets, CORS).

CI/CD ships safely & often

A pipeline takes one commit through to a live, repeatable deployment.

Config & secrets via env variables

Never hardcode; same code in dev & prod; managed DBs for backups & security.

Cloud models trade control for ease

IaaS → PaaS → serverless: less to manage as you move up. Pick per requirement.

Deploying isn't the finish line

Monitor, log, alert, patch & maintain — operations is continuous.

Learning Outcomes, Next Steps & Preparation

LO	Covered today by…
LO2	Applying cloud platforms & tools to deploy a full-stack app: frontend, backend API, managed database & environment configuration
LO3	Best practices for delivery & operations: CI/CD workflows, secure config, health checks, monitoring, logging & maintenance
LO4	Selecting & justifying a hosting stack and service model (IaaS/PaaS/serverless) for a project (case study + activities)



Next & beyond

Wrapping the module: integrate everything into the team full-stack project — built, collaborated on with Git/CI-CD, and now deployed & operated in the cloud.



Before the lab / project

- Create free accounts: Vercel/Netlify, Render/Railway, Supabase/Neon
- Deploy a managed PostgreSQL; copy its connection string
- Deploy your API; set env vars; verify /health responds
- Deploy your frontend with VITE_API_URL; fix CORS end-to-end

References & Further Reading

- **Platform documentation (primary sources)**
  - Vercel — vercel.com/docs · Netlify — docs.netlify.com · Firebase Hosting — firebase.google.com/docs/hosting
  - Render — render.com/docs · Railway — docs.railway.app · Heroku Dev Center — devcenter.heroku.com
  - AWS — docs.aws.amazon.com · Microsoft Azure — learn.microsoft.com/azure · Google Cloud — cloud.google.com/docs
  - Databases: Supabase — supabase.com/docs · Neon — neon.tech/docs
- **DevOps & configuration concepts**
  - The Twelve-Factor App — 12factor.net (config, env vars, deployment)
  - Docker documentation — docs.docker.com · DORA / Accelerate metrics — dora.dev
- **Recommended books**
  - Kim, Behr & Spafford (2018) The Phoenix Project. IT Revolution.
  - Kim et al. (2021) The DevOps Handbook, 2nd edn. · Forsgren, Humble & Kim (2018) Accelerate.